```
"use client";

import React, { useEffect, useState } from "react";
import { MoreVertical } from "lucide-react";
import Link from "next/link";
import { usePathname } from "next/navigation";

type AppliedJob = {
 id: string;
 title: string;
 company: string;
 description: string;
 salary: string;
 tags: string[];
 status: string;
};

const MyApplicationsApplied = () => {
 const pathname = usePathname();
 const [appliedJobs, setAppliedJobs] =
useState<AppliedJob[]>([]);
 const [loading, setLoading] = useState(true);

 const tabs = [
   { name: "Saved", path: "/saved", count: 0 },
   { name: "Applied", path: "/applied", count: 0 },
   { name: "Interviews", path: "/interviews", count: 0 },
   { name: "Archived", path: "/archived", count: 0 },
 ];

 useEffect(() => {
   const fetchApplied = async () => {
```

```
    try {
      const res = await fetch("/api/applications", { cache:
"no-store" });
      const data = await res.json();
      if (!res.ok) return;

      const mapped = (data || []).map((item: any) => ({
        id: item.id,
        title: item.jobTitle,
        company: item.company,
        description: item.description || "",
        salary:
          item.salaryMin || item.salaryMax
            ? `$${item.salaryMin ?? 0} - $${item.salaryMax ??
0}`
            : "Negotiable",
        tags: [item.status],
        status: item.status,
      }));

      setAppliedJobs(mapped.filter((job: AppliedJob) =>
job.status !== "interview" && job.status !== "withdrawn"));
    } catch (error) {
      console.error(error);
    } finally {
      setLoading(false);
    }
  };

  fetchApplied();
 }, []);
```

```
 return (
   <div className="min-h-screen bg-[#f8fafc] font-sans pb-12">
     <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between">
       <div className="flex items-center gap-2">
         <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
         <span className="text-xl font-bold
tracking-tight">NexHire</span>
       </div>

       <nav className="hidden md:flex items-center gap-8 text-sm
font-medium">
         <Link href="/home_page"><button
className="hover:text-gray-300
transition-colors">Home</button></Link>
         <Link href="/saved"><button className="text-[#40b594]
border-b-2 border-[#40b594] pb-1">My Jobs</button></Link>
         <Link href="/message"><button
className="hover:text-gray-300
transition-colors">Messages</button></Link>
         <Link href="/notification"><button
className="hover:text-gray-300
transition-colors">Notification</button></Link>
         <Link href="/setting"><button
className="hover:text-gray-300
transition-colors">Settings</button></Link>
       </nav>

       <Link href="/profile">
         <div className="flex items-center gap-4">
           <div className="text-right">
```

```
            <p className="text-[10px] text-gray-400 uppercase
tracking-wider">User name</p>
            <p className="text-sm font-bold">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#2d4f45] rounded-full
flex items-center justify-center font-bold text-white">U</div>
        </div>
      </Link>
    </header>

    <main className="max-w-5xl mx-auto p-12">
      <div className="mb-10">
        <h1 className="text-4xl font-extrabold text-[#1a1a1a]
mb-2">My Applications</h1>
        <p className="text-gray-400 font-medium">Track and
manage your job applications</p>
      </div>

      <div className="flex gap-10 border-b border-gray-400
mb-8">
        {tabs.map((tab) => {
          const isActive = pathname === tab.path;
          return (
            <Link key={tab.name} href={tab.path}>
              <button
                className={`pb-4 px-1 flex flex-col items-start
relative transition-all ${
                  isActive ? "text-[#153a30]" : "text-gray-400
hover:text-gray-600"
                }`}
              >
```

```
                  <span className="text-xs font-bold
mb-1">{tab.name === "Applied" ? appliedJobs.length : 0}</span>
                  <span className="text-lg
font-bold">{tab.name}</span>
                  {isActive && <div className="absolute bottom-0
left-0 w-full h-[3px] bg-[#153a30] rounded-full"></div>}
                </button>
              </Link>
            );
          })}
        </div>

        <div className="flex gap-8 mb-8 text-sm font-bold
text-gray-600">
          <span>Total: {appliedJobs.length}</span>
          <span>Accepted: {appliedJobs.filter((j) => j.status ===
"accepted").length}</span>
          <span>Rejected: {appliedJobs.filter((j) => j.status ===
"rejected").length}</span>
        </div>

        {loading ? (
          <div className="text-sm text-gray-500">Loading
applications...</div>
        ) : appliedJobs.length === 0 ? (
          <div className="text-sm text-gray-500">No applications
yet.</div>
        ) : (
          <div className="space-y-4">
            {appliedJobs.map((job) => (
              <div key={job.id} className="py-6 border-b
border-gray-400">
```

```
                <div className="flex justify-between items-start
mb-2">
                  <div>
                    <h2 className="text-xl font-extrabold
text-[#1a1a1a] mb-1">{job.title}</h2>
                    <p className="text-gray-400 text-sm font-bold
mb-4">{job.company}</p>
                  </div>
                  <div className="flex items-center gap-4">
                    <span className="bg-[#e4f6f1] text-[#153a30]
px-4 py-1.5 rounded-full font-bold text-xs border
border-[#40b594]/20">
                      {job.status}
                    </span>
                    <button className="text-gray-400
hover:text-black transition-colors">
                      <MoreVertical size={22} />
                    </button>
                  </div>
                </div>

                <p className="text-gray-400 text-sm mb-6
max-w-2xl">{job.description}</p>

                <div className="flex items-center gap-6">
                  <span className="text-lg font-extrabold
text-[#1a1a1a]">{job.salary}</span>
                  <div className="flex gap-2">
                    {job.tags.map((tag) => (
                      <span
                        key={tag}
```

```
                        className="bg-[#153a30] text-white
text-[10px] font-bold px-3 py-1.5 rounded-md uppercase
tracking-wider"
                      >
                        {tag}
                      </span>
                    ))}
                  </div>
                </div>
              </div>
            ))}
          </div>
        )}
      </main>
    </div>
  );
};

export default MyApplicationsApplied;
```

```
"use client";
import React from 'react';
import { Settings, Calendar, Ban, MoreVertical } from
'lucide-react';
import Link from 'next/link';
import { usePathname } from 'next/navigation';
const MyApplicationsArchived = () => {
const pathname = usePathname();

 const tabs = [
 { name: 'Saved', path: '/saved', count: 1 },
```

```
 { name: 'Applied', path: '/applied', count: 1 },
 { name: 'Interviews', path: '/interviews', count: 2 },
 { name: 'Archived', path: '/archived', count: 1 },
];
 const archivedJobs = [
   {
     id: 1,
     title: 'Full Stack Developer',
     company: 'Company name',
     archivedDate: 'Mar 30, 2026',
     salary: '$190k - $230k',
     tags: ['Part time', 'Remote', 'Senior'],
   }
 ];

 return (
   <div className="min-h-screen bg-[#f8fafc] font-sans pb-12">
     {/* HEADER NAVIGATION */}
     <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between">
       <div className="flex items-center gap-2">
         <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
         <span className="text-xl font-bold
tracking-tight">NexHire</span>
       </div>

        <nav className="hidden md:flex items-center gap-8
text-sm font-medium">
         <Link href="/home_page">
           <button className="hover:text-gray-300
transition-colors">
```

```
            Home
          </button>
        </Link>
        <Link href="/saved">
           <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">My Jobs</button>
         </Link>
         <Link href="/message">
        <button className="hover:text-gray-300
transition-colors">Messages</button>
        </Link>
        <Link href="/notification">
          <button className="hover:text-gray-300
transition-colors">Notification</button>
        </Link>
        <Link href="/setting">
        <button className="hover:text-gray-300
transition-colors">Settings</button>
        </Link>
      </nav>

      <Link href="/profile">
      <div className="flex items-center gap-4">
        <div className="text-right">
          <p className="text-[10px] text-gray-400 uppercase
tracking-wider">User name</p>
          <p className="text-sm font-bold">Profile</p>
        </div>
        <div className="w-10 h-10 bg-[#2d4f45] rounded-full
flex items-center justify-center font-bold text-white">U</div>
      </div>
    </Link>
```

```
      </header>

      <main className="max-w-5xl mx-auto p-12">
        <div className="mb-10">
          <h1 className="text-4xl font-extrabold text-[#1a1a1a]
mb-2">My Applications</h1>
          <p className="text-gray-400 font-medium">Track and
manage your job applications</p>
        </div>

        {/* TABS */}
         <div className="flex gap-10 border-b border-gray-400
mb-8">
 {tabs.map((tab) => {
   const isActive = pathname === tab.path;

   return (
     <Link key={tab.name} href={tab.path}>
       <button
         className={`pb-4 px-1 flex flex-col items-start
relative transition-all ${
           isActive ? 'text-[#153a30]' : 'text-gray-400
hover:text-gray-600'
         }`}
       >
         <span className="text-xs font-bold
mb-1">{tab.count}</span>
         <span className="text-lg font-bold">{tab.name}</span>

         {isActive && (
           <div className="absolute bottom-0 left-0 w-full
h-[3px] bg-[#153a30] rounded-full"></div>
```

```
          )}
        </button>
      </Link>
    );
  })}
</div>
        {/* ARCHIVED LISTINGS */}
        <div className="space-y-8">
          {archivedJobs.map((job) => (
            <div key={job.id} className="pb-8 border-b
border-gray-400 opacity-60">
              <div className="flex justify-between items-start
mb-2">
                <div>
                  <h2 className="text-2xl font-extrabold
text-gray-500 line-through mb-1">
                    {job.title}
                  </h2>
                  <p className="text-gray-400 text-sm font-bold
mb-4">{job.company}</p>

                  <div className="flex items-center gap-4
text-gray-400 text-[11px] font-bold">
                    <div className="flex items-center gap-1">
                      <Calendar size={14} />
                      Archived on {job.archivedDate}
                    </div>
                    <div className="flex items-center gap-1">
                      <Ban size={14} />
                      Inactive
                    </div>
                  </div>
```

```
                </div>

                <button className="text-gray-400 hover:text-black
transition-colors">
                  <MoreVertical size={24} />
                </button>
              </div>

              <div className="flex items-center gap-6 mt-4">
                <span className="text-xl font-extrabold
text-gray-400">{job.salary}</span>
                <div className="flex gap-2">
                  {job.tags.map((tag) => (
                    <span
                      key={tag}
                      className="bg-gray-500 text-white
text-[10px] font-bold px-3 py-1.5 rounded-md grayscale"
                    >
                      {tag}
                    </span>
                  ))}
                </div>
              </div>
            </div>
          ))}
        </div>
      </main>
    </div>
  );
};

export default MyApplicationsArchived;
```

```
"use client";
import React from 'react';
import { Calendar, Clock, Video, MapPin, ExternalLink,
MoreVertical } from 'lucide-react';
import Link from 'next/link';

const MyInterviews = () => {
 // Navigation tabs for the "My Jobs" section
 const tabs = [
   { name: 'Saved', path: '/saved', count: 1, active: false },
   { name: 'Applied', path: '/applied', count: 1, active: false
},
   { name: 'Interviews', path: '/interviews', count: 2, active:
true }, // Explicitly Active
   { name: 'Archived', path: '/archived', count: 1, active:
false },
 ];

 const interviewJobs = [
   {
     id: 1,
     title: 'Full Stack Developer',
     company: 'NexHire Solutions',
     date: 'Mar 30, 2026',
     time: '4:30 pm',
     location: 'Google Meet',
     isRemote: true,
     salary: '$190k - $230k',
     tags: ['Part time', 'Remote', 'Senior'],
   },
   {
     id: 2,
```

```
    title: 'Computer Science',
    company: 'Global Tech',
    date: 'April 02, 2026',
    time: '10:00 am',
    location: '128 Kampuchea Krom, TK',
    isRemote: false,
    salary: '$150k - $200k',
    tags: ['Full time', 'On-site', 'Mid-level'],
  }
];

return (
  <div className="min-h-screen bg-[#f8fafc] font-sans pb-12">
    {/* HEADER NAVIGATION */}
    <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between">
      <div className="flex items-center gap-2">
        <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
        <span className="text-xl font-bold
tracking-tight">NexHire</span>
      </div>

      <nav className="hidden md:flex items-center gap-8 text-sm
font-medium">
        <Link href="/home_page"><button
className="hover:text-gray-300">Home</button></Link>
        <Link href="/saved"><button className="text-[#40b594]
border-b-2 border-[#40b594] pb-1">My Jobs</button></Link>
        <Link href="/message"><button
className="hover:text-gray-300">Messages</button></Link>
```

```
          <Link href="/notification"><button
className="hover:text-gray-300">Notification</button></Link>
          <Link href="/setting"><button
className="hover:text-gray-300">Settings</button></Link>
        </nav>

        <Link href="/profile">
        <div className="flex items-center gap-4">
          <div className="text-right">
            <p className="text-[10px] text-gray-400 uppercase
tracking-wider">User name</p>
            <p className="text-sm font-bold">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#2d4f45] rounded-full
flex items-center justify-center font-bold text-white">U</div>
        </div>
      </Link>
      </header>

      <main className="max-w-5xl mx-auto p-12">
        <div className="mb-10">
          <h1 className="text-4xl font-extrabold text-[#1a1a1a]
mb-2">My Applications</h1>
          <p className="text-gray-400 font-medium">Manage your
upcoming interviews and meetings</p>
        </div>

        {/* TABS - Focused on Interviews */}
        <div className="flex gap-10 border-b border-gray-100
mb-10">
          {tabs.map((tab) => (
            <Link key={tab.name} href={tab.path}>
```

```
            <button
              className={`pb-4 px-1 flex flex-col items-start
relative transition-all ${
                tab.active ? 'text-[#153a30]' : 'text-gray-300
hover:text-gray-500'
              }`}
            >
              <span className="text-xs font-bold
mb-1">{tab.count}</span>
              <span className="text-lg
font-bold">{tab.name}</span>
              {tab.active && (
                <div className="absolute bottom-0 left-0 w-full
h-[3.5px] bg-[#153a30] rounded-full"></div>
              )}
            </button>
          </Link>
        ))}
      </div>

      {/* INTERVIEW LISTINGS */}
      <div className="space-y-12">
        {interviewJobs.map((job) => (
          <div key={job.id} className="pb-10 border-b
border-gray-100 last:border-0">
            <div className="flex justify-between items-start">
              <div>
                <h2 className="text-2xl font-black
text-[#1a1a1a] mb-1">{job.title}</h2>
                <p className="text-gray-400 text-sm font-bold
mb-6 uppercase tracking-tight">{job.company}</p>
```

```
                {/* INTERVIEW TIME & LOCATION CARD */}
                <div className="flex flex-wrap items-center
gap-6 text-gray-600 text-xs font-bold">
                  <div className="flex items-center gap-2
bg-white px-4 py-2 rounded-xl shadow-sm border border-gray-50">
                    <Calendar size={16}
className="text-[#40b594]" />
                    {job.date}
                  </div>
                  <div className="flex items-center gap-2
bg-white px-4 py-2 rounded-xl shadow-sm border border-gray-50">
                    <Clock size={16} className="text-[#40b594]"
/>
                    {job.time}
                  </div>
                  <div className="flex items-center gap-2
bg-white px-4 py-2 rounded-xl shadow-sm border border-gray-50">
                    {job.isRemote ? (
                      <Video size={16}
className="text-[#40b594]" />
                    ) : (
                      <MapPin size={16}
className="text-[#40b594]" />
                    )}
                    {job.location}
                  </div>
                </div>
              </div>

              <div className="flex items-center gap-3">
                <button className="bg-[#153a30] text-white px-6
py-3 rounded-2xl font-bold text-sm hover:bg-[#0d2a23]
```

```
transition-all flex items-center gap-2 shadow-lg
shadow-[#153a30]/20">
                  {job.isRemote ? 'Join Call' : 'View Map'}
                  <ExternalLink size={16} />
                </button>
                <button className="text-gray-300
hover:text-black transition-colors">
                  <MoreVertical size={24} />
                </button>
              </div>
            </div>

            <div className="flex items-center gap-6 mt-8">
              <span className="text-2xl font-black
text-[#153a30]">{job.salary}</span>
              <div className="flex gap-2">
                {job.tags.map((tag) => (
                  <span
                    key={tag}
                    className="bg-[#f1fcf9] text-[#153a30]
text-[10px] font-black px-4 py-2 rounded-lg border
border-[#e4f6f1]"
                  >
                    {tag.toUpperCase()}
                  </span>
                ))}
              </div>
            </div>
          </div>
        ))}
      </div>
    </main>
```

```
    </div>
  );
};

export default MyInterviews;
```

```
"use client";

import React, { useEffect, useState } from "react";
import { Bookmark, MoreVertical } from "lucide-react";
import Link from "next/link";
import { usePathname } from "next/navigation";

type SavedJob = {
  id: string;
  title: string;
  company: string;
  description: string;
  salary: string;
  tags: string[];
};

const MyApplications = () => {
  const pathname = usePathname();
  const [savedJobs, setSavedJobs] = useState<SavedJob[]>([]);
  const [loading, setLoading] = useState(true);

  const tabs = [
    { name: "Saved", path: "/saved", count: 0 },
    { name: "Applied", path: "/applied", count: 0 },
    { name: "Interviews", path: "/interviews", count: 0 },
    { name: "Archived", path: "/archived", count: 0 },
```

```
 ];

 useEffect(() => {
   const fetchSaved = async () => {
     try {
       const res = await fetch("/api/saved-jobs", { cache:
"no-store" });
       const data = await res.json();
       if (!res.ok) return;

       const mapped = (data || []).map((item: any) => ({
         id: item.jobId,
         title: item.title,
         company: item.company,
         description: item.description,
         salary: item.salary,
         tags: [item.type, item.arrangement,
item.experience].filter(Boolean),
       }));

       setSavedJobs(mapped);
     } catch (error) {
       console.error(error);
     } finally {
       setLoading(false);
     }
   };

   fetchSaved();
 }, []);

 return (
```

```
   <div className="min-h-screen bg-[#f8fafc] font-sans pb-12">
     <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between">
       <div className="flex items-center gap-2">
         <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
         <span className="text-xl font-bold
tracking-tight">NexHire</span>
       </div>

       <nav className="hidden md:flex items-center gap-8 text-sm
font-medium">
         <Link href="/home_page"><button
className="hover:text-gray-300
transition-colors">Home</button></Link>
         <Link href="/saved"><button className="text-[#40b594]
border-b-2 border-[#40b594] pb-1">My Jobs</button></Link>
         <Link href="/message"><button
className="hover:text-gray-300
transition-colors">Messages</button></Link>
         <Link href="/notification"><button
className="hover:text-gray-300
transition-colors">Notification</button></Link>
         <Link href="/setting"><button
className="hover:text-gray-300
transition-colors">Settings</button></Link>
       </nav>

       <Link href="/profile">
         <div className="flex items-center gap-4">
           <div className="text-right">
```

```
            <p className="text-[10px] text-gray-400 uppercase
tracking-wider">User name</p>
            <p className="text-sm font-bold">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#2d4f45] rounded-full
flex items-center justify-center font-bold text-white">U</div>
        </div>
      </Link>
    </header>

    <main className="max-w-5xl mx-auto p-12">
      <div className="mb-10">
        <h1 className="text-4xl font-extrabold text-[#1a1a1a]
mb-2">My Applications</h1>
        <p className="text-gray-400 font-medium">Track and
manage your job applications</p>
      </div>

      <div className="flex gap-10 border-b border-gray-400
mb-8">
        {tabs.map((tab) => {
          const isActive = pathname === tab.path;
          return (
            <Link key={tab.name} href={tab.path}>
              <button
                className={`pb-4 px-1 flex flex-col items-start
relative transition-all ${
                  isActive ? "text-[#153a30]" : "text-gray-400
hover:text-gray-600"
                }`}
              >
```

```
                  <span className="text-xs font-bold
mb-1">{tab.name === "Saved" ? savedJobs.length : 0}</span>
                  <span className="text-lg
font-bold">{tab.name}</span>
                  {isActive && <div className="absolute bottom-0
left-0 w-full h-[3px] bg-[#153a30] rounded-full"></div>}
                </button>
              </Link>
            );
          })}
        </div>

        {loading ? (
          <div className="text-sm text-gray-500">Loading saved
jobs...</div>
        ) : savedJobs.length === 0 ? (
          <div className="text-sm text-gray-500">No saved jobs
yet.</div>
        ) : (
          <div className="space-y-4">
            {savedJobs.map((job) => (
              <div key={job.id} className="py-6 border-b
border-gray-400 group">
                <div className="flex justify-between items-start
mb-2">
                  <div>
                    <h2 className="text-xl font-extrabold
text-[#1a1a1a] mb-1">{job.title}</h2>
                    <p className="text-gray-400 text-sm font-bold
mb-4">{job.company}</p>
                  </div>
                  <div className="flex items-center gap-3">
```

```
                    <Link href={`/jobs/${job.id}/apply`}>
                      <button className="bg-[#153a30] text-white
px-6 py-1.5 rounded-lg font-bold text-sm hover:bg-[#0d2a23]
transition-colors">
                        Apply
                      </button>
                    </Link>
                    <button className="text-black">
                      <Bookmark size={22} fill="currentColor" />
                    </button>
                    <button className="text-gray-400
hover:text-black">
                      <MoreVertical size={22} />
                    </button>
                  </div>
                </div>

                <p className="text-gray-400 text-sm mb-6
max-w-2xl">{job.description}</p>

                <div className="flex items-center gap-6">
                  <span className="text-lg font-extrabold
text-[#1a1a1a]">{job.salary}</span>
                  <div className="flex gap-2">
                    {job.tags.map((tag) => (
                      <span
                        key={tag}
                        className="bg-[#153a30] text-white
text-[10px] font-bold px-3 py-1.5 rounded-md uppercase
tracking-wider"
                      >
                        {tag}
```

```
                    </span>
                  ))}
                </div>
              </div>
            </div>
          ))}
        </div>
      )}
    </main>
  </div>
);
};

export default MyApplications;
```

```
"use client";

import React, { useState, useEffect, useRef } from "react";
import {
 Search,
 ChevronDown,
 Bookmark,
 BookmarkCheck,
 X,
 MapPin,
 Clock,
 DollarSign,
 SlidersHorizontal,
} from "lucide-react";
import Link from "next/link";
import UserNavProfile from "@/components/ui/UserNavProfile";
```

```
const jobTypeOptions = ["Full-time", "Part-time", "Contract",
"Freelance", "Internship"];
const experienceOptions = ["Entry Level", "Mid Level", "Senior",
"Lead / Manager", "Executive"];
const salaryOptions = ["$50k - $100k", "$100k - $150k", "$150k -
$200k", "$200k+"];

const tagColors: Record<string, string> = {
 "Full-time": "bg-emerald-50 text-emerald-700
border-emerald-100",
 "Part-time": "bg-blue-50 text-blue-700 border-blue-100",
 Contract: "bg-orange-50 text-orange-700 border-orange-100",
 Freelance: "bg-pink-50 text-pink-700 border-pink-100",
 Internship: "bg-yellow-50 text-yellow-700 border-yellow-100",
 Remote: "bg-violet-50 text-violet-700 border-violet-100",
 "On-site": "bg-gray-50 text-gray-600 border-gray-200",
 Hybrid: "bg-teal-50 text-teal-700 border-teal-100",
 Senior: "bg-amber-50 text-amber-700 border-amber-100",
 "Mid Level": "bg-sky-50 text-sky-700 border-sky-100",
 "Entry Level": "bg-rose-50 text-rose-700 border-rose-100",
 Executive: "bg-indigo-50 text-indigo-700 border-indigo-100",
 "Lead / Manager": "bg-purple-50 text-purple-700
border-purple-100",
};

const JobSeekerHome = () => {
 const [searchQuery, setSearchQuery] = useState("");
 const [showFilters, setShowFilters] = useState(false);
 const [savedJobs, setSavedJobs] = useState<string[]>([]);
 const [sortBy, setSortBy] = useState("Most Relevant");
```

```
 const [activeDropdown, setActiveDropdown] = useState<string |
null>(null);

 const [filters, setFilters] = useState({
   jobType: "",
   experience: "",
   salary: "",
   arrangement: "",
 });

 const dropdownRef = useRef<HTMLDivElement>(null);

 useEffect(() => {
   const handleClickOutside = (e: MouseEvent) => {
     if (dropdownRef.current &&
!dropdownRef.current.contains(e.target as Node)) {
       setActiveDropdown(null);
     }
   };
   document.addEventListener("mousedown", handleClickOutside);
   return () => document.removeEventListener("mousedown",
handleClickOutside);
 }, []);

 const clearFilters = () =>
   setFilters({ jobType: "", experience: "", salary: "",
arrangement: "" });

 const activeFilterCount =
Object.values(filters).filter(Boolean).length;
```

```
 const pillFilters: { key: keyof typeof filters; label: string;
options: string[] }[] = [
   { key: "jobType", label: "Job Type", options: jobTypeOptions
},
   { key: "arrangement", label: "Work Mode", options:
["On-site", "Remote", "Hybrid"] },
   { key: "experience", label: "Experience", options:
experienceOptions },
   { key: "salary", label: "Salary", options: salaryOptions },
 ];

 return (
   <div className="min-h-screen font-sans pb-16" style={{
background: "#f0f4f3" }}>
     <header className="flex items-center justify-between
bg-[#051612] px-8 py-4 text-white">
       <div className="flex items-center gap-2">
         <img src="/logo.png" alt="NexHire" className="h-8 w-8"
/>
         <span className="text-xl font-bold
tracking-tight">NexHire</span>
       </div>

       <nav className="hidden items-center gap-8 text-sm
font-medium md:flex">
         <Link href="/home_page">
           <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Home</button>
         </Link>
         <Link href="/saved">
           <button className="transition-colors
hover:text-gray-300">My Jobs</button>
```

```
          </Link>
          <Link href="/message">
            <button className="transition-colors
hover:text-gray-300">Messages</button>
          </Link>
          <Link href="/notification">
            <button className="transition-colors
hover:text-gray-300">Notification</button>
          </Link>
          <Link href="/setting">
            <button className="transition-colors
hover:text-gray-300">Settings</button>
          </Link>
        </nav>

        <UserNavProfile />
      </header>

      <section className="bg-[#051612] text-white pt-16 pb-28
px-8">
        <div className="max-w-4xl mx-auto">
          <h1 className="text-5xl font-extrabold mb-3
leading-tight tracking-tight">
            Your Next Great Role
          </h1>
          <p className="text-gray-400 text-base mb-10 max-w-md
leading-relaxed font-medium">
            Discover opportunities matched to your skills,
experience, and career goals.
          </p>

          <div className="flex gap-3 mb-5">
```

```
          <div className="flex-1 relative">
            <Search className="absolute left-4 top-1/2
-translate-y-1/2 text-gray-400" size={18} />
            <input
              type="text"
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
              placeholder="Job title, company, or skill..."
              className="w-full bg-white text-[#071a15] py-3.5
pl-11 pr-4 rounded-xl text-sm font-medium focus:outline-none
focus:ring-2 focus:ring-[#40b594]/40 placeholder-gray-400"
            />
          </div>
          <button className="bg-[#40b594] text-[#051612] px-8
py-3.5 rounded-xl font-extrabold text-sm hover:bg-[#34a382]
transition-all">
            Search
          </button>
        </div>

        <div className="flex flex-wrap items-center gap-2.5"
ref={dropdownRef}>
          <button
            onClick={() => setShowFilters(!showFilters)}
            className={`flex items-center gap-1.5 px-4 py-2
rounded-lg text-xs font-bold border transition-all ${
              showFilters
                ? "bg-[#40b594] border-[#40b594]
text-[#051612]"
                : "bg-white/5 border-white/10 text-gray-300
hover:bg-white/10"
            }`}
```

```
          >
            <SlidersHorizontal size={12} />
            Filters
            {activeFilterCount > 0 && (
              <span className="bg-[#051612] text-[#40b594]
rounded-full w-4 h-4 flex items-center justify-center
text-[10px] font-extrabold">
                {activeFilterCount}
              </span>
            )}
          </button>

          {showFilters &&
            pillFilters.map(({ key, label, options }) => (
              <div key={key} className="relative">
                <button
                  onClick={() =>
setActiveDropdown(activeDropdown === key ? null : key)}
                  className={`flex items-center gap-1.5 px-4
py-2 rounded-lg text-xs font-bold border transition-all ${
                    filters[key]
                      ? "bg-[#40b594] border-[#40b594]
text-[#051612]"
                      : "bg-white/5 border-white/10
text-gray-300 hover:bg-white/10"
                  }`}
                >
                  {filters[key] || label}
                  {filters[key] ? (
                    <X
                      size={11}
                      onClick={(e) => {
```

```
                          e.stopPropagation();
                          setFilters((p) => ({ ...p, [key]: ""
}));
                        }}
                      />
                    ) : (
                      <ChevronDown
                        size={11}
                        className={`transition-transform
${activeDropdown === key ? "rotate-180" : ""}`}
                      />
                    )}
                  </button>

                  {activeDropdown === key && (
                    <div className="absolute left-0 top-full mt-2
w-52 bg-[#0d2219] border border-white/10 rounded-xl shadow-2xl
z-50 overflow-hidden">
                      <div className="py-1.5">
                        {options.map((opt) => (
                          <button
                            key={opt}
                            onClick={() => {
                              setFilters((p) => ({ ...p, [key]:
opt }));
                              setActiveDropdown(null);
                            }}
                            className={`w-full text-left px-4
py-2.5 text-xs font-semibold transition-colors ${
                              filters[key] === opt
                                ? "bg-[#40b594]/20
text-[#40b594]"
```

```
                                : "text-gray-300 hover:bg-white/5
hover:text-white"
                            }`}
                          >
                            {opt}
                          </button>
                        ))}
                      </div>
                    </div>
                  )}
                </div>
              ))}

            {activeFilterCount > 0 && (
              <button
                onClick={clearFilters}
                className="text-xs font-bold text-gray-500
hover:text-gray-300 transition-colors px-1"
              >
                Clear all
              </button>
            )}
          </div>
        </div>
      </section>
    </div>
  );
};

export default JobSeekerHome;
```

```
"use client";

import React, { useEffect, useMemo, useRef, useState } from
"react";
import {
 Search,
 Paperclip,
 Send,
 Pencil,
 Trash2,
 X,
} from "lucide-react";
import Link from "next/link";

type Conversation = {
 id: string;
 employerId: string;
 jobSeekerId: string;
 jobId: string | null;
 lastMessageAt: string;
 employerName: string;
 employerImage: string | null;
 seekerFirstName: string;
 seekerLastName: string;
 seekerImage: string | null;
};

type Message = {
 id: string;
 conversationId: string;
 senderId: string;
 text: string | null;
```

```
 attachmentUrl: string | null;
 attachmentName: string | null;
 attachmentType: string | null;
 editedAt: string | null;
 createdAt: string;
};

type Me = {
 userId: string;
 role: "employer" | "job_seeker";
 email: string;
};

async function safeFetchJson(url: string, options?: RequestInit)
{
 const res = await fetch(url, options);
 const text = await res.text();

 let data: any = null;

 try {
   data = text ? JSON.parse(text) : null;
 } catch {
   console.error(`Non-JSON response from ${url}:`, text);
   throw new Error(`Invalid JSON response from ${url}`);
 }

 if (!res.ok) {
   throw new Error(data?.error || `Request failed:
${res.status}`);
 }
```

```
 return data;
}

function formatTime(dateString: string) {
 return new Date(dateString).toLocaleTimeString("en-US", {
   hour: "numeric",
   minute: "2-digit",
 });
}

function formatConversationTime(dateString: string) {
 const date = new Date(dateString);
 const now = new Date();
 const sameDay = date.toDateString() === now.toDateString();

 if (sameDay) return formatTime(dateString);

 const yesterday = new Date();
 yesterday.setDate(now.getDate() - 1);

 if (date.toDateString() === yesterday.toDateString()) return
"Yesterday";

 return date.toLocaleDateString("en-US", {
   month: "short",
   day: "2-digit",
 });
}

function canEdit(createdAt: string) {
 return Date.now() - new Date(createdAt).getTime() <= 15 * 60 *
1000;
```

```
}

export default function Messages() {
 const [me, setMe] = useState<Me | null>(null);
 const [conversations, setConversations] =
useState<Conversation[]>([]);
 const [activeChat, setActiveChat] = useState<string>("");
 const [messages, setMessages] = useState<Message[]>([]);
 const [message, setMessage] = useState("");
 const [search, setSearch] = useState("");
 const [loadingChats, setLoadingChats] = useState(true);
 const [loadingMessages, setLoadingMessages] = useState(false);
 const [sending, setSending] = useState(false);
 const [editingId, setEditingId] = useState<string |
null>(null);
 const [editingText, setEditingText] = useState("");
 const [attachment, setAttachment] = useState<File |
null>(null);

 const attachmentInputRef = useRef<HTMLInputElement |
null>(null);

 useEffect(() => {
   const loadMe = async () => {
     try {
       const data = await safeFetchJson("/api/auth/me");
       setMe(data);
     } catch (error) {
       console.error("LOAD ME ERROR:", error);
     }
   };
```

```
  loadMe();
}, []);

const fetchConversations = async () => {
  try {
    setLoadingChats(true);
    const data = await safeFetchJson("/api/conversations");
    setConversations(data || []);
    if (!activeChat && data?.length) {
      setActiveChat(data[0].id);
    }
  } catch (error) {
    console.error("FETCH CONVERSATIONS ERROR:", error);
    setConversations([]);
  } finally {
    setLoadingChats(false);
  }
};

useEffect(() => {
  fetchConversations();
}, []);

const fetchMessages = async (conversationId: string) => {
  try {
    setLoadingMessages(true);
    const data = await
safeFetchJson(`/api/conversations/${conversationId}/messages`);
    setMessages(data || []);
  } catch (error) {
    console.error("FETCH MESSAGES ERROR:", error);
    setMessages([]);
```

```
  } finally {
    setLoadingMessages(false);
  }
};

useEffect(() => {
  if (activeChat) {
    fetchMessages(activeChat);
  }
}, [activeChat]);

const filteredConversations = useMemo(() => {
  const q = search.trim().toLowerCase();
  if (!q) return conversations;

  return conversations.filter((conversation) =>
    (conversation.employerName || "").toLowerCase().includes(q)
  );
}, [conversations, search]);

const activeConversation = conversations.find((c) => c.id ===
activeChat);

const sendMessage = async () => {
  if (!activeChat || (!message.trim() && !attachment)) return;

  try {
    setSending(true);

    const formData = new FormData();
    formData.append("text", message.trim());
    if (attachment) {
```

```
        formData.append("attachment", attachment);
      }

      await
safeFetchJson(`/api/conversations/${activeChat}/messages`, {
        method: "POST",
        body: formData,
      });

      setMessage("");
      setAttachment(null);
      if (attachmentInputRef.current)
attachmentInputRef.current.value = "";

      await fetchMessages(activeChat);
      await fetchConversations();
    } catch (error: any) {
      console.error("SEND MESSAGE ERROR:", error);
      alert(error.message || "Failed to send message");
    } finally {
      setSending(false);
    }
  };

  const saveEdit = async () => {
    if (!editingId) return;

    try {
      await safeFetchJson(`/api/messages/${editingId}`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text: editingText }),
```

```
    });

    setEditingId(null);
    setEditingText("");
    await fetchMessages(activeChat);
  } catch (error: any) {
    console.error("EDIT MESSAGE ERROR:", error);
    alert(error.message || "Failed to edit message");
  }
};

const deleteForMe = async (messageId: string) => {
  try {
    await safeFetchJson(`/api/messages/${messageId}`, {
      method: "DELETE",
    });

    await fetchMessages(activeChat);
  } catch (error: any) {
    console.error("DELETE MESSAGE ERROR:", error);
    alert(error.message || "Failed to delete message");
  }
};

return (
  <div className="min-h-screen bg-[#f0f4f3] font-sans">
    <header className="flex items-center justify-between
bg-[#051612] px-8 py-4 text-white">
      <div className="flex items-center gap-2">
        <img src="/logo.png" alt="NexHire" className="h-8 w-8"
/>
```

```
        <span className="text-xl font-bold
tracking-tight">NexHire</span>
      </div>

      <nav className="hidden items-center gap-8 text-sm
font-medium md:flex">
        <Link href="/home_page"><button
className="transition-colors
hover:text-gray-300">Home</button></Link>
        <Link href="/saved"><button
className="transition-colors hover:text-gray-300">My
Jobs</button></Link>
        <Link href="/message"><button className="border-b-2
border-[#40b594] pb-1 text-[#40b594]">Messages</button></Link>
        <Link href="/notification"><button
className="transition-colors
hover:text-gray-300">Notification</button></Link>
        <Link href="/setting"><button
className="transition-colors
hover:text-gray-300">Settings</button></Link>
      </nav>

      <Link href="/profile">
        <div className="flex items-center gap-4">
          <div className="text-right">
            <p className="text-[10px] uppercase tracking-wider
text-gray-400">User name</p>
            <p className="text-sm font-bold">Profile</p>
          </div>
          <div className="flex h-10 w-10 items-center
justify-center rounded-full bg-[#2d4f45] font-bold text-white">
            U
```

```
          </div>
        </div>
      </Link>
    </header>

    <main className="mx-auto max-w-7xl px-6 py-10 md:px-10">
      <div className="mb-8">
        <p className="mb-1 text-xs font-bold uppercase
tracking-widest text-[#40b594]">Inbox</p>
        <h1 className="text-4xl font-extrabold
text-[#071a15]">Messages</h1>
        <p className="mt-1 font-medium text-[#4a5a55]">
          Communicate with employers directly
        </p>
      </div>

      <div className="flex h-[680px] overflow-hidden
rounded-2xl border border-gray-100 bg-white shadow-sm">
        <div className="flex w-80 flex-shrink-0 flex-col
border-r border-gray-100">
          <div className="border-b border-gray-100 p-5">
            <div className="relative">
              <Search className="absolute left-3.5 top-1/2
-translate-y-1/2 text-[#6b7f79]" size={16} />
              <input
                type="text"
                placeholder="Search conversations..."
                value={search}
                onChange={(e) => setSearch(e.target.value)}
                className="w-full rounded-xl bg-[#f0f4f3]
py-2.5 pl-10 pr-4 text-sm font-medium text-[#071a15]
```

```
placeholder-[#6b7f79] focus:outline-none focus:ring-2
focus:ring-[#40b594]/30"
               />
             </div>
           </div>

           <div className="px-5 py-3">
             <p className="text-[10px] font-bold uppercase
tracking-widest text-[#6b7f79]">
               Recent
             </p>
           </div>

           <div className="flex-1 overflow-y-auto">
             {loadingChats ? (
               <div className="px-5 py-4 text-sm
text-[#6b7f79]">Loading conversations...</div>
             ) : filteredConversations.length === 0 ? (
               <div className="px-5 py-4 text-sm
text-[#6b7f79]">No conversations yet.</div>
             ) : (
               filteredConversations.map((chat) => {
                 const isActive = activeChat === chat.id;

                 return (
                   <div
                     key={chat.id}
                     onClick={() => setActiveChat(chat.id)}
                     className={`cursor-pointer border-l-4 px-5
py-4 transition-all ${
                       isActive
                         ? "border-l-[#40b594] bg-[#f0f9f6]"
```

```
                        : "border-l-transparent
hover:bg-[#f8faf9]"
                    }`}
                  >
                    <div className="flex items-center gap-4">
                      <div className="relative flex-shrink-0">
                        <div className="flex h-12 w-12
items-center justify-center rounded-full bg-[#051612] text-base
font-extrabold text-white">
                          {(chat.employerName ||
"E").charAt(0)}
                        </div>
                        <span className="absolute bottom-0
right-0 h-3 w-3 rounded-full border-2 border-white bg-[#40b594]"
/>
                      </div>

                      <div className="min-w-0 flex-1">
                        <div className="mb-0.5 flex
items-center justify-between">
                          <h3 className={`truncate text-sm
font-extrabold ${isActive ? "text-[#071a15]" :
"text-[#1a2e29]"}`}>
                            {chat.employerName}
                          </h3>
                          <span className="ml-2 flex-shrink-0
text-[10px] font-semibold text-[#6b7f79]">

{formatConversationTime(chat.lastMessageAt)}
                          </span>
                        </div>
```

```
                        <p className="truncate text-xs
font-bold tracking-wide text-[#40b594]">
                          Employer
                        </p>

                        <p className="mt-1 text-[11px]
font-semibold text-[#7a8b86]">
                          Active
                        </p>
                      </div>
                    </div>
                  </div>
                );
              })
            )}
          </div>
        </div>

        <div className="flex min-w-0 flex-1 flex-col">
          <div className="border-b border-gray-100 bg-white
px-7 py-5">
            {activeConversation ? (
              <div className="flex items-center gap-4">
                <div className="relative flex-shrink-0">
                  <div className="flex h-12 w-12 items-center
justify-center rounded-full bg-[#051612] text-sm font-extrabold
text-white">
                    {activeConversation.employerName.charAt(0)}
                  </div>
                  <span className="absolute bottom-0 right-0
h-3 w-3 rounded-full border-2 border-white bg-[#40b594]" />
                </div>
```

```
                <div className="min-w-0">
                  <h3 className="truncate text-base
font-extrabold text-[#071a15]">
                    {activeConversation.employerName}
                  </h3>
                  <p className="truncate text-sm font-medium
text-[#4f655f]">Employer</p>
                  <p className="mt-1 text-[11px] font-bold
uppercase tracking-wider text-[#40b594]">
                    Active
                  </p>
                </div>
              </div>
            ) : (
              <p className="text-sm text-[#6b7f79]">Select a
conversation</p>
            )}
          </div>

          <div className="flex-1 space-y-6 overflow-y-auto
bg-[#f8faf9] px-7 py-6">
            {loadingMessages ? (
              <div className="text-sm text-[#6b7f79]">Loading
messages...</div>
            ) : messages.length === 0 ? (
              <div className="text-sm text-[#6b7f79]">No
messages yet.</div>
            ) : (
              <>
                <div className="flex items-center gap-4">
                  <div className="h-px flex-1 bg-gray-200" />
```

```
                  <span className="text-[10px] font-bold
uppercase tracking-widest text-[#6b7f79]">
                    Conversation
                  </span>
                  <div className="h-px flex-1 bg-gray-200" />
                </div>

                {messages.map((msg) => {
                  const mine = me?.userId === msg.senderId;

                  if (editingId === msg.id) {
                    return (
                      <div key={msg.id} className="flex
flex-col items-end gap-2">
                        <div className="w-full max-w-md
rounded-2xl border border-gray-200 bg-white p-4 shadow-sm">
                          <textarea
                            value={editingText}
                            onChange={(e) =>
setEditingText(e.target.value)}
                            rows={3}
                            className="w-full resize-none
bg-transparent text-sm text-[#071a15] outline-none"
                          />
                          <div className="mt-3 flex justify-end
gap-2">
                            <button
                              onClick={() => {
                                setEditingId(null);
                                setEditingText("");
                              }}
```

```
                              className="rounded-xl border
border-gray-200 px-3 py-2 text-xs font-bold text-[#6b7f79]"
                            >
                              Cancel
                            </button>
                            <button
                              onClick={saveEdit}
                              className="rounded-xl
bg-[#051612] px-3 py-2 text-xs font-bold text-white"
                            >
                              Save
                            </button>
                          </div>
                        </div>
                      </div>
                    );
                  }

                  return mine ? (
                    <div key={msg.id} className="flex flex-col
items-end gap-1">
                      <div className="max-w-sm rounded-2xl
rounded-br-md border border-gray-200 bg-white px-5 py-3.5
shadow-sm">
                        {msg.text && (
                          <p className="text-sm leading-relaxed
text-[#071a15]">{msg.text}</p>
                        )}
                        {msg.attachmentUrl && (
                          <a
                            href={msg.attachmentUrl}
                            target="_blank"
```

```
                              rel="noreferrer"
                              className="mt-2 block text-xs
font-bold text-[#40b594] underline"
                            >
                              {msg.attachmentName || "Open
attachment"}
                            </a>
                          )}
                          {msg.editedAt && (
                            <p className="mt-2 text-[10px]
font-semibold text-[#6b7f79]">Edited</p>
                          )}
                        </div>
                        <div className="mr-1 flex items-center
gap-2">
                          <span className="text-[10px]
font-semibold text-[#6b7f79]">
                            {formatTime(msg.createdAt)}
                          </span>
                          {canEdit(msg.createdAt) && (
                            <button
                              onClick={() => {
                                setEditingId(msg.id);
                                setEditingText(msg.text || "");
                              }}
                              className="text-[#6b7f79]
hover:text-[#071a15]"
                            >
                              <Pencil size={12} />
                            </button>
                          )}
                          <button
```

```
                          onClick={() => deleteForMe(msg.id)}
                          className="text-[#6b7f79]
hover:text-red-500"
                        >
                          <Trash2 size={12} />
                        </button>
                      </div>
                    </div>
                  ) : (
                    <div key={msg.id} className="flex items-end
gap-3">
                      <div className="flex h-8 w-8
flex-shrink-0 items-center justify-center rounded-full
bg-[#051612] text-xs font-extrabold text-white">

{activeConversation?.employerName.charAt(0)}
                      </div>
                      <div className="flex flex-col items-start
gap-1">
                        <div className="max-w-sm rounded-2xl
rounded-bl-md bg-[#051612] px-5 py-3.5 text-white shadow-sm">
                          {msg.text && <p className="text-sm
leading-relaxed">{msg.text}</p>}
                          {msg.attachmentUrl && (
                            <a
                              href={msg.attachmentUrl}
                              target="_blank"
                              rel="noreferrer"
                              className="mt-2 block text-xs
font-bold text-[#40b594] underline"
                            >
```

```
                                {msg.attachmentName || "Open
attachment"}
                                </a>
                              )}
                            </div>
                            <span className="ml-1 text-[10px]
font-semibold text-[#6b7f79]">
                              {formatTime(msg.createdAt)}
                            </span>
                          </div>
                        </div>
                      );
                    })}
                  </>
                )}
              </div>

              <div className="border-t border-gray-100 bg-white
px-7 py-5">
                {attachment && (
                  <div className="mb-3 flex items-center
justify-between rounded-xl border border-gray-200 bg-[#f8faf9]
px-4 py-2">
                    <p className="truncate text-xs font-semibold
text-[#071a15]">
                      {attachment.name}
                    </p>
                    <button
                      onClick={() => {
                        setAttachment(null);
                        if (attachmentInputRef.current)
attachmentInputRef.current.value = "";
```

```
                }}
                className="text-[#6b7f79] hover:text-red-500"
              >
                <X size={14} />
              </button>
            </div>
          )}

          <div className="flex items-center gap-3 rounded-2xl
border border-gray-200 bg-[#f0f4f3] px-4 py-3 transition-all
focus-within:border-[#40b594] focus-within:ring-2
focus-within:ring-[#40b594]/20">
            <button
              className="flex-shrink-0 text-[#6b7f79]
transition-colors hover:text-[#071a15]"
              onClick={() =>
attachmentInputRef.current?.click()}
            >
              <Paperclip size={19} />
            </button>

            <input
              ref={attachmentInputRef}
              type="file"
              className="hidden"
              onChange={(e) =>
setAttachment(e.target.files?.[0] || null)}
            />

            <input
              type="text"
              placeholder="Write a message..."
```

```
                  value={message}
                  onChange={(e) => setMessage(e.target.value)}
                  onKeyDown={(e) => {
                    if (e.key === "Enter") sendMessage();
                  }}
                  className="flex-1 border-none bg-transparent
text-sm font-medium text-[#071a15] placeholder-[#6b7f79]
outline-none"
                />
                <button
                  onClick={sendMessage}
                  disabled={sending || (!message.trim() &&
!attachment)}
                  className={`flex-shrink-0 rounded-xl p-2
transition-all ${
                    message.trim() || attachment
                      ? "bg-[#051612] text-white shadow-sm
hover:bg-[#0d2a23]"
                      : "cursor-not-allowed bg-[#d1e8e3]
text-[#6b7f79]"
                  }`}
                >
                  <Send size={17} />
                </button>
              </div>
            </div>
          </div>
        </div>
      </main>
    </div>
  );
}
```

```
"use client";

import React, { useEffect, useState } from "react";
import {
 ArrowRight,
 Bell,
 CheckCircle,
 Clock,
 MessageSquare,
 MoreVertical,
 Video,
} from "lucide-react";
import Link from "next/link";

type NotificationItem = {
 id: string;
 type: "Interview" | "Message" | "Application" | "System";
 title: string;
 description: string;
 time: string;
 actionText: string;
 read: boolean;
};

function formatRelativeTime(dateString: string) {
 const date = new Date(dateString);
 const diff = Date.now() - date.getTime();
 const mins = Math.floor(diff / (1000 * 60));
 const hours = Math.floor(diff / (1000 * 60 * 60));
 const days = Math.floor(diff / (1000 * 60 * 60 * 24));
```

```
 if (mins < 1) return "Just now";
 if (mins < 60) return `${mins}m ago`;
 if (hours < 24) return `${hours}h ago`;
 return `${days}d ago`;
}

const typeStyle: Record<
 NotificationItem["type"],
 { badge: string; iconBg: string; iconColor: string; button:
string; icon: React.ElementType }
> = {
 Interview: {
   badge: "bg-emerald-50 text-emerald-700 ring-1
ring-emerald-200",
   iconBg: "bg-[#051612]",
   iconColor: "text-[#40b594]",
   button: "bg-[#051612] hover:bg-[#0d2a23]",
   icon: Video,
 },
 Message: {
   badge: "bg-blue-50 text-blue-700 ring-1 ring-blue-200",
   iconBg: "bg-[#e8eeff]",
   iconColor: "text-[#3553c7]",
   button: "bg-[#3553c7] hover:bg-[#2942a6]",
   icon: MessageSquare,
 },
 Application: {
   badge: "bg-amber-50 text-amber-700 ring-1 ring-amber-200",
   iconBg: "bg-[#051612]",
   iconColor: "text-[#40b594]",
   button: "bg-[#051612] hover:bg-[#0d2a23]",
```

```
    icon: Bell,
  },
  System: {
    badge: "bg-gray-100 text-gray-700 ring-1 ring-gray-200",
    iconBg: "bg-[#051612]",
    iconColor: "text-[#40b594]",
    button: "bg-[#051612] hover:bg-[#0d2a23]",
    icon: Bell,
  },
};

export default function NotificationsInterviews() {
  const [notifications, setNotifications] =
useState<NotificationItem[]>([]);
  const [loading, setLoading] = useState(true);

  const unreadCount = notifications.filter((item) =>
!item.read).length;

  useEffect(() => {
    const fetchNotifications = async () => {
      try {
        const res = await fetch("/api/notifications", { cache:
"no-store" });
        const data = await res.json();

        if (!res.ok) return;

        const mapped = (data || []).map((item: any) => ({
          id: item.id,
          type:
            item.type === "interview"
```

```
            ? "Interview"
            : item.type === "message"
              ? "Message"
              : item.type === "application"
                ? "Application"
                : "System",
        title: item.title,
        description: item.description,
        time: formatRelativeTime(item.createdAt),
        actionText:
          item.type === "message"
            ? "View Message"
            : item.type === "interview"
              ? "View Interview"
              : "Open",
        read: Boolean(item.readAt),
      }));

      setNotifications(mapped);
    } catch (error) {
      console.error(error);
    } finally {
      setLoading(false);
    }
  };

  fetchNotifications();
}, []);

const markAllRead = () => {
  setNotifications((prev) => prev.map((item) => ({ ...item,
read: true })));
```

```
 };

 const markOneRead = (id: string) => {
   setNotifications((prev) =>
     prev.map((item) => (item.id === id ? { ...item, read: true
} : item))
   );
 };

 return (
   <div className="min-h-screen bg-[#f0f4f3] pb-16 font-sans">
     <header className="sticky top-0 z-50 flex items-center
justify-between bg-[#051612] px-8 py-4 text-white shadow-lg">
       <div className="flex items-center gap-2.5">
         <img src="/logo.png" alt="NexHire" className="h-8 w-8"
/>
         <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
       </div>

       <nav className="hidden items-center gap-8 text-sm
font-semibold md:flex">
         <Link href="/home_page"><button
className="text-gray-300 transition-colors
hover:text-white">Home</button></Link>
         <Link href="/saved"><button className="text-gray-300
transition-colors hover:text-white">My Jobs</button></Link>
         <Link href="/message"><button className="text-gray-300
transition-colors hover:text-white">Messages</button></Link>
         <Link href="/notification"><button
className="border-b-2 border-[#40b594] pb-1
text-[#40b594]">Notification</button></Link>
```

```
        <Link href="/setting"><button className="text-gray-300
transition-colors hover:text-white">Settings</button></Link>
      </nav>

      <Link href="/profile">
        <div className="group flex cursor-pointer items-center
gap-3">
          <div className="text-right">
            <p className="text-[10px] uppercase tracking-widest
text-gray-500">User Name</p>
            <p className="text-sm font-bold text-white
transition-colors group-hover:text-[#40b594]">Profile</p>
          </div>
          <div className="flex h-10 w-10 items-center
justify-center rounded-full bg-[#2d4f45] text-sm font-extrabold
text-white">
            U
          </div>
        </div>
      </Link>
    </header>

    <main className="mx-auto max-w-4xl px-8 py-10">
      <div className="mb-10 flex items-start justify-between">
        <div>
          <p className="mb-1 text-xs font-bold uppercase
tracking-widest text-[#40b594]">Activity</p>
          <h1 className="text-4xl font-extrabold leading-tight
text-[#071a15]">Notifications</h1>
          <p className="mt-1 font-medium text-[#4a5a55]">Keep
track of your interview and message updates.</p>
        </div>
```

```
        <div className="mt-2 flex items-center gap-3">
          {unreadCount > 0 && (
            <div className="flex items-center gap-2 rounded-xl
border border-gray-200 bg-white px-4 py-2 shadow-sm">
              <Bell size={15} className="text-[#40b594]" />
              <span className="text-sm font-extrabold
text-[#071a15]">{unreadCount} unread</span>
            </div>
          )}
          <button
            type="button"
            onClick={markAllRead}
            className="flex items-center gap-2 rounded-xl
border border-gray-200 bg-white px-4 py-2 text-sm font-bold
text-[#071a15] shadow-sm transition-all hover:border-[#40b594]
hover:text-[#40b594]"
          >
            <CheckCircle size={16} />
            Mark all read
          </button>
        </div>
      </div>

      {loading ? (
        <div className="bg-white rounded-2xl p-8 border
border-gray-100 text-sm text-[#6b7f79]">
          Loading notifications...
        </div>
      ) : (
        <div className="space-y-4">
          {notifications.map((notification) => {
```

```
            const style = typeStyle[notification.type];
            const Icon = style.icon;

            return (
              <div
                key={notification.id}
                className={`overflow-hidden rounded-2xl border
bg-white shadow-sm transition-all ${
                  notification.read
                    ? "border-gray-100"
                    : "border-l-4 border-b-gray-100
border-l-[#40b594] border-r-gray-100 border-t-gray-100"
                } hover:border-l-[#40b594] hover:shadow-md`}
              >
                <div className="p-7">
                  <div className="mb-4 flex items-start
justify-between">
                    <div className="flex items-center gap-3">
                      <div className={`flex h-10 w-10
flex-shrink-0 items-center justify-center rounded-xl
${style.iconBg}`}>
                        <Icon size={18}
className={style.iconColor} />
                      </div>

                      <div>
                        <span className={`rounded-full px-2.5
py-1 text-xs font-extrabold ${style.badge}`}>
                          {notification.type}
                        </span>
                        <div className="mt-1.5 flex
items-center gap-1.5">
```

```
                            <Clock size={12}
className="text-[#6b7f79]" />
                            <span className="text-xs
font-semibold text-[#6b7f79]">{notification.time}</span>
                          </div>
                        </div>
                      </div>

                      <div className="flex items-center gap-2">
                        {!notification.read && <span
className="h-2.5 w-2.5 flex-shrink-0 rounded-full bg-[#40b594]"
/>}
                        <button
                          type="button"
                          onClick={() =>
markOneRead(notification.id)}
                          className="rounded-lg p-1.5
text-[#6b7f79] transition-all hover:bg-[#f0f4f3]
hover:text-[#071a15]"
                        >
                          <MoreVertical size={18} />
                        </button>
                      </div>
                    </div>

                    <div className="ml-1 pl-13">
                      <h3 className="mb-1 text-base
font-extrabold leading-snug text-[#071a15]">
                        {notification.title}
                      </h3>
                      <p className="text-sm font-medium
leading-relaxed text-[#4a5a55]">
```

```
                        {notification.description}
                      </p>
                    </div>

                    <div className="mt-5 flex items-center
justify-between">
                      <button
                        type="button"
                        className={`flex items-center gap-2
rounded-xl px-5 py-2.5 text-sm font-bold text-white shadow-sm
transition-all ${style.button}`}
                      >
                        {notification.actionText}
                        <ArrowRight size={16} />
                      </button>

                      {!notification.read && (
                        <button
                          type="button"
                          onClick={() =>
markOneRead(notification.id)}
                          className="text-xs font-bold
text-[#6b7f79] transition-colors hover:text-[#40b594]"
                        >
                          Mark as read
                        </button>
                      )}
                    </div>
                  </div>
                </div>
              );
            })}
```

```
          </div>
        )}

        {!loading && notifications.length === 0 && (
          <div className="py-24 text-center">
            <div className="mx-auto mb-4 flex h-16 w-16
items-center justify-center rounded-2xl border border-gray-100
bg-white shadow-sm">
              <Bell size={28} className="text-[#40b594]" />
            </div>
            <p className="text-lg font-extrabold
text-[#071a15]">All caught up!</p>
            <p className="mt-1 text-sm text-[#4a5a55]">No new
notifications right now.</p>
          </div>
        )}
      </main>
    </div>
  );
}
```

```
"use client";

import React, { useEffect, useState } from "react";
import Link from "next/link";
import {
  Phone,
  MapPin,
  GraduationCap,
  User2,
  Pencil,
```

```
 Save,
 X,
 Briefcase,
 Camera,
 Trash2,
 FileText,
 Upload,
} from "lucide-react";
import ImageCropModal from "@/components/ui/ImageCropModal";

type JobSeekerForm = {
 description: string;
 firstName: string;
 lastName: string;
 contact: string;
 address: string;
 educationLevel: string;
 schoolUniversity: string;
 year: string;
 cvFileName: string;
 cvUrl: string;
 profileImageUrl: string;
};

const cardClass =
 "rounded-[28px] border border-[#e6eeea] bg-white
shadow-[0_18px_50px_rgba(7,28,22,0.06)]";

export default function JobSeekerProfile() {
 const fileInputRef = React.useRef<HTMLInputElement |
null>(null);
 const cvInputRef = React.useRef<HTMLInputElement | null>(null);
```

```
 const [isEditing, setIsEditing] = useState(false);
 const [loading, setLoading] = useState(false);
 const [pageLoading, setPageLoading] = useState(true);

 const [profileImage, setProfileImage] = React.useState<File |
null>(null);
 const [sourceImageUrl, setSourceImageUrl] =
React.useState<string | null>(null);
 const [croppedPreviewUrl, setCroppedPreviewUrl] =
React.useState<string | null>(null);
 const [cropOpen, setCropOpen] = React.useState(false);

 const [cvFile, setCvFile] = React.useState<File | null>(null);

 const emptyForm: JobSeekerForm = {
   description: "",
   firstName: "",
   lastName: "",
   contact: "",
   address: "",
   educationLevel: "",
   schoolUniversity: "",
   year: "",
   cvFileName: "",
   cvUrl: "",
   profileImageUrl: "",
 };

 const [saved, setSaved] =
React.useState<JobSeekerForm>(emptyForm);
```

```
 const [draft, setDraft] =
React.useState<JobSeekerForm>(emptyForm);

 useEffect(() => {
   const loadProfile = async () => {
     try {
       const res = await fetch("/api/auth/job-seeker-profile", {
cache: "no-store" });
       const data = await res.json();

       if (!res.ok || !data) {
         setSaved(emptyForm);
         setDraft(emptyForm);
         return;
       }

       const mapped: JobSeekerForm = {
         description: data.description || "",
         firstName: data.firstName || "",
         lastName: data.lastName || "",
         contact: data.contact || "",
         address: data.address || "",
         educationLevel: data.educationLevel || "",
         schoolUniversity: data.schoolUniversity || "",
         year: data.year || "",
         cvFileName: data.cvFileName || "",
         cvUrl: data.cvUrl || "",
         profileImageUrl: data.profileImage || "",
       };

       setSaved(mapped);
       setDraft(mapped);
```

```
    } catch (error) {
      console.error("LOAD JOB SEEKER PROFILE ERROR:", error);
    } finally {
      setPageLoading(false);
    }
  };

  loadProfile();
}, []);

const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
) => {
  setDraft((prev) => ({
    ...prev,
    [e.target.name]: e.target.value,
  }));
};

const openFilePicker = () => fileInputRef.current?.click();
const openCvPicker = () => cvInputRef.current?.click();

const handleCircleAction = () => {
  if (sourceImageUrl || saved.profileImageUrl) {
    setCropOpen(true);
    return;
  }
  openFilePicker();
};

const handleDeleteImage = () => {
```

```
   if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

   setSourceImageUrl(null);
   setCroppedPreviewUrl(null);
   setProfileImage(null);
   setCropOpen(false);

   setSaved((prev) => ({ ...prev, profileImageUrl: "" }));
   setDraft((prev) => ({ ...prev, profileImageUrl: "" }));

   if (fileInputRef.current) fileInputRef.current.value = "";
 };

 const handleImageChange = (e:
React.ChangeEvent<HTMLInputElement>) => {
   const file = e.target.files?.[0] || null;
   if (!file) return;

   if (!["image/jpeg", "image/png",
"image/webp"].includes(file.type)) {
     alert("Only JPG, PNG, and WEBP images are allowed");
     e.target.value = "";
     return;
   }

   if (file.size > 5 * 1024 * 1024) {
     alert("Image must be 5MB or smaller");
     e.target.value = "";
     return;
   }
```

```
   if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

   const objectUrl = URL.createObjectURL(file);
   setSourceImageUrl(objectUrl);
   setCroppedPreviewUrl(null);
   setProfileImage(null);
   setCropOpen(true);

   e.target.value = "";
 };

 const handleCropSave = (file: File, previewUrl: string) => {
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
   setProfileImage(file);
   setCroppedPreviewUrl(previewUrl);
   setCropOpen(false);
 };

 const handleCropClose = () => setCropOpen(false);

 const handleCvChange = (e: React.ChangeEvent<HTMLInputElement>)
=> {
   const file = e.target.files?.[0] || null;
   if (!file) return;

   const allowedTypes = [
     "application/pdf",
     "application/msword",
```

```
"application/vnd.openxmlformats-officedocument.wordprocessingml.
document",
   ];

   if (!allowedTypes.includes(file.type)) {
     alert("Only PDF, DOC, and DOCX files are allowed");
     e.target.value = "";
     return;
   }

   if (file.size > 10 * 1024 * 1024) {
     alert("CV must be 10MB or smaller");
     e.target.value = "";
     return;
   }

   setCvFile(file);
   setDraft((prev) => ({
     ...prev,
     cvFileName: file.name,
   }));

   e.target.value = "";
 };

 const handleRemoveCv = () => {
   setCvFile(null);
   setDraft((prev) => ({
     ...prev,
     cvFileName: "",
     cvUrl: "",
```

```
    }));
    setSaved((prev) => ({
      ...prev,
      cvFileName: "",
      cvUrl: "",
    }));

    if (cvInputRef.current) cvInputRef.current.value = "";
  };

  React.useEffect(() => {
    return () => {
      if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
      if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
    };
  }, [sourceImageUrl, croppedPreviewUrl]);

  const handleEdit = () => {
    setDraft(saved);
    setIsEditing(true);
  };

  const handleCancel = () => {
    setDraft(saved);
    setCvFile(null);
    setIsEditing(false);
  };

  const handleSave = async () => {
    try {
      setLoading(true);
```

```
    const formData = new FormData();
    formData.append("description", draft.description);
    formData.append("firstName", draft.firstName);
    formData.append("lastName", draft.lastName);
    formData.append("contact", draft.contact);
    formData.append("address", draft.address);
    formData.append("educationLevel", draft.educationLevel);
    formData.append("schoolUniversity",
draft.schoolUniversity);
    formData.append("year", draft.year);

    if (profileImage) {
      formData.append("profileImage", profileImage);
    }

    if (cvFile) {
      formData.append("cvFile", cvFile);
    }

    const res = await fetch("/api/auth/job-seeker-profile", {
      method: "POST",
      body: formData,
    });

    const data = await res.json();

    if (!res.ok) {
      alert(data.error || "Failed to update profile");
      return;
    }
```

```
     const nextSaved: JobSeekerForm = {
       ...draft,
       cvUrl: data.cvUrl || draft.cvUrl,
       cvFileName: data.cvFileName || draft.cvFileName,
       profileImageUrl: data.profileImageUrl ||
draft.profileImageUrl,
     };

     setSaved(nextSaved);
     setDraft(nextSaved);
     setCvFile(null);
     setIsEditing(false);
   } catch (error) {
     console.error("JOB SEEKER UPDATE ERROR:", error);
     alert("Something went wrong");
   } finally {
     setLoading(false);
   }
 };

 const fullName = `${saved.firstName} ${saved.lastName}`.trim()
|| "Profile";
 const profile = isEditing ? draft : saved;
 const displayImageSrc = croppedPreviewUrl || sourceImageUrl ||
saved.profileImageUrl || "";

 const completionFields = [
   saved.firstName,
   saved.lastName,
   saved.contact,
   saved.address,
   saved.description,
```

```
    saved.educationLevel,
    saved.schoolUniversity,
    saved.year,
    saved.cvFileName || saved.cvUrl,
    saved.profileImageUrl || croppedPreviewUrl || sourceImageUrl,
  ];

  const completion = Math.round(
    (completionFields.filter(Boolean).length /
completionFields.length) * 100
  );

  if (pageLoading) {
    return (
      <div className="min-h-screen bg-[#edf4f1] flex items-center
justify-center text-sm text-[#58706a]">
        Loading profile...
      </div>
    );
  }

  return (
    <>
      <div className="min-h-screen bg-[#edf4f1] pb-16 font-sans
text-[#10211d]">
        <header className="sticky top-0 z-50 flex items-center
justify-between bg-[#051612] px-8 py-4 text-white shadow-lg">
          <div className="flex items-center gap-2">
            <img src="/logo.png" alt="NexHire" className="h-8
w-8" />
            <span className="text-xl font-bold
tracking-tight">NexHire</span>
```

```
        </div>

        <nav className="hidden items-center gap-8 text-sm
font-medium md:flex">
          <Link href="/home_page"><button
className="hover:text-gray-300">Home</button></Link>
          <Link href="/saved"><button
className="hover:text-gray-300">My Jobs</button></Link>
          <Link href="/message"><button
className="hover:text-gray-300">Messages</button></Link>
          <Link href="/notification"><button
className="hover:text-gray-300">Notification</button></Link>
          <Link href="/setting"><button
className="hover:text-gray-300">Settings</button></Link>
        </nav>

        <div className="flex items-center gap-4">
          <div className="text-right">
            <p className="text-[10px] uppercase tracking-wider
text-gray-400">User name</p>
            <p className="text-sm font-bold truncate
max-w-[140px]">{fullName}</p>
          </div>
          <div className="flex h-10 w-10 items-center
justify-center overflow-hidden rounded-full bg-[#2d4f45]
font-bold text-white">
            {saved.profileImageUrl ? (
              <img src={saved.profileImageUrl} alt={fullName}
className="h-full w-full object-cover" />
            ) : (
              saved.firstName.charAt(0) || "U"
            )}
```

```
          </div>
        </div>
      </header>

      <section className="relative overflow-hidden
bg-[#051612]">
        <div className="absolute inset-0
bg-[radial-gradient(circle_at_top_left,_rgba(64,181,148,0.22),_t
ransparent_35%),radial-gradient(circle_at_bottom_right,_rgba(255
,255,255,0.08),_transparent_28%)]" />
        <div className="relative mx-auto max-w-7xl px-8 py-12
md:py-16">
          <div className="max-w-3xl">
            <h1 className="text-3xl font-black tracking-tight
text-white md:text-5xl">
              {fullName}
            </h1>
            <p className="mt-4 max-w-2xl text-sm leading-7
text-white/70 md:text-base">
              Professional profile, photo, education, and CV
storage in one clean layout.
            </p>
          </div>
        </div>
      </section>

      <main className="mx-auto max-w-7xl px-8 pt-8">
        <section className={`${cardClass} mb-8 p-6 md:p-8`}>
          <div className="flex flex-col gap-8 lg:flex-row
lg:items-center lg:justify-between">
            <div className="flex flex-col items-center gap-6
text-center lg:flex-row lg:text-left">
```

```
                <div className="relative">
                  <div className="flex h-36 w-36 items-center
justify-center overflow-hidden rounded-full border-[6px]
border-white bg-[linear-gradient(135deg,#21483d,#40b594)]
shadow-[0_18px_50px_rgba(0,0,0,0.18)] md:h-40 md:w-40">
                    {displayImageSrc ? (
                      <img src={displayImageSrc} alt="Profile"
className="h-full w-full object-cover" />
                    ) : (
                      <svg className="h-20 w-20 text-white/85"
fill="currentColor" viewBox="0 0 24 24">
                        <path d="M12 12c2.21 0 4-1.79
4-4s-1.79-4-4-4-4 1.79-4 4 1.79 4 4 4zm0 2c-2.67 0-8 1.34-8
4v2h16v-2c0-2.66-5.33-4-8-4z" />
                      </svg>
                    )}
                  </div>

                  <button
                    type="button"
                    onClick={handleCircleAction}
                    className="absolute bottom-1 right-1 flex
h-12 w-12 items-center justify-center rounded-full border-4
border-white bg-[#00a37b] text-white shadow-lg"
                  >
                    {displayImageSrc ? <Pencil size={18} /> :
<Camera size={20} />}
                  </button>

                  {displayImageSrc && isEditing && (
                    <button
                      type="button"
```

```
                    onClick={handleDeleteImage}
                    className="absolute left-1 top-1 flex h-10
w-10 items-center justify-center rounded-full border-4
border-white bg-white text-[#d11a2a] shadow-md"
                  >
                    <Trash2 size={16} />
                  </button>
                )}

                <input
                  ref={fileInputRef}
                  type="file"
                  accept="image/jpeg,image/png,image/webp"
                  onChange={handleImageChange}
                  className="hidden"
                />
              </div>

              <div className="max-w-2xl">
                <h2 className="text-2xl font-black
text-[#0d211b] md:text-3xl">{fullName}</h2>
                <p className="mt-2 text-sm font-bold
text-[#40b594]">Open to opportunities</p>
                <p className="mt-4 text-sm leading-7
text-[#58706a]">{saved.description || "No description yet."}</p>
              </div>
            </div>

            <div className="flex flex-wrap justify-center gap-3
lg:justify-end">
              {isEditing ? (
                <>
```

```
                    <button
                      type="button"
                      onClick={handleCancel}
                      className="rounded-2xl border
border-[#d7e5df] bg-white px-5 py-3 text-sm font-bold
text-[#17332b]"
                    >
                      <span className="inline-flex items-center
gap-2"><X size={16} />Cancel</span>
                    </button>
                    <button
                      type="button"
                      onClick={handleSave}
                      disabled={loading}
                      className="rounded-2xl bg-[#051612] px-5
py-3 text-sm font-bold text-white disabled:opacity-60"
                    >
                      <span className="inline-flex items-center
gap-2"><Save size={16} />{loading ? "Saving..." : "Save
Changes"}</span>
                    </button>
                  </>
                ) : (
                  <button
                    type="button"
                    onClick={handleEdit}
                    className="rounded-2xl bg-[#051612] px-5 py-3
text-sm font-bold text-white"
                  >
                    <span className="inline-flex items-center
gap-2"><Pencil size={16} />Edit Profile</span>
                  </button>
```

```
              )}
            </div>
          </div>
        </section>

        <div className="grid grid-cols-1 gap-8
lg:grid-cols-[0.95fr_2.05fr]">
          <aside className="space-y-8">
            <section className={`${cardClass} p-6`}>
              <p className="mb-2 text-[10px] font-bold
uppercase tracking-[0.22em] text-[#40b594]">
                Profile Completion
              </p>
              <div className="mb-3 flex items-end
justify-between">
                <p className="text-3xl font-black
text-[#0c201a]">{completion}%</p>
                <p className="text-xs font-semibold
text-[#6a817b]">Complete your candidate profile</p>
              </div>
              <div className="h-3 w-full rounded-full
bg-[#e5efeb]">
                <div className="h-3 rounded-full bg-[#40b594]"
style={{ width: `${completion}%` }} />
              </div>
            </section>

            <section className={`${cardClass} p-6`}>
              <h3 className="mb-5 text-base font-extrabold
text-[#071a15]">Quick Info</h3>
              <div className="space-y-4">
```

```
                  <InfoRow icon={Phone} label="Contact"
value={saved.contact || "Not added"} />
                  <InfoRow icon={MapPin} label="Address"
value={saved.address || "Not added"} />
                  <InfoRow icon={GraduationCap} label="Education"
value={saved.educationLevel || "Not added"} />
                </div>
              </section>

              <section className={`${cardClass} p-6`}>
                <div className="mb-5 flex items-center gap-3">
                  <div className="flex h-11 w-11 items-center
justify-center rounded-2xl bg-[#eff8f5] text-[#40b594]">
                    <FileText size={18} />
                  </div>
                  <div>
                    <h3 className="text-base font-extrabold
text-[#071a15]">CV Storage</h3>
                    <p className="text-xs text-[#6a817b]">Store
one CV in this profile</p>
                  </div>
                </div>

                <div className="rounded-3xl border border-dashed
border-[#cfe2db] bg-[#f8fcfa] p-4">
                  <p className="text-sm font-semibold
text-[#17332b]">
                    {saved.cvFileName || "No CV uploaded yet"}
                  </p>
                  {saved.cvUrl ? (
                    <a
                      href={saved.cvUrl}
```

```
                    target="_blank"
                    rel="noreferrer"
                    className="mt-3 inline-flex items-center
gap-2 text-sm font-bold text-[#0a7e61]"
                  >
                    <FileText size={16} />
                    View uploaded CV
                  </a>
                ) : (
                  <p className="mt-2 text-xs
text-[#7a8f89]">PDF, DOC, or DOCX</p>
                )}
              </div>
            </section>
          </aside>

          <div className="space-y-8">
            <section className={`${cardClass} p-6 md:p-8`}>
              <div className="mb-6 flex items-center
justify-between">
                <h3 className="text-xl font-black
text-[#0d211b]">Personal Information</h3>
                <User2 className="text-[#40b594]" size={22} />
              </div>

              <div className="grid grid-cols-1 gap-5
md:grid-cols-2">
                <EditableField editing={isEditing} label="First
Name" name="firstName" value={profile.firstName}
onChange={handleChange} />
```

```
                  <EditableField editing={isEditing} label="Last
Name" name="lastName" value={profile.lastName}
onChange={handleChange} />
                  <EditableField editing={isEditing}
label="Contact" name="contact" value={profile.contact}
onChange={handleChange} className="md:col-span-2" />
                  <EditableField editing={isEditing}
label="Address" name="address" value={profile.address}
onChange={handleChange} className="md:col-span-2" />
                </div>
              </section>

              <section className={`${cardClass} p-6 md:p-8`}>
                <div className="mb-6 flex items-center
justify-between">
                  <h3 className="text-xl font-black
text-[#0d211b]">Education Details</h3>
                  <Briefcase className="text-[#40b594]" size={22}
/>
                </div>

                <div className="grid grid-cols-1 gap-5
md:grid-cols-3">
                  <EditableSelect
                    editing={isEditing}
                    label="Education Level"
                    name="educationLevel"
                    value={profile.educationLevel}
                    onChange={handleChange}
                    options={["High School", "Associate",
"Bachelor", "Master", "PhD"]}
                  />
```

```
                  <EditableField editing={isEditing}
label="School / University" name="schoolUniversity"
value={profile.schoolUniversity} onChange={handleChange} />
                  <EditableField editing={isEditing} label="Year"
name="year" value={profile.year} onChange={handleChange} />
                </div>
              </section>

              <section className={`${cardClass} p-6 md:p-8`}>
                <h3 className="mb-6 text-xl font-black
text-[#0d211b]">About Candidate</h3>
                {isEditing ? (
                  <textarea
                    name="description"
                    value={profile.description}
                    onChange={handleChange}
                    className="h-36 w-full resize-none
rounded-3xl border border-[#d9e8e2] bg-[#f6fbf8] px-5 py-4
text-sm"
                  />
                ) : (
                  <p className="text-sm leading-8
text-[#405752]">{saved.description || "Not added"}</p>
                )}
              </section>

              <section className={`${cardClass} p-6 md:p-8`}>
                <div className="mb-6 flex items-center
justify-between">
                  <div>
                    <h3 className="text-xl font-black
text-[#0d211b]">Upload CV</h3>
```

```
                    <p className="mt-1 text-sm
text-[#6a817b]">Store your resume directly in this profile</p>
                  </div>
                  <FileText className="text-[#40b594]" size={22}
/>
                </div>

                <div className="rounded-[28px] border
border-dashed border-[#cfe2db] bg-[#f8fcfa] p-6">
                  <div className="flex flex-col gap-4 md:flex-row
md:items-center md:justify-between">
                    <div>
                      <p className="text-sm font-bold
text-[#17332b]">
                        {profile.cvFileName || saved.cvFileName
|| "No CV selected"}
                      </p>
                      <p className="mt-1 text-xs text-[#7a8f89]">
                        Accepted formats: PDF, DOC, DOCX. Max
size: 10MB.
                      </p>
                    </div>

                    <div className="flex flex-wrap gap-3">
                      {isEditing && (
                        <>
                          <button
                            type="button"
                            onClick={openCvPicker}
                            className="inline-flex items-center
gap-2 rounded-2xl bg-[#051612] px-4 py-3 text-sm font-bold
text-white"
```

```
                        >
                          <Upload size={16} />
                          Upload CV
                        </button>

                        {(profile.cvFileName ||
saved.cvFileName) && (
                          <button
                            type="button"
                            onClick={handleRemoveCv}
                            className="inline-flex items-center
gap-2 rounded-2xl border border-[#d7e5df] bg-white px-4 py-3
text-sm font-bold text-[#b42318]"
                          >
                            <Trash2 size={16} />
                            Remove CV
                          </button>
                        )}
                      </>
                    )}

                    {!isEditing && saved.cvUrl && (
                      <a
                        href={saved.cvUrl}
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center
gap-2 rounded-2xl bg-[#eff8f5] px-4 py-3 text-sm font-bold
text-[#0a7e61]"
                      >
                        <FileText size={16} />
                        View CV
```

```
                    </a>
                  )}
                </div>
              </div>

              <input
                ref={cvInputRef}
                type="file"

accept=".pdf,.doc,.docx,application/pdf,application/msword,application/vnd.openxmlformats-officedocument.wordprocessingml.document"
                onChange={handleCvChange}
                className="hidden"
              />
            </div>
          </section>
        </div>
      </div>
    </main>
  </div>

  <ImageCropModal
    open={cropOpen}
    imageSrc={sourceImageUrl || saved.profileImageUrl}
    title="Adjust profile photo"
    onClose={handleCropClose}
    onSave={handleCropSave}
    onUploadAnother={openFilePicker}
    onDelete={handleDeleteImage}
    hasImage={!!(sourceImageUrl || saved.profileImageUrl)}
  />
```

```
    </>
  );
}

function InfoRow({
  icon: Icon,
  label,
  value,
}: {
  icon: React.ComponentType<{ size?: number }>;
  label: string;
  value: string;
}) {
  return (
    <div className="flex items-start gap-4 rounded-2xl
bg-[#f6fbf8] px-4 py-3">
      <div className="mt-0.5 rounded-xl bg-white p-2
text-[#40b594] shadow-sm">
        <Icon size={16} />
      </div>
      <div>
        <p className="text-[10px] font-bold uppercase
tracking-[0.22em] text-[#6f8780]">{label}</p>
        <p className="mt-1 text-sm font-bold
text-[#10211d]">{value}</p>
      </div>
    </div>
  );
}

function EditableField({
  editing,
```

```
 label,
 name,
 value,
 onChange,
 className = "",
}: {
 editing: boolean;
 label: string;
 name: string;
 value: string;
 onChange: (e: React.ChangeEvent<HTMLInputElement |
HTMLTextAreaElement | HTMLSelectElement>) => void;
 className?: string;
}) {
 return (
   <div className={className}>
     <p className="mb-2 text-sm font-bold
text-[#27413a]">{label}</p>
     {editing ? (
       <input
         name={name}
         value={value}
         onChange={onChange}
         className="w-full rounded-2xl border border-[#d9e8e2]
bg-[#f6fbf8] px-4 py-3 text-sm"
       />
     ) : (
       <div className="rounded-2xl border border-[#edf3f0]
bg-[#f9fcfb] px-4 py-3 text-sm font-semibold">
         {value || "Not added"}
       </div>
     )}
```

```
    </div>
  );
}

function EditableSelect({
  editing,
  label,
  name,
  value,
  onChange,
  options,
}: {
  editing: boolean;
  label: string;
  name: string;
  value: string;
  onChange: (e: React.ChangeEvent<HTMLInputElement |
HTMLTextAreaElement | HTMLSelectElement>) => void;
  options: string[];
}) {
  return (
    <div>
      <p className="mb-2 text-sm font-bold
text-[#27413a]">{label}</p>
      {editing ? (
        <select
          name={name}
          value={value}
          onChange={onChange}
          className="w-full rounded-2xl border border-[#d9e8e2]
bg-[#f6fbf8] px-4 py-3 text-sm"
        >
```

```
          <option value="">Select option</option>
          {options.map((option) => (
            <option key={option} value={option}>
              {option}
            </option>
          ))}
        </select>
      ) : (
        <div className="rounded-2xl border border-[#edf3f0]
bg-[#f9fcfb] px-4 py-3 text-sm font-semibold">
          {value || "Not added"}
        </div>
      )}
    </div>
  );
}
```

```
"use client";

import React, { useState } from "react";
import {
  Shield,
  AlertTriangle,
  Mail,
  Save,
  User,
  MapPin,
  Trash2,
  LogOut,
  ChevronRight,
  Lock,
```

```
 Eye,
 EyeOff,
 Briefcase,
 ExternalLink,
 UserCog,
 FileText,
} from "lucide-react";
import Link from "next/link";

const JobSeekerSettings = () => {
 const [activeTab, setActiveTab] = useState("general");
 const [showCurrent, setShowCurrent] = useState(false);
 const [showNew, setShowNew] = useState(false);
 const [deleteText, setDeleteText] = useState("");

 const [settings, setSettings] = useState({
   accountEmail: "user@example.com",
   recoveryEmail: "backup@example.com",
 });

 const tabs = [
   { id: "general", name: "General", icon: UserCog },
   { id: "security", name: "Security", icon: Shield },
   { id: "danger", name: "Danger Zone", icon: AlertTriangle },
 ];

 const inputClass =
   "w-full rounded-xl border border-gray-200 bg-[#f8faf9] px-4
py-3 text-sm font-medium text-[#071a15] placeholder-[#9ab0aa]
transition-all focus:border-[#40b594] focus:outline-none
focus:ring-2 focus:ring-[#40b594]/30";
```

```
 const labelClass = "mb-2 block text-sm font-extrabold
text-[#071a15]";

 return (
   <div className="min-h-screen bg-[#f0f4f3] pb-16 font-sans">
     <header className="sticky top-0 z-50 flex items-center
justify-between bg-[#051612] px-8 py-4 text-white shadow-lg">
       <div className="flex items-center gap-2">
         <img src="/logo.png" alt="NexHire" className="h-8 w-8"
/>
         <span className="text-xl font-bold
tracking-tight">NexHire</span>
       </div>

       <nav className="hidden items-center gap-8 text-sm
font-medium md:flex">
         <Link href="/home_page">
           <button className="transition-colors
hover:text-gray-300">Home</button>
         </Link>
         <Link href="/saved">
           <button className="transition-colors
hover:text-gray-300">My Jobs</button>
         </Link>
         <Link href="/message">
           <button className="transition-colors
hover:text-gray-300">Messages</button>
         </Link>
         <Link href="/notification">
           <button className="transition-colors
hover:text-gray-300">Notification</button>
         </Link>
```

```
        <button className="border-b-2 border-[#40b594] pb-1
text-[#40b594]">Settings</button>
      </nav>

      <Link href="/my_profile">
        <div className="group flex cursor-pointer items-center
gap-3">
          <div className="text-right">
            <p className="text-[10px] uppercase tracking-wider
text-gray-400">User name</p>
            <p className="text-sm font-bold text-white
transition-colors group-hover:text-[#40b594]">
              Profile
            </p>
          </div>
          <div className="flex h-10 w-10 items-center
justify-center rounded-full bg-[#2d4f45] font-bold text-white">
            U
          </div>
        </div>
      </Link>
    </header>

    <main className="mx-auto max-w-6xl px-6 py-10 md:px-10">
      <div className="mb-8">
        <p className="mb-1 text-xs font-bold uppercase
tracking-widest text-[#40b594]">Account</p>
        <h1 className="text-4xl font-extrabold
text-[#071a15]">Settings</h1>
        <p className="mt-1 font-medium text-[#4a5a55]">
          Manage account controls, email settings, and security
        </p>
```

```
      </div>

      <div className="flex flex-col gap-6 lg:flex-row">
        <aside className="flex-shrink-0 lg:w-72">
          <div className="overflow-hidden rounded-2xl border
border-gray-100 bg-white shadow-sm">
            {tabs.map((tab, idx) => {
              const Icon = tab.icon;
              const isActive = activeTab === tab.id;
              const isDanger = tab.id === "danger";

              return (
                <button
                  key={tab.id}
                  onClick={() => setActiveTab(tab.id)}
                  className={`w-full px-5 py-4 text-sm
font-bold transition-all ${
                    idx !== 0 ? "border-t border-gray-100" : ""
                  } ${
                    isActive
                      ? isDanger
                        ? "bg-red-50 text-red-600"
                        : "bg-[#f0f9f6] text-[#071a15]"
                      : isDanger
                        ? "text-red-400 hover:bg-red-50
hover:text-red-600"
                        : "text-[#4a5a55] hover:bg-[#f8faf9]
hover:text-[#071a15]"
                  }`}
                >
                  <div className="flex items-center
justify-between">
```

```
                    <div className="flex items-center gap-3">
                      <div
                        className={`flex h-8 w-8 items-center
justify-center rounded-lg ${
                          isActive
                            ? isDanger
                              ? "bg-red-100"
                              : "bg-[#d1e8e3]"
                            : "bg-[#f0f4f3]"
                        }`}
                      >
                        <Icon
                          size={16}
                          className={
                            isActive
                              ? isDanger
                                ? "text-red-600"
                                : "text-[#40b594]"
                              : isDanger
                                ? "text-red-400"
                                : "text-[#6b7f79]"
                          }
                        />
                      </div>
                      {tab.name}
                    </div>

                    {isActive && (
                      <ChevronRight
                        size={16}
                        className={isDanger ? "text-red-400" :
"text-[#40b594]"}
```

```
                        />
                      )}
                    </div>
                  </button>
                );
              })}
            </div>

            <button className="mt-4 flex w-full items-center
gap-3 rounded-2xl border border-gray-100 bg-white px-5 py-4
text-sm font-bold text-[#4a5a55] shadow-sm transition-all
hover:border-red-200 hover:bg-red-50 hover:text-red-600">
              <div className="flex h-8 w-8 items-center
justify-center rounded-lg bg-[#f0f4f3]">
                <LogOut size={16} className="text-[#6b7f79]" />
              </div>
              Sign Out
            </button>
          </aside>

          <div className="min-w-0 flex-1">
            {activeTab === "general" && (
              <div className="space-y-6">
                <div className="overflow-hidden rounded-2xl
border border-gray-100 bg-white shadow-sm">
                  <div className="border-b border-gray-100 px-8
py-6">
                    <h2 className="text-lg font-extrabold
text-[#071a15]">Profile Overview</h2>
                    <p className="mt-0.5 text-sm font-medium
text-[#4a5a55]">
```

```
                      Personal information, photo, and CV are
managed from your profile page
                    </p>
                  </div>

                  <div className="p-8">
                    <div className="grid gap-4 md:grid-cols-2">
                      <SummaryCard icon={User} label="Full Name"
value="Marsslu SMC" />
                      <SummaryCard icon={MapPin} label="Location"
value="Phnom Penh, Cambodia" />
                      <SummaryCard icon={Briefcase}
label="Education" value="Bachelor" />
                      <SummaryCard icon={FileText} label="CV"
value="Resume available" />
                    </div>

                    <div className="mt-6 rounded-2xl border
border-[#d1e8e3] bg-[#f0f9f6] p-5">
                      <div className="flex flex-col gap-4
md:flex-row md:items-center md:justify-between">
                        <div>
                          <p className="text-sm font-extrabold
text-[#071a15]">
                            Edit your profile from the profile
page
                          </p>
                          <p className="mt-1 text-sm font-medium
text-[#4a5a55]">
                            Keep your public information, profile
image, and CV in one place instead of duplicating forms here.
                          </p>
```

```
                        </div>
                        <Link href="/my_profile">
                          <button className="inline-flex
items-center gap-2 rounded-xl bg-[#051612] px-5 py-3 text-sm
font-extrabold text-white transition-all hover:bg-[#0d2a23]">
                            Open Profile
                            <ExternalLink size={16} />
                          </button>
                        </Link>
                      </div>
                    </div>
                  </div>
                </div>

                <div className="overflow-hidden rounded-2xl
border border-gray-100 bg-white shadow-sm">
                  <div className="border-b border-gray-100 px-8
py-6">
                    <h2 className="text-lg font-extrabold
text-[#071a15]">Account Emails</h2>
                    <p className="mt-0.5 text-sm font-medium
text-[#4a5a55]">
                      Manage the emails used for your account and
recovery
                    </p>
                  </div>

                  <div className="p-8">
                    <form className="space-y-6">
                      <div>
                        <label className={labelClass}>Account
Email</label>
```

```
                        <div className="relative">
                          <Mail size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                          <input
                            type="email"
                            value={settings.accountEmail}
                            onChange={(e) =>
                              setSettings((prev) => ({ ...prev,
accountEmail: e.target.value }))
                            }
                            className={`${inputClass} pl-10`}
                          />
                        </div>
                        <p className="ml-1 mt-2 text-xs
font-semibold text-[#6b7f79]">
                          Used for login and account
notifications
                        </p>
                      </div>

                      <div>
                        <label className={labelClass}>Recovery
Email</label>
                        <div className="relative">
                          <Mail size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                          <input
                            type="email"
                            value={settings.recoveryEmail}
                            onChange={(e) =>
                              setSettings((prev) => ({ ...prev,
recoveryEmail: e.target.value }))
```

```
                          }
                          className={`${inputClass} pl-10`}
                        />
                      </div>
                      <p className="ml-1 mt-2 text-xs
font-semibold text-[#6b7f79]">
                        Used for recovery and backup
communication
                      </p>
                    </div>

                    <div className="pt-2">
                      <button
                        type="button"
                        className="flex items-center gap-2
rounded-xl bg-[#051612] px-6 py-3 text-sm font-extrabold
text-white transition-all hover:bg-[#0d2a23]"
                      >
                        <Save size={16} />
                        Save Changes
                      </button>
                    </div>
                  </form>
                </div>
              </div>
            </div>
          )}

          {activeTab === "security" && (
            <div className="overflow-hidden rounded-2xl border
border-gray-100 bg-white shadow-sm">
```

```
              <div className="border-b border-gray-100 px-8
py-6">
                <h2 className="text-lg font-extrabold
text-[#071a15]">Security</h2>
                <p className="mt-0.5 text-sm font-medium
text-[#4a5a55]">
                  Update your account password
                </p>
              </div>

              <div className="p-8">
                <form className="max-w-md space-y-6">
                  <div>
                    <label className={labelClass}>Current
Password</label>
                    <div className="relative">
                      <Lock size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                      <input
                        type={showCurrent ? "text" :
"password"}
                        placeholder="Enter current password"
                        className={`${inputClass} pl-10 pr-10`}
                      />
                      <button
                        type="button"
                        onClick={() =>
setShowCurrent(!showCurrent)}
                        className="absolute right-4 top-1/2
-translate-y-1/2 text-[#6b7f79] transition-colors
hover:text-[#071a15]"
                      >
```

```
                        {showCurrent ? <EyeOff size={16} /> :
<Eye size={16} />}
                      </button>
                    </div>
                  </div>

                  <div>
                    <label className={labelClass}>New
Password</label>
                    <div className="relative">
                      <Lock size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                      <input
                        type={showNew ? "text" : "password"}
                        placeholder="Enter new password"
                        className={`${inputClass} pl-10 pr-10`}
                      />
                      <button
                        type="button"
                        onClick={() => setShowNew(!showNew)}
                        className="absolute right-4 top-1/2
-translate-y-1/2 text-[#6b7f79] transition-colors
hover:text-[#071a15]"
                      >
                        {showNew ? <EyeOff size={16} /> : <Eye
size={16} />}
                      </button>
                    </div>
                  </div>

                  <div className="rounded-xl border
border-[#d1e8e3] bg-[#f0f9f6] p-4">
```

```
                    <p className="mb-2 text-xs font-extrabold
text-[#071a15]">Password requirements</p>
                    {["At least 8 characters", "One uppercase
letter", "One number or symbol"].map((req) => (
                      <div key={req} className="mt-1.5 flex
items-center gap-2">
                        <div className="h-1.5 w-1.5
rounded-full bg-[#40b594]" />
                        <p className="text-xs font-medium
text-[#4a5a55]">{req}</p>
                      </div>
                    ))}
                  </div>

                  <div className="pt-2">
                    <button
                      type="button"
                      className="flex items-center gap-2
rounded-xl bg-[#051612] px-6 py-3 text-sm font-extrabold
text-white transition-all hover:bg-[#0d2a23]"
                    >
                      <Shield size={16} />
                      Update Password
                    </button>
                  </div>
                </form>
              </div>
            </div>
          )}

          {activeTab === "danger" && (
```

```
            <div className="overflow-hidden rounded-2xl border
border-red-100 bg-white shadow-sm">
              <div className="border-b border-red-100
bg-red-50/50 px-8 py-6">
                <h2 className="text-lg font-extrabold
text-red-600">Danger Zone</h2>
                <p className="mt-0.5 text-sm font-medium
text-red-400">
                  These actions are permanent and cannot be
undone
                </p>
              </div>

              <div className="p-8">
                <div className="rounded-2xl border
border-red-100 bg-red-50/30 p-7">
                  <div className="mb-6 flex items-start gap-4">
                    <div className="flex h-10 w-10
flex-shrink-0 items-center justify-center rounded-xl
bg-red-100">
                      <Trash2 size={18}
className="text-red-600" />
                    </div>
                    <div>
                      <h3 className="text-base font-extrabold
text-[#071a15]">Delete Job Seeker Account</h3>
                      <p className="mt-1 text-sm font-medium
leading-relaxed text-[#4a5a55]">
                        This will permanently delete your
account, saved jobs, messages, and profile history. This action
cannot be reversed.
                      </p>
```

```
                    </div>
                  </div>

                  <div className="space-y-4">
                    <div>
                      <label className="mb-2 block text-sm
font-extrabold text-[#071a15]">
                        Type <span className="font-black
tracking-widest text-red-600">DELETE</span> to confirm
                      </label>
                      <input
                        type="text"
                        placeholder="Type DELETE here"
                        value={deleteText}
                        onChange={(e) =>
setDeleteText(e.target.value)}
                        className="w-full max-w-sm rounded-xl
border border-red-200 bg-white px-4 py-3 text-sm font-medium
text-[#071a15] placeholder-red-200 transition-all
focus:border-red-400 focus:outline-none focus:ring-2
focus:ring-red-300/40"
                      />
                    </div>

                    <button
                      type="button"
                      disabled={deleteText !== "DELETE"}
                      className={`flex items-center gap-2
rounded-xl px-6 py-3 text-sm font-extrabold transition-all ${
                        deleteText === "DELETE"
                          ? "bg-red-600 text-white
hover:bg-red-700"
```

```
                          : "cursor-not-allowed bg-red-100
text-red-300"
                        }`}
                      >
                        <Trash2 size={16} />
                        Delete Account
                      </button>
                    </div>
                  </div>
                </div>
              </div>
            )}
          </div>
        </div>
      </main>
    </div>
  );
};

function SummaryCard({
  icon: Icon,
  label,
  value,
}: {
  icon: React.ComponentType<{ size?: number; className?: string
}>;
  label: string;
  value: string;
}) {
  return (
    <div className="rounded-2xl border border-gray-100
bg-[#f8faf9] p-5">
```

```
     <div className="mb-3 flex h-10 w-10 items-center
justify-center rounded-xl bg-[#eaf5f1]">
       <Icon size={18} className="text-[#40b594]" />
     </div>
     <p className="text-[11px] font-black uppercase
tracking-wider text-[#6b7f79]">{label}</p>
     <p className="mt-1 text-sm font-extrabold
text-[#071a15]">{value}</p>
   </div>
 );
}

export default JobSeekerSettings;
```

```
"use client";

import React, { useRef } from "react";
import {
 Users,
 Plus,
 Eye,
 FileText,
 TrendingUp,
 ArrowUpRight,
} from "lucide-react";
import Link from "next/link";
import EmployerNavProfile from
"@/components/ui/EmployerNavProfile";

const EmployerDashboard = () => {
 const applicantsRef = useRef<HTMLDivElement | null>(null);
```

```
 const handleScrollToApplicants = () => {
   applicantsRef.current?.scrollIntoView({ behavior: "smooth"
});
 };

 return (
   <div className="min-h-screen font-sans pb-16" style={{
background: "#f0f4f3" }}>
     <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
       <div className="flex items-center gap-2.5">
         <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
         <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
       </div>

       <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
         <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Dashboard</button>
         <Link href="/post_job">
           <button className="text-gray-300 hover:text-white
transition-colors">Post Job</button>
         </Link>
         <Link href="/employer_message">
           <button className="text-gray-300 hover:text-white
transition-colors">Messages</button>
         </Link>
         <Link href="/employer_notification">
```

```
          <button className="text-gray-300 hover:text-white
transition-colors">Notification</button>
        </Link>
        <Link href="/subscription">
          <button className="text-gray-300 hover:text-white
transition-colors">Subscription</button>
        </Link>
        <Link href="/employer_setting">
          <button className="text-gray-300 hover:text-white
transition-colors">Settings</button>
        </Link>
      </nav>

      <EmployerNavProfile />
    </header>

    <main className="max-w-7xl mx-auto px-6 md:px-10 py-10">
      <div className="flex items-center justify-between mb-10">
        <div>
          <p className="text-xs font-bold uppercase
tracking-widest text-[#40b594] mb-1">
            Dashboard Overview
          </p>
          <h1 className="text-4xl font-extrabold text-[#071a15]
leading-tight">
            Welcome back
          </h1>
        </div>
        <Link href="/post_job">
          <button className="flex items-center gap-2
bg-[#051612] text-white px-6 py-3 rounded-xl font-bold text-sm
hover:bg-[#0d2a23] transition-all shadow-md">
```

```
            <Plus size={18} /> Post New Job
          </button>
        </Link>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-5
mb-8">
        {[
          {
            label: "Total Jobs Posted",
            value: "0",
            icon: FileText,
            bg: "bg-[#051612]",
            iconBg: "bg-[#0d2a23]",
            trend: "0 total listings",
          },
          {
            label: "Total Applicants",
            value: "0",
            icon: Users,
            bg: "bg-[#133228]",
            iconBg: "bg-[#1a4035]",
            trend: "0 received",
          },
          {
            label: "Active Jobs",
            value: "0",
            icon: TrendingUp,
            bg: "bg-[#40b594]",
            iconBg: "bg-[#33997a]",
            trend: "0 open roles",
            dark: true,
```

```
            },
          ].map(({ label, value, icon: Icon, bg, iconBg, trend,
dark }) => (
            <div key={label} className={`${bg} rounded-2xl p-7
flex flex-col gap-5 shadow-sm`}>
              <div className="flex items-start justify-between">
                <div className={`${iconBg} p-3 rounded-xl`}>
                  <Icon className={dark ? "text-[#051612]" :
"text-[#40b594]"} size={24} />
                </div>
                <ArrowUpRight className={dark ? "text-[#051612]
opacity-60" : "text-[#40b594] opacity-60"} size={18} />
              </div>
              <div>
                <p className={`text-5xl font-extrabold ${dark ?
"text-[#051612]" : "text-white"} leading-none mb-1`}>
                  {value}
                </p>
                <p className={`text-sm font-semibold ${dark ?
"text-[#071a15]" : "text-gray-300"}`}>
                  {label}
                </p>
                <p className={`text-xs mt-1 ${dark ?
"text-[#133228]" : "text-[#40b594]"} font-medium`}>
                  {trend}
                </p>
              </div>
            </div>
          ))}
        </div>
```

```
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6
mb-6">
        <div className="lg:col-span-2 bg-white rounded-2xl
shadow-sm border border-gray-100 overflow-hidden">
          <div className="flex items-center justify-between
px-8 py-6 border-b border-gray-100">
            <h2 className="text-lg font-extrabold
text-[#071a15]">Recent Jobs</h2>
            <span className="text-xs font-bold text-[#40b594]
bg-emerald-50 px-3 py-1 rounded-full">
              0 listings
            </span>
          </div>
          <div className="px-8 py-6 text-sm text-[#6b7f79]">
            No jobs posted yet.
          </div>
        </div>

        <div className="bg-white rounded-2xl shadow-sm border
border-gray-100 p-6 flex flex-col gap-4 h-fit sticky top-24">
          <h2 className="text-lg font-extrabold text-[#071a15]
mb-2">Quick Actions</h2>

          <Link href="/post_job" className="block">
            <button className="w-full bg-[#051612] text-white
py-4 rounded-xl font-bold text-sm flex items-center
justify-center gap-2.5 hover:bg-[#0d2a23] transition-all
shadow-sm">
              <Plus size={18} /> Post a New Job
            </button>
          </Link>
```

```
          <button
            onClick={handleScrollToApplicants}
            className="w-full border-2 border-[#d1e8e3]
text-[#071a15] py-4 rounded-xl font-bold text-sm flex
items-center justify-center gap-2.5 hover:bg-[#f0f9f6]
hover:border-[#40b594] transition-all"
          >
            <Eye size={18} /> View Applicants
          </button>

          <Link href="/post_job">
            <button className="w-full border-2 border-[#d1e8e3]
text-[#071a15] py-4 rounded-xl font-bold text-sm flex
items-center justify-center gap-2.5 hover:bg-[#f0f9f6]
hover:border-[#40b594] transition-all">
              <FileText size={18} /> Drafts
            </button>
          </Link>
        </div>
      </div>

      <section ref={applicantsRef} className="bg-white
rounded-2xl shadow-sm border border-gray-100 overflow-hidden">
        <div className="flex items-center justify-between px-8
py-6 border-b border-gray-100">
          <h2 className="text-lg font-extrabold
text-[#071a15]">Recent Applicants</h2>
          <span className="text-xs font-bold text-[#40b594]
bg-emerald-50 px-3 py-1 rounded-full">
            0 total
          </span>
        </div>
```

```
          <div className="px-8 py-6 text-sm text-[#6b7f79]">
            No applicants yet.
          </div>
        </section>
      </main>
    </div>
  );
};

export default EmployerDashboard;
```

```
"use client";

import React, { useEffect, useMemo, useRef, useState } from
"react";
import {
  Search,
  Paperclip,
  Send,
  Pencil,
  Trash2,
  X,
} from "lucide-react";
import Link from "next/link";

type Conversation = {
  id: string;
  employerId: string;
  jobSeekerId: string;
  jobId: string | null;
  lastMessageAt: string;
```

```
 employerName: string;
 employerImage: string | null;
 seekerFirstName: string;
 seekerLastName: string;
 seekerImage: string | null;
};

type Message = {
 id: string;
 conversationId: string;
 senderId: string;
 text: string | null;
 attachmentUrl: string | null;
 attachmentName: string | null;
 attachmentType: string | null;
 editedAt: string | null;
 createdAt: string;
};

type Me = {
 userId: string;
 role: "employer" | "job_seeker";
 email: string;
};

async function safeFetchJson(url: string, options?: RequestInit)
{
 const res = await fetch(url, options);
 const text = await res.text();

 let data: any = null;
```

```
 try {
   data = text ? JSON.parse(text) : null;
 } catch {
   console.error(`Non-JSON response from ${url}:`, text);
   throw new Error(`Invalid JSON response from ${url}`);
 }

 if (!res.ok) {
   throw new Error(data?.error || `Request failed:
${res.status}`);
 }

 return data;
}

function formatTime(dateString: string) {
 return new Date(dateString).toLocaleTimeString("en-US", {
   hour: "numeric",
   minute: "2-digit",
 });
}

function formatConversationTime(dateString: string) {
 const date = new Date(dateString);
 const now = new Date();
 const sameDay = date.toDateString() === now.toDateString();

 if (sameDay) return formatTime(dateString);

 const yesterday = new Date();
 yesterday.setDate(now.getDate() - 1);
```

```
 if (date.toDateString() === yesterday.toDateString()) return
"Yesterday";

 return date.toLocaleDateString("en-US", {
   month: "short",
   day: "2-digit",
 });
}

function canEdit(createdAt: string) {
 return Date.now() - new Date(createdAt).getTime() <= 15 * 60 *
1000;
}

export default function Messages() {
 const [me, setMe] = useState<Me | null>(null);
 const [conversations, setConversations] =
useState<Conversation[]>([]);
 const [activeChat, setActiveChat] = useState<string>("");
 const [messages, setMessages] = useState<Message[]>([]);
 const [message, setMessage] = useState("");
 const [search, setSearch] = useState("");
 const [loadingChats, setLoadingChats] = useState(true);
 const [loadingMessages, setLoadingMessages] = useState(false);
 const [sending, setSending] = useState(false);
 const [editingId, setEditingId] = useState<string |
null>(null);
 const [editingText, setEditingText] = useState("");
 const [attachment, setAttachment] = useState<File |
null>(null);
```

```
 const attachmentInputRef = useRef<HTMLInputElement |
null>(null);

 useEffect(() => {
   const loadMe = async () => {
     try {
       const data = await safeFetchJson("/api/auth/me");
       setMe(data);
     } catch (error) {
       console.error("LOAD ME ERROR:", error);
     }
   };

   loadMe();
 }, []);

 const fetchConversations = async () => {
   try {
     setLoadingChats(true);
     const data = await safeFetchJson("/api/conversations");
     setConversations(data || []);
     if (!activeChat && data?.length) {
       setActiveChat(data[0].id);
     }
   } catch (error) {
     console.error("FETCH CONVERSATIONS ERROR:", error);
     setConversations([]);
   } finally {
     setLoadingChats(false);
   }
 };
```

```
 useEffect(() => {
   fetchConversations();
 }, []);

 const fetchMessages = async (conversationId: string) => {
   try {
     setLoadingMessages(true);
     const data = await
safeFetchJson(`/api/conversations/${conversationId}/messages`);
     setMessages(data || []);
   } catch (error) {
     console.error("FETCH MESSAGES ERROR:", error);
     setMessages([]);
   } finally {
     setLoadingMessages(false);
   }
 };

 useEffect(() => {
   if (activeChat) {
     fetchMessages(activeChat);
   }
 }, [activeChat]);

 const filteredConversations = useMemo(() => {
   const q = search.trim().toLowerCase();
   if (!q) return conversations;

   return conversations.filter((conversation) => {
     const name = `${conversation.seekerFirstName}
${conversation.seekerLastName}`.toLowerCase();
     return name.includes(q);
```

```
    });
  }, [conversations, search]);

  const activeConversation = conversations.find((c) => c.id ===
activeChat);

  const sendMessage = async () => {
    if (!activeChat || (!message.trim() && !attachment)) return;

    try {
      setSending(true);

      const formData = new FormData();
      formData.append("text", message.trim());
      if (attachment) {
        formData.append("attachment", attachment);
      }

      await
safeFetchJson(`/api/conversations/${activeChat}/messages`, {
        method: "POST",
        body: formData,
      });

      setMessage("");
      setAttachment(null);
      if (attachmentInputRef.current)
attachmentInputRef.current.value = "";

      await fetchMessages(activeChat);
      await fetchConversations();
    } catch (error: any) {
```

```
    console.error("SEND MESSAGE ERROR:", error);
    alert(error.message || "Failed to send message");
  } finally {
    setSending(false);
  }
};

const saveEdit = async () => {
  if (!editingId) return;

  try {
    await safeFetchJson(`/api/messages/${editingId}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text: editingText }),
    });

    setEditingId(null);
    setEditingText("");
    await fetchMessages(activeChat);
  } catch (error: any) {
    console.error("EDIT MESSAGE ERROR:", error);
    alert(error.message || "Failed to edit message");
  }
};

const deleteForMe = async (messageId: string) => {
  try {
    await safeFetchJson(`/api/messages/${messageId}`, {
      method: "DELETE",
    });
```

```
      await fetchMessages(activeChat);
    } catch (error: any) {
      console.error("DELETE MESSAGE ERROR:", error);
      alert(error.message || "Failed to delete message");
    }
  };

  return (
    <div className="min-h-screen bg-[#f0f4f3] font-sans">
      <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
        <div className="flex items-center gap-2.5">
          <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
          <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
        </div>

        <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
          <Link href="/dashboard"><button
className="text-gray-300 hover:text-white
transition-colors">Dashboard</button></Link>
          <Link href="/post_job"><button className="text-gray-300
hover:text-white transition-colors">Post Job</button></Link>
          <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Messages</button>
          <Link href="/employer_notification"><button
className="text-gray-300 hover:text-white
transition-colors">Notification</button></Link>
```

```
        <Link href="/subscription"><button
className="text-gray-300 hover:text-white
transition-colors">Subscription</button></Link>
        <Link href="/employer_setting"><button
className="text-gray-300 hover:text-white
transition-colors">Settings</button></Link>
      </nav>

      <Link href="/employer_profile">
        <div className="flex items-center gap-3 cursor-pointer
group">
          <div className="text-right">
            <p className="text-[10px] text-gray-500 uppercase
tracking-widest">Company</p>
            <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#40b594] rounded-full
flex items-center justify-center font-extrabold text-[#051612]
text-sm">
            C
          </div>
        </div>
      </Link>
    </header>

    <main className="mx-auto max-w-7xl px-6 py-10 md:px-10">
      <div className="mb-8">
        <p className="mb-1 text-xs font-bold uppercase
tracking-widest text-[#40b594]">Inbox</p>
        <h1 className="text-4xl font-extrabold
text-[#071a15]">Messages</h1>
```

```
          <p className="mt-1 font-medium text-[#4a5a55]">
            Communicate with applicants directly
          </p>
        </div>

        <div className="flex h-[680px] overflow-hidden
rounded-2xl border border-gray-100 bg-white shadow-sm">
          <div className="flex w-80 flex-shrink-0 flex-col
border-r border-gray-100">
            <div className="border-b border-gray-100 p-5">
              <div className="relative">
                <Search className="absolute left-3.5 top-1/2
-translate-y-1/2 text-[#6b7f79]" size={16} />
                <input
                  type="text"
                  placeholder="Search conversations..."
                  value={search}
                  onChange={(e) => setSearch(e.target.value)}
                  className="w-full rounded-xl bg-[#f0f4f3]
py-2.5 pl-10 pr-4 text-sm font-medium text-[#071a15]
placeholder-[#6b7f79] focus:outline-none focus:ring-2
focus:ring-[#40b594]/30"
                />
              </div>
            </div>

            <div className="px-5 py-3">
              <p className="text-[10px] font-bold uppercase
tracking-widest text-[#6b7f79]">
                Recent
              </p>
            </div>
```

```
          <div className="flex-1 overflow-y-auto">
            {loadingChats ? (
              <div className="px-5 py-4 text-sm
text-[#6b7f79]">Loading conversations...</div>
            ) : filteredConversations.length === 0 ? (
              <div className="px-5 py-4 text-sm
text-[#6b7f79]">No conversations yet.</div>
            ) : (
              filteredConversations.map((chat) => {
                const isActive = activeChat === chat.id;
                const displayName = `${chat.seekerFirstName}
${chat.seekerLastName}`.trim();

                return (
                  <div
                    key={chat.id}
                    onClick={() => setActiveChat(chat.id)}
                    className={`cursor-pointer border-l-4 px-5
py-4 transition-all ${
                      isActive
                        ? "border-l-[#40b594] bg-[#f0f9f6]"
                        : "border-l-transparent
hover:bg-[#f8faf9]"
                    }`}
                  >
                    <div className="flex items-center gap-4">
                      <div className="relative flex-shrink-0">
                        <div className="flex h-12 w-12
items-center justify-center rounded-full bg-[#051612] text-base
font-extrabold text-white">
                          {(displayName || "A").charAt(0)}
```

```
                        </div>
                        <span className="absolute bottom-0
right-0 h-3 w-3 rounded-full border-2 border-white bg-[#40b594]"
/>
                      </div>

                      <div className="min-w-0 flex-1">
                        <div className="mb-0.5 flex
items-center justify-between">
                          <h3 className={`truncate text-sm
font-extrabold ${isActive ? "text-[#071a15]" :
"text-[#1a2e29]"}`}>
                            {displayName}
                          </h3>
                          <span className="ml-2 flex-shrink-0
text-[10px] font-semibold text-[#6b7f79]">

{formatConversationTime(chat.lastMessageAt)}
                          </span>
                        </div>

                        <p className="truncate text-xs
font-bold tracking-wide text-[#40b594]">
                          Applicant
                        </p>

                        <p className="mt-1 text-[11px]
font-semibold text-[#7a8b86]">
                          Active
                        </p>
                      </div>
                    </div>
```

```
                  </div>
                );
              })
            )}
          </div>
        </div>

        <div className="flex min-w-0 flex-1 flex-col">
          <div className="border-b border-gray-100 bg-white
px-7 py-5">
            {activeConversation ? (
              <div className="flex items-center gap-4">
                <div className="relative flex-shrink-0">
                  <div className="flex h-12 w-12 items-center
justify-center rounded-full bg-[#051612] text-sm font-extrabold
text-white">

{activeConversation.seekerFirstName.charAt(0)}
                  </div>
                  <span className="absolute bottom-0 right-0
h-3 w-3 rounded-full border-2 border-white bg-[#40b594]" />
                </div>

                <div className="min-w-0">
                  <h3 className="truncate text-base
font-extrabold text-[#071a15]">
                    {activeConversation.seekerFirstName}
{activeConversation.seekerLastName}
                  </h3>
                  <p className="truncate text-sm font-medium
text-[#4f655f]">Applicant</p>
```

```
                  <p className="mt-1 text-[11px] font-bold
uppercase tracking-wider text-[#40b594]">
                    Active
                  </p>
                </div>
              </div>
            ) : (
              <p className="text-sm text-[#6b7f79]">Select a
conversation</p>
            )}
          </div>

          <div className="flex-1 space-y-6 overflow-y-auto
bg-[#f8faf9] px-7 py-6">
            {loadingMessages ? (
              <div className="text-sm text-[#6b7f79]">Loading
messages...</div>
            ) : messages.length === 0 ? (
              <div className="text-sm text-[#6b7f79]">No
messages yet.</div>
            ) : (
              <>
                <div className="flex items-center gap-4">
                  <div className="h-px flex-1 bg-gray-200" />
                  <span className="text-[10px] font-bold
uppercase tracking-widest text-[#6b7f79]">
                    Conversation
                  </span>
                  <div className="h-px flex-1 bg-gray-200" />
                </div>

                {messages.map((msg) => {
```

```
                    const mine = me?.userId === msg.senderId;

                    if (editingId === msg.id) {
                      return (
                        <div key={msg.id} className="flex
flex-col items-end gap-2">
                          <div className="w-full max-w-md
rounded-2xl border border-gray-200 bg-white p-4 shadow-sm">
                            <textarea
                              value={editingText}
                              onChange={(e) =>
setEditingText(e.target.value)}
                              rows={3}
                              className="w-full resize-none
bg-transparent text-sm text-[#071a15] outline-none"
                            />
                            <div className="mt-3 flex justify-end
gap-2">
                              <button
                                onClick={() => {
                                  setEditingId(null);
                                  setEditingText("");
                                }}
                                className="rounded-xl border
border-gray-200 px-3 py-2 text-xs font-bold text-[#6b7f79]"
                              >
                                Cancel
                              </button>
                              <button
                                onClick={saveEdit}
                                className="rounded-xl
bg-[#051612] px-3 py-2 text-xs font-bold text-white"
```

```
                              >
                                Save
                              </button>
                            </div>
                          </div>
                        </div>
                      );
                    }

                    return mine ? (
                      <div key={msg.id} className="flex flex-col
items-end gap-1">
                        <div className="max-w-sm rounded-2xl
rounded-br-md border border-gray-200 bg-white px-5 py-3.5
shadow-sm">
                          {msg.text && (
                            <p className="text-sm leading-relaxed
text-[#071a15]">{msg.text}</p>
                          )}
                          {msg.attachmentUrl && (
                            <a
                              href={msg.attachmentUrl}
                              target="_blank"
                              rel="noreferrer"
                              className="mt-2 block text-xs
font-bold text-[#40b594] underline"
                            >
                              {msg.attachmentName || "Open
attachment"}
                            </a>
                          )}
                          {msg.editedAt && (
```

```
                            <p className="mt-2 text-[10px]
font-semibold text-[#6b7f79]">Edited</p>
                          )}
                        </div>
                        <div className="mr-1 flex items-center
gap-2">
                          <span className="text-[10px]
font-semibold text-[#6b7f79]">
                            {formatTime(msg.createdAt)}
                          </span>
                          {canEdit(msg.createdAt) && (
                            <button
                              onClick={() => {
                                setEditingId(msg.id);
                                setEditingText(msg.text || "");
                              }}
                              className="text-[#6b7f79]
hover:text-[#071a15]"
                            >
                              <Pencil size={12} />
                            </button>
                          )}
                          <button
                            onClick={() => deleteForMe(msg.id)}
                            className="text-[#6b7f79]
hover:text-red-500"
                          >
                            <Trash2 size={12} />
                          </button>
                        </div>
                      </div>
                    ) : (
```

```
                    <div key={msg.id} className="flex items-end
gap-3">
                      <div className="flex h-8 w-8
flex-shrink-0 items-center justify-center rounded-full
bg-[#051612] text-xs font-extrabold text-white">

{activeConversation?.seekerFirstName.charAt(0)}
                      </div>
                      <div className="flex flex-col items-start
gap-1">
                        <div className="max-w-sm rounded-2xl
rounded-bl-md bg-[#051612] px-5 py-3.5 text-white shadow-sm">
                          {msg.text && <p className="text-sm
leading-relaxed">{msg.text}</p>}
                          {msg.attachmentUrl && (
                            <a
                              href={msg.attachmentUrl}
                              target="_blank"
                              rel="noreferrer"
                              className="mt-2 block text-xs
font-bold text-[#40b594] underline"
                            >
                              {msg.attachmentName || "Open
attachment"}
                            </a>
                          )}
                        </div>
                        <span className="ml-1 text-[10px]
font-semibold text-[#6b7f79]">
                          {formatTime(msg.createdAt)}
                        </span>
                      </div>
```

```
                    </div>
                  );
                })}
              </>
            )}
          </div>

          <div className="border-t border-gray-100 bg-white
px-7 py-5">
            {attachment && (
              <div className="mb-3 flex items-center
justify-between rounded-xl border border-gray-200 bg-[#f8faf9]
px-4 py-2">
                <p className="truncate text-xs font-semibold
text-[#071a15]">{attachment.name}</p>
                <button
                  onClick={() => {
                    setAttachment(null);
                    if (attachmentInputRef.current)
attachmentInputRef.current.value = "";
                  }}
                  className="text-[#6b7f79] hover:text-red-500"
                >
                  <X size={14} />
                </button>
              </div>
            )}

            <div className="flex items-center gap-3 rounded-2xl
border border-gray-200 bg-[#f0f4f3] px-4 py-3 transition-all
focus-within:border-[#40b594] focus-within:ring-2
focus-within:ring-[#40b594]/20">
```

```
                <button
                  className="flex-shrink-0 text-[#6b7f79]
transition-colors hover:text-[#071a15]"
                  onClick={() =>
attachmentInputRef.current?.click()}
                >
                  <Paperclip size={19} />
                </button>

                <input
                  ref={attachmentInputRef}
                  type="file"
                  className="hidden"
                  onChange={(e) =>
setAttachment(e.target.files?.[0] || null)}
                />

                <input
                  type="text"
                  placeholder="Write a message..."
                  value={message}
                  onChange={(e) => setMessage(e.target.value)}
                  onKeyDown={(e) => {
                    if (e.key === "Enter") sendMessage();
                  }}
                  className="flex-1 border-none bg-transparent
text-sm font-medium text-[#071a15] placeholder-[#6b7f79]
outline-none"
                />
                <button
                  onClick={sendMessage}
```

```
                disabled={sending || (!message.trim() &&
!attachment)}
                className={`flex-shrink-0 rounded-xl p-2
transition-all ${
                  message.trim() || attachment
                    ? "bg-[#051612] text-white shadow-sm
hover:bg-[#0d2a23]"
                    : "cursor-not-allowed bg-[#d1e8e3]
text-[#6b7f79]"
                }`}
              >
                <Send size={17} />
              </button>
            </div>
          </div>
        </div>
      </div>
    </main>
  </div>
 );
}

"use client";

import React, { useEffect, useState } from "react";
import {
 CheckCircle,
 Clock,
 MoreVertical,
 ArrowRight,
 Bell,
 User,
```

```
 Zap,
 Send,
} from "lucide-react";
import Link from "next/link";

type NotificationItem = {
 id: string;
 type: "Applicant" | "NexHire" | "Interview" | "Application" |
"System";
 title: string;
 description: string;
 time: string;
 actionText: string;
 read: boolean;
};

const formatRelativeTime = (dateString: string) => {
 const date = new Date(dateString);
 const diffMs = Date.now() - date.getTime();

 const minutes = Math.floor(diffMs / (1000 * 60));
 const hours = Math.floor(diffMs / (1000 * 60 * 60));
 const days = Math.floor(diffMs / (1000 * 60 * 60 * 24));

 if (minutes < 1) return "Just now";
 if (minutes < 60) return `${minutes}m ago`;
 if (hours < 24) return `${hours}h ago`;
 return `${days}d ago`;
};

const normalizeType = (type: string): NotificationItem["type"]
=> {
```

```
 if (type === "application") return "Application";
 if (type === "interview") return "Interview";
 if (type === "system") return "System";
 if (type === "message") return "Applicant";
 return "NexHire";
};

const Notifications = () => {
 const [notifications, setNotifications] =
useState<NotificationItem[]>([]);
 const [loading, setLoading] = useState(true);

 const [form, setForm] = useState({
   applicationId: "",
   type: "application",
   title: "",
   description: "",
   link: "/notification",
 });

 const [sending, setSending] = useState(false);

 const unreadCount = notifications.filter((n) =>
!n.read).length;

 const fetchNotifications = async () => {
   try {
     setLoading(true);

     const res = await fetch("/api/notifications", { cache:
"no-store" });
     const text = await res.text();
```

```
    let data: any = [];
    try {
      data = text ? JSON.parse(text) : [];
    } catch {
      console.error("Invalid notifications response:", text);
      setNotifications([]);
      return;
    }

    if (res.status === 401) {
      setNotifications([]);
      return;
    }

    if (!res.ok) {
      console.error(data.error || "Failed to fetch
notifications");
      setNotifications([]);
      return;
    }

    const mapped: NotificationItem[] = (data || []).map((notif:
any) => ({
      id: notif.id,
      type: normalizeType(notif.type),
      title: notif.title,
      description: notif.description,
      time: formatRelativeTime(notif.createdAt),
      actionText:
        notif.type === "interview"
          ? "View Interview"
```

```
          : notif.type === "application"
            ? "View Application"
            : "Open",
      read: Boolean(notif.readAt),
    }));

    setNotifications(mapped);
  } catch (error) {
    console.error("FETCH NOTIFICATIONS ERROR:", error);
    setNotifications([]);
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchNotifications();
}, []);

const markAllRead = () => {
  setNotifications((prev) => prev.map((n) => ({ ...n, read:
true })));
};

const markOneRead = (id: string) => {
  setNotifications((prev) =>
    prev.map((n) => (n.id === id ? { ...n, read: true } : n))
  );
};

const sendEmployerNotification = async () => {
```

```
  if (!form.applicationId.trim() || !form.title.trim() ||
!form.description.trim()) {
    alert("Please fill application ID, title, and
description");
    return;
  }

  try {
    setSending(true);

    const res = await fetch("/api/notifications", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        applicationId: form.applicationId,
        type: form.type,
        title: form.title,
        description: form.description,
        link: form.link,
      }),
    });

    const text = await res.text();

    let data: any = null;
    try {
      data = text ? JSON.parse(text) : null;
    } catch {
      console.error("Invalid send notification response:",
text);
```

```
      alert("Server returned invalid response");
      return;
    }

    if (!res.ok) {
      alert(data?.error || "Failed to send notification");
      return;
    }

    alert("Notification sent successfully");

    setForm({
      applicationId: "",
      type: "application",
      title: "",
      description: "",
      link: "/notification",
    });

    fetchNotifications();
  } catch (error) {
    console.error("SEND NOTIFICATION ERROR:", error);
    alert("Something went wrong");
  } finally {
    setSending(false);
  }
};

const typeStyle: Record<
  NotificationItem["type"],
  { badge: string; iconBg: string; iconColor: string; icon:
React.ElementType }
```

```
> = {
  Applicant: {
    badge: "bg-blue-50 text-blue-700 ring-1 ring-blue-200",
    iconBg: "bg-[#051612]",
    iconColor: "text-[#40b594]",
    icon: User,
  },
  NexHire: {
    badge: "bg-emerald-50 text-emerald-700 ring-1
ring-emerald-200",
    iconBg: "bg-[#40b594]",
    iconColor: "text-[#051612]",
    icon: Zap,
  },
  Interview: {
    badge: "bg-purple-50 text-purple-700 ring-1
ring-purple-200",
    iconBg: "bg-[#051612]",
    iconColor: "text-[#40b594]",
    icon: Bell,
  },
  Application: {
    badge: "bg-amber-50 text-amber-700 ring-1 ring-amber-200",
    iconBg: "bg-[#051612]",
    iconColor: "text-[#40b594]",
    icon: User,
  },
  System: {
    badge: "bg-gray-100 text-gray-700 ring-1 ring-gray-200",
    iconBg: "bg-[#40b594]",
    iconColor: "text-[#051612]",
    icon: Zap,
```

```
    },
  };

  return (
    <div className="min-h-screen font-sans pb-16" style={{
background: "#f0f4f3" }}>
      <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
        <div className="flex items-center gap-2.5">
          <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
          <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
        </div>

        <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
          <Link href="/dashboard">
            <button className="text-gray-300 hover:text-white
transition-colors">
              Dashboard
            </button>
          </Link>
          <Link href="/post_job">
            <button className="text-gray-300 hover:text-white
transition-colors">
              Post Job
            </button>
          </Link>
          <Link href="/employer_message">
            <button className="text-gray-300 hover:text-white
transition-colors">
```

```
            Messages
          </button>
        </Link>
        <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">
          Notification
        </button>
        <Link href="/subscription">
          <button className="text-gray-300 hover:text-white
transition-colors">
            Subscription
          </button>
        </Link>
        <Link href="/employer_setting">
          <button className="text-gray-300 hover:text-white
transition-colors">
            Settings
          </button>
        </Link>
      </nav>

      <Link href="/employer_profile">
        <div className="flex items-center gap-3 cursor-pointer
group">
          <div className="text-right">
            <p className="text-[10px] text-gray-500 uppercase
tracking-widest">
              Company
            </p>
            <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors">
              Profile
```

```
            </p>
          </div>
          <div className="w-10 h-10 bg-[#40b594] rounded-full
flex items-center justify-center font-extrabold text-[#051612]
text-sm">
            C
          </div>
        </div>
      </Link>
    </header>

    <main className="max-w-4xl mx-auto px-8 py-10">
      <div className="flex justify-between items-start mb-10">
        <div>
          <p className="text-xs font-bold uppercase
tracking-widest text-[#40b594] mb-1">
            Activity
          </p>
          <h1 className="text-4xl font-extrabold text-[#071a15]
leading-tight">
            Notifications
          </h1>
          <p className="text-[#4a5a55] font-medium mt-1">
            Keep track of your hiring activity
          </p>
        </div>

        <div className="flex items-center gap-3 mt-2">
          {unreadCount > 0 && (
            <div className="flex items-center gap-2 bg-white
border border-gray-200 rounded-xl px-4 py-2 shadow-sm">
              <Bell size={15} className="text-[#40b594]" />
```

```
                <span className="text-sm font-extrabold
text-[#071a15]">
                  {unreadCount} unread
                </span>
              </div>
            )}
            <button
              onClick={markAllRead}
              className="flex items-center gap-2 bg-white border
border-gray-200 text-[#071a15] px-4 py-2 rounded-xl font-bold
text-sm hover:border-[#40b594] hover:text-[#40b594]
transition-all shadow-sm"
            >
              <CheckCircle size={16} />
              Mark all read
            </button>
          </div>
        </div>

        <div className="bg-white rounded-2xl border
border-gray-100 shadow-sm p-7 mb-8">
          <div className="flex items-center justify-between
mb-5">
            <div>
              <h2 className="text-lg font-extrabold
text-[#071a15]">
                Send Notification To Job Seeker
              </h2>
              <p className="text-sm text-[#6b7f79] mt-1">
                Use application ID to send accept, reject, or
interview notice.
              </p>
```

```
          </div>
          <div className="w-10 h-10 rounded-xl bg-[#f0f9f6]
flex items-center justify-center border border-[#d1e8e3]">
            <Send size={18} className="text-[#40b594]" />
          </div>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
          <div>
            <label className="block text-sm font-bold
text-[#071a15] mb-2">
              Application ID
            </label>
            <input
              type="text"
              value={form.applicationId}
              onChange={(e) =>
                setForm((prev) => ({ ...prev, applicationId:
e.target.value }))
              }
              placeholder="Enter application ID"
              className="w-full rounded-xl border
border-gray-200 bg-[#f8faf9] px-4 py-3 text-sm text-[#071a15]
focus:outline-none focus:ring-2 focus:ring-[#40b594]/30
focus:border-[#40b594]"
            />
          </div>

          <div>
            <label className="block text-sm font-bold
text-[#071a15] mb-2">
              Notification Type
```

```
            </label>
            <select
              value={form.type}
              onChange={(e) =>
                setForm((prev) => ({ ...prev, type:
e.target.value }))
              }
              className="w-full rounded-xl border
border-gray-200 bg-[#f8faf9] px-4 py-3 text-sm text-[#071a15]
focus:outline-none focus:ring-2 focus:ring-[#40b594]/30
focus:border-[#40b594]"
            >
              <option value="application">Application</option>
              <option value="interview">Interview</option>
              <option value="system">System</option>
            </select>
          </div>

          <div className="md:col-span-2">
            <label className="block text-sm font-bold
text-[#071a15] mb-2">
              Title
            </label>
            <input
              type="text"
              value={form.title}
              onChange={(e) =>
                setForm((prev) => ({ ...prev, title:
e.target.value }))
              }
              placeholder="Example: Your application has been
accepted"
```

```
              className="w-full rounded-xl border
border-gray-200 bg-[#f8faf9] px-4 py-3 text-sm text-[#071a15]
focus:outline-none focus:ring-2 focus:ring-[#40b594]/30
focus:border-[#40b594]"
            />
          </div>

          <div className="md:col-span-2">
            <label className="block text-sm font-bold
text-[#071a15] mb-2">
              Description
            </label>
            <textarea
              rows={4}
              value={form.description}
              onChange={(e) =>
                setForm((prev) => ({ ...prev, description:
e.target.value }))
              }
              placeholder="Write the message for the candidate"
              className="w-full rounded-xl border
border-gray-200 bg-[#f8faf9] px-4 py-3 text-sm text-[#071a15]
resize-none focus:outline-none focus:ring-2
focus:ring-[#40b594]/30 focus:border-[#40b594]"
            />
          </div>
        </div>

        <div className="mt-5 flex justify-end">
          <button
            onClick={sendEmployerNotification}
            disabled={sending}
```

```
            className="flex items-center gap-2 bg-[#051612]
text-white px-5 py-3 rounded-xl font-bold text-sm
hover:bg-[#0d2a23] transition-all shadow-sm disabled:opacity-60"
          >
            <Send size={16} />
            {sending ? "Sending..." : "Send Notification"}
          </button>
        </div>
      </div>

      <div className="space-y-4">
        {loading ? (
          <div className="bg-white rounded-2xl border
border-gray-100 p-8 text-center text-[#6b7f79] font-medium
shadow-sm">
            Loading notifications...
          </div>
        ) : notifications.length > 0 ? (
          notifications.map((notif) => {
            const style = typeStyle[notif.type];
            const Icon = style.icon;

            return (
              <div
                key={notif.id}
                className={`bg-white rounded-2xl border
transition-all shadow-sm overflow-hidden ${
                  notif.read
                    ? "border-gray-100"
                    : "border-l-4 border-l-[#40b594]
border-t-gray-100 border-r-gray-100 border-b-gray-100"
                } hover:shadow-md hover:border-l-[#40b594]`}
```

```
                >
                  <div className="p-7">
                    <div className="flex items-start
justify-between mb-4">
                      <div className="flex items-center gap-3">
                        <div
                          className={`w-10 h-10 rounded-xl
${style.iconBg} flex items-center justify-center flex-shrink-0`}
                        >
                          <Icon size={18}
className={style.iconColor} />
                        </div>

                        <div>
                          <span
                            className={`text-xs font-extrabold
px-2.5 py-1 rounded-full ${style.badge}`}
                          >
                            {notif.type}
                          </span>
                          <div className="flex items-center
gap-1.5 mt-1.5">
                            <Clock size={12}
className="text-[#6b7f79]" />
                            <span className="text-xs
font-semibold text-[#6b7f79]">
                              {notif.time}
                            </span>
                          </div>
                        </div>
                      </div>
```

```
                    <div className="flex items-center gap-2">
                      {!notif.read && (
                        <span className="w-2.5 h-2.5
rounded-full bg-[#40b594] flex-shrink-0" />
                      )}
                      <button
                        onClick={() => markOneRead(notif.id)}
                        className="p-1.5 rounded-lg
text-[#6b7f79] hover:bg-[#f0f4f3] hover:text-[#071a15]
transition-all"
                      >
                        <MoreVertical size={18} />
                      </button>
                    </div>
                  </div>

                  <div className="pl-13 ml-1">
                    <h3 className="text-base font-extrabold
text-[#071a15] leading-snug mb-1">
                      {notif.title}
                    </h3>
                    <p className="text-sm text-[#4a5a55]
font-medium leading-relaxed">
                      {notif.description}
                    </p>
                  </div>

                  <div className="mt-5 flex items-center
justify-between">
                    <button className="flex items-center gap-2
bg-[#051612] text-white px-5 py-2.5 rounded-xl font-bold text-sm
hover:bg-[#0d2a23] transition-all shadow-sm">
```

```
                        {notif.actionText}
                        <ArrowRight size={16} />
                      </button>

                      {!notif.read && (
                        <button
                          onClick={() => markOneRead(notif.id)}
                          className="text-xs font-bold
text-[#6b7f79] hover:text-[#40b594] transition-colors"
                        >
                          Mark as read
                        </button>
                      )}
                    </div>
                  </div>
                </div>
              );
            })
          ) : (
            <div className="text-center py-24">
              <div className="w-16 h-16 bg-white rounded-2xl flex
items-center justify-center mx-auto mb-4 shadow-sm border
border-gray-100">
                <Bell size={28} className="text-[#40b594]" />
              </div>
              <p className="text-lg font-extrabold
text-[#071a15]">All caught up!</p>
              <p className="text-sm text-[#4a5a55] mt-1">
                No new notifications right now.
              </p>
            </div>
          )}
```

```
        </div>
      </main>
    </div>
  );
};

export default Notifications;

// app/employer_profile/page.tsx
"use client";

import React from "react";
import Link from "next/link";
import {
  Globe,
  MapPin,
  Users,
  Mail,
  Phone,
  Briefcase,
  CheckCircle2,
  Building2,
  Pencil,
  TrendingUp,
  Save,
  X,
  Sparkles,
  Camera,
  Trash2,
  FileText,
  Upload,
} from "lucide-react";
```

```
import ImageCropModal from "@/components/ui/ImageCropModal";

type EmployerForm = {
 companyDescription: string;
 companyName: string;
 industry: string;
 companySize: string;
 currentAddress: string;
 foundedYear: string;
 country: string;
 contact: string;
 websiteLink: string;
 companyFileName: string;
 companyFileUrl: string;
 profileImageUrl: string;
};

const cardClass =
 "rounded-[28px] border border-[#e6eeea] bg-white
shadow-[0_18px_50px_rgba(7,28,22,0.06)]";

export default function EmployerProfile() {
 const fileInputRef = React.useRef<HTMLInputElement |
null>(null);
 const companyFileInputRef = React.useRef<HTMLInputElement |
null>(null);

 const [activeTab, setActiveTab] = React.useState<"overview" |
"details">("overview");
 const [isEditing, setIsEditing] = React.useState(false);
 const [loading, setLoading] = React.useState(false);
```

```
 const [profileImage, setProfileImage] = React.useState<File |
null>(null);
 const [sourceImageUrl, setSourceImageUrl] =
React.useState<string | null>(null);
 const [croppedPreviewUrl, setCroppedPreviewUrl] =
React.useState<string | null>(null);
 const [cropOpen, setCropOpen] = React.useState(false);

 const [companyFile, setCompanyFile] = React.useState<File |
null>(null);

 const [saved, setSaved] = React.useState<EmployerForm>({
   companyDescription:
     "NexHire is a modern hiring company focused on helping
employers find qualified talent faster with a cleaner and more
human recruitment experience.",
   companyName: "NexHire Solutions",
   industry: "Technology & Software",
   companySize: "50 - 100 Employees",
   currentAddress: "Phnom Penh, Cambodia",
   foundedYear: "2021",
   country: "Cambodia",
   contact: "+855 23 456 789",
   websiteLink: "https://www.nexhire.com",
   companyFileName: "",
   companyFileUrl: "",
   profileImageUrl: "",
 });

 const [draft, setDraft] = React.useState<EmployerForm>(saved);

 const stats = [
```

```
  { label: "Active Jobs", value: "12", icon: Briefcase },
  { label: "Hires", value: "148", icon: Users },
  { label: "Growth", value: "+26%", icon: TrendingUp },
];

const openJobs = [
  { title: "Senior Software Engineer", type: "Full-time",
salary: "$2,000 - $3,500" },
  { title: "Product Designer", type: "Full-time", salary:
"$1,500 - $2,500" },
  { title: "Data Analyst", type: "Contract", salary: "$1,200 -
$2,000" },
];

const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
) => {
  setDraft((prev) => ({
    ...prev,
    [e.target.name]: e.target.value,
  }));
};

const openFilePicker = () => fileInputRef.current?.click();
const openCompanyFilePicker = () =>
companyFileInputRef.current?.click();

const handleCircleAction = () => {
  if (sourceImageUrl || saved.profileImageUrl) {
    setCropOpen(true);
    return;
```

```
    }
    openFilePicker();
  };

  const handleDeleteImage = () => {
    if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
    if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

    setSourceImageUrl(null);
    setCroppedPreviewUrl(null);
    setProfileImage(null);
    setCropOpen(false);

    setSaved((prev) => ({ ...prev, profileImageUrl: "" }));
    setDraft((prev) => ({ ...prev, profileImageUrl: "" }));

    if (fileInputRef.current) fileInputRef.current.value = "";
  };

  const handleImageChange = (e:
React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0] || null;
    if (!file) return;

    if (!["image/jpeg", "image/png",
"image/webp"].includes(file.type)) {
      alert("Only JPG, PNG, and WEBP images are allowed");
      e.target.value = "";
      return;
    }
```

```
   if (file.size > 5 * 1024 * 1024) {
     alert("Image must be 5MB or smaller");
     e.target.value = "";
     return;
   }

   if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

   const objectUrl = URL.createObjectURL(file);
   setSourceImageUrl(objectUrl);
   setCroppedPreviewUrl(null);
   setProfileImage(null);
   setCropOpen(true);

   e.target.value = "";
 };

 const handleCropSave = (file: File, previewUrl: string) => {
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
   setProfileImage(file);
   setCroppedPreviewUrl(previewUrl);
   setCropOpen(false);
 };

 const handleCropClose = () => setCropOpen(false);

 const handleCompanyFileChange = (e:
React.ChangeEvent<HTMLInputElement>) => {
   const file = e.target.files?.[0] || null;
```

```
  if (!file) return;

  const allowedTypes = [
    "application/pdf",
    "application/msword",

"application/vnd.openxmlformats-officedocument.wordprocessingml.
document",
  ];

  if (!allowedTypes.includes(file.type)) {
    alert("Only PDF, DOC, and DOCX files are allowed");
    e.target.value = "";
    return;
  }

  if (file.size > 10 * 1024 * 1024) {
    alert("File must be 10MB or smaller");
    e.target.value = "";
    return;
  }

  setCompanyFile(file);
  setDraft((prev) => ({
    ...prev,
    companyFileName: file.name,
  }));

  e.target.value = "";
};

const handleRemoveCompanyFile = () => {
```

```
   setCompanyFile(null);
   setDraft((prev) => ({
     ...prev,
     companyFileName: "",
     companyFileUrl: "",
   }));
   setSaved((prev) => ({
     ...prev,
     companyFileName: "",
     companyFileUrl: prev.companyFileName ===
draft.companyFileName ? "" : prev.companyFileUrl,
   }));

   if (companyFileInputRef.current)
companyFileInputRef.current.value = "";
 };

 React.useEffect(() => {
   return () => {
     if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
     if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
   };
 }, [sourceImageUrl, croppedPreviewUrl]);

 const handleEdit = () => {
   setDraft(saved);
   setIsEditing(true);
 };

 const handleCancel = () => {
   setDraft(saved);
```

```
    setCompanyFile(null);
    setIsEditing(false);
  };

  const handleSave = async () => {
    try {
      setLoading(true);

      const formData = new FormData();
      formData.append("companyDescription",
draft.companyDescription);
      formData.append("companyName", draft.companyName);
      formData.append("industry", draft.industry);
      formData.append("companySize", draft.companySize);
      formData.append("currentAddress", draft.currentAddress);
      formData.append("foundedYear", draft.foundedYear);
      formData.append("country", draft.country);
      formData.append("contact", draft.contact);
      formData.append("websiteLink", draft.websiteLink);

      if (profileImage) {
        formData.append("profileImage", profileImage);
      }

      if (companyFile) {
        formData.append("companyFile", companyFile);
      }

      const res = await fetch("/api/auth/employer-profile", {
        method: "POST",
        body: formData,
      });
```

```
      const text = await res.text();
      const result = text ? JSON.parse(text) : {};

      if (!res.ok) {
        alert(result.error || "Failed to update profile");
        return;
      }

      setSaved({
        ...draft,
        companyFileUrl: result.companyFileUrl ||
draft.companyFileUrl,
        companyFileName: result.companyFileName ||
draft.companyFileName,
        profileImageUrl: result.profileImageUrl ||
draft.profileImageUrl || saved.profileImageUrl,
      });
      setCompanyFile(null);
      setIsEditing(false);
    } catch (error) {
      console.error("EMPLOYER UPDATE ERROR:", error);
      alert("Something went wrong");
    } finally {
      setLoading(false);
    }
  };

  const profile = isEditing ? draft : saved;
  const displayImageSrc = croppedPreviewUrl || sourceImageUrl ||
saved.profileImageUrl || "";
```

```
 const completionFields = [
   saved.companyDescription,
   saved.companyName,
   saved.industry,
   saved.companySize,
   saved.currentAddress,
   saved.foundedYear,
   saved.country,
   saved.contact,
   saved.websiteLink,
   saved.companyFileName || saved.companyFileUrl,
   saved.profileImageUrl || croppedPreviewUrl || sourceImageUrl,
 ];
 const completion = Math.round(
   (completionFields.filter(Boolean).length /
completionFields.length) * 100
 );

 return (
   <>
     <div className="min-h-screen bg-[#edf4f1] pb-16 font-sans
text-[#10211d]">
       <header className="sticky top-0 z-50 flex items-center
justify-between bg-[#051612] px-8 py-4 text-white shadow-lg">
         <div className="flex items-center gap-2.5">
           <img src="/logo.png" alt="NexHire" className="h-8
w-8" />
           <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
         </div>
```

```
        <nav className="hidden items-center gap-8 text-sm
font-semibold md:flex">
          <Link href="/dashboard"><button
className="text-gray-300
hover:text-white">Dashboard</button></Link>
          <Link href="/post_job"><button
className="text-gray-300 hover:text-white">Post
Job</button></Link>
          <Link href="/employer_message"><button
className="text-gray-300
hover:text-white">Messages</button></Link>
          <Link href="/employer_notification"><button
className="text-gray-300
hover:text-white">Notification</button></Link>
          <Link href="/employer_setting"><button
className="text-gray-300
hover:text-white">Settings</button></Link>
        </nav>

        <Link href="/employer_profile">
          <div className="group flex cursor-pointer
items-center gap-3">
            <div className="text-right">
              <p className="text-[10px] uppercase
tracking-widest text-gray-500">Company</p>
              <p className="text-sm font-bold text-white
group-hover:text-[#40b594]">Profile</p>
            </div>
            <div className="flex h-10 w-10 items-center
justify-center rounded-full bg-[#40b594] text-sm font-extrabold
text-[#051612]">
              {saved.companyName.charAt(0) || "C"}
```

```
            </div>
          </div>
        </Link>
      </header>

      <section className="relative overflow-hidden
bg-[#051612]">
        <div className="absolute inset-0 opacity-15">
          <div
            className="h-full w-full"
            style={{
              backgroundImage:
                "linear-gradient(#40b594 1px, transparent 1px),
linear-gradient(90deg, #40b594 1px, transparent 1px)",
              backgroundSize: "46px 46px",
            }}
          />
        </div>

        <div className="relative mx-auto max-w-7xl px-8 py-12
md:py-16">
          <div className="max-w-3xl">

            <h1 className="text-3xl font-black tracking-tight
text-white md:text-5xl">
              {saved.companyName}
            </h1>
            <p className="mt-4 max-w-2xl text-sm leading-7
text-white/68 md:text-base">
              Company identity, business details, hiring
summary, and stored company file in one clean layout without
overlap.
```

```
            </p>
          </div>
        </div>
      </section>

      <main className="mx-auto max-w-7xl px-8 pt-8">
        <section className={`${cardClass} mb-8 p-6 md:p-8`}>
          <div className="flex flex-col gap-8 lg:flex-row
lg:items-center lg:justify-between">
            <div className="flex flex-col items-center gap-6
text-center lg:flex-row lg:text-left">
              <div className="relative">
                <div className="flex h-36 w-36 items-center
justify-center overflow-hidden rounded-full border-[6px]
border-white bg-[linear-gradient(135deg,#21483d,#40b594)]
shadow-[0_18px_50px_rgba(0,0,0,0.18)] md:h-40 md:w-40">
                  {displayImageSrc ? (
                    <img src={displayImageSrc} alt="Company
profile" className="h-full w-full object-cover" />
                  ) : (
                    <Building2 className="h-16 w-16
text-white/85" />
                  )}
                </div>

                <button
                  type="button"
                  onClick={handleCircleAction}
                  className="absolute bottom-1 right-1 flex
h-12 w-12 items-center justify-center rounded-full border-4
border-white bg-[#00a37b] text-white shadow-lg"
                >
```

```
                  {displayImageSrc ? <Pencil size={18} /> :
<Camera size={20} />}
                </button>

                {displayImageSrc && isEditing && (
                  <button
                    type="button"
                    onClick={handleDeleteImage}
                    className="absolute left-1 top-1 flex h-10
w-10 items-center justify-center rounded-full border-4
border-white bg-white text-[#d11a2a] shadow-md"
                  >
                    <Trash2 size={16} />
                  </button>
                )}

                <input
                  ref={fileInputRef}
                  type="file"
                  accept="image/jpeg,image/png,image/webp"
                  onChange={handleImageChange}
                  className="hidden"
                />
              </div>

              <div className="max-w-2xl">
                <div className="flex items-center
justify-center gap-2 lg:justify-start">
                  <h2 className="text-2xl font-black
text-[#0d211b] md:text-3xl">
                    {saved.companyName}
                  </h2>
```

```
                    <CheckCircle2 className="text-[#40b594]"
size={20} />
                  </div>
                  <p className="mt-2 text-sm font-bold
text-[#40b594]">{saved.industry}</p>
                  <p className="mt-4 text-sm leading-7
text-[#58706a]">
                    {saved.companyDescription}
                  </p>
                </div>
              </div>

              <div className="flex flex-wrap justify-center gap-3
lg:justify-end">
                {isEditing ? (
                  <>
                    <button
                      type="button"
                      onClick={handleCancel}
                      className="rounded-2xl border
border-[#d7e5df] bg-white px-5 py-3 text-sm font-bold
text-[#17332b]"
                    >
                      <span className="inline-flex items-center
gap-2"><X size={16} />Cancel</span>
                    </button>
                    <button
                      type="button"
                      onClick={handleSave}
                      disabled={loading}
                      className="rounded-2xl bg-[#051612] px-5
py-3 text-sm font-bold text-white disabled:opacity-60"
```

```
                    >
                      <span className="inline-flex items-center
gap-2"><Save size={16} />{loading ? "Saving..." : "Save
Changes"}</span>
                    </button>
                  </>
                ) : (
                  <button
                    type="button"
                    onClick={handleEdit}
                    className="rounded-2xl bg-[#051612] px-5 py-3
text-sm font-bold text-white"
                  >
                    <span className="inline-flex items-center
gap-2"><Pencil size={16} />Edit Profile</span>
                  </button>
                )}
              </div>
            </div>
          </section>

          <div className="mb-8 grid grid-cols-1 gap-4
md:grid-cols-3">
            {stats.map(({ label, value, icon: Icon }) => (
              <div key={label} className={`${cardClass} p-5`}>
                <div className="mb-4 flex h-11 w-11 items-center
justify-center rounded-2xl bg-[#eff8f5] text-[#40b594]">
                  <Icon size={18} />
                </div>
                <p className="text-3xl font-black
text-[#0c201a]">{value}</p>
```

```
                <p className="mt-1 text-xs font-bold uppercase
tracking-[0.22em] text-[#718881]">
                  {label}
                </p>
              </div>
            ))}
          </div>

          <div className="grid grid-cols-1 gap-8
lg:grid-cols-[0.95fr_2.05fr]">
            <aside className="space-y-8">
              <section className={`${cardClass} p-6`}>
                <p className="mb-2 text-[10px] font-bold
uppercase tracking-[0.22em] text-[#40b594]">
                  Profile Completion
                </p>
                <div className="mb-3 flex items-end
justify-between">
                  <p className="text-3xl font-black
text-[#0c201a]">{completion}%</p>
                  <p className="text-xs font-semibold
text-[#6a817b]">Complete your company profile</p>
                </div>
                <div className="h-3 w-full rounded-full
bg-[#e5efeb]">
                  <div className="h-3 rounded-full bg-[#40b594]"
style={{ width: `${completion}%` }} />
                </div>
              </section>

              <section className={`${cardClass} p-6`}>
```

```
                <h3 className="mb-5 text-base font-extrabold
text-[#071a15]">Quick Info</h3>
                <div className="space-y-4">
                  <InfoRow icon={Globe} label="Website"
value={saved.websiteLink || "Not added"} />
                  <InfoRow icon={MapPin} label="Location"
value={saved.currentAddress || "Not added"} />
                  <InfoRow icon={Users} label="Company Size"
value={saved.companySize || "Not added"} />
                  <InfoRow icon={Phone} label="Contact"
value={saved.contact || "Not added"} />
                </div>
              </section>

              <section className={`${cardClass} p-6`}>
                <div className="mb-5 flex items-center gap-3">
                  <div className="flex h-11 w-11 items-center
justify-center rounded-2xl bg-[#eff8f5] text-[#40b594]">
                    <FileText size={18} />
                  </div>
                  <div>
                    <h3 className="text-base font-extrabold
text-[#071a15]">Company File</h3>
                    <p className="text-xs text-[#6a817b]">Store
one company document</p>
                  </div>
                </div>

                <div className="rounded-3xl border border-dashed
border-[#cfe2db] bg-[#f8fcfa] p-4">
                  <p className="text-sm font-semibold
text-[#17332b]">
```

```
                    {saved.companyFileName || "No company file
uploaded yet"}
                  </p>
                  {saved.companyFileUrl ? (
                    <a
                      href={saved.companyFileUrl}
                      target="_blank"
                      rel="noreferrer"
                      className="mt-3 inline-flex items-center
gap-2 text-sm font-bold text-[#0a7e61]"
                    >
                      <FileText size={16} />
                      View uploaded file
                    </a>
                  ) : (
                    <p className="mt-2 text-xs
text-[#7a8f89]">PDF, DOC, or DOCX</p>
                  )}
                </div>
              </section>
            </aside>

            <div className="space-y-8">
              <section className={`${cardClass} p-3`}>
                <div className="flex flex-wrap gap-2">
                  {(["overview", "details"] as const).map((tab)
=> (
                    <button
                      key={tab}
                      type="button"
                      onClick={() => setActiveTab(tab)}
```

```
                    className={`rounded-2xl px-5 py-3 text-sm
font-extrabold ${
                      activeTab === tab ? "bg-[#051612]
text-white" : "text-[#5f7770]"
                    }`}
                  >
                    {tab === "overview" ? "Overview" : "Company
Details"}
                  </button>
                ))}
              </div>
            </section>

            {activeTab === "overview" ? (
              <>
                <section className={`${cardClass} p-6 md:p-8`}>
                  <h3 className="mb-6 text-xl font-black
text-[#0d211b]">About Company</h3>
                  {isEditing ? (
                    <textarea
                      name="companyDescription"
                      value={profile.companyDescription}
                      onChange={handleChange}
                      className="h-40 w-full resize-none
rounded-3xl border border-[#d9e8e2] bg-[#f6fbf8] px-5 py-4
text-sm"
                    />
                  ) : (
                    <p className="text-sm leading-8
text-[#405752]">{saved.companyDescription}</p>
                  )}
                </section>
```

```
                <section className={`${cardClass} p-6 md:p-8`}>
                  <div className="mb-6 flex items-center
justify-between">
                    <h3 className="text-xl font-black
text-[#0d211b]">Current Open Positions</h3>
                  </div>

                  <div className="space-y-4">
                    {openJobs.map((job) => (
                      <div
                        key={job.title}
                        className="flex items-center
justify-between rounded-3xl border border-[#ebf1ee] bg-[#f9fcfb]
px-5 py-4"
                      >
                        <div className="flex items-center
gap-4">
                          <div className="flex h-11 w-11
items-center justify-center rounded-2xl bg-[#eff8f5]
text-[#16322b]">
                            <Briefcase size={18} />
                          </div>
                          <div>
                            <h4 className="text-sm font-black
text-[#10211d]">{job.title}</h4>
                            <p className="mt-1 text-xs
font-semibold text-[#59706a]">
                              {job.type} · <span
className="text-[#40b594]">{job.salary}</span>
                            </p>
                          </div>
```

```
                        </div>
                        <span className="rounded-full bg-white
px-3 py-1 text-[11px] font-bold uppercase tracking-[0.18em]
text-[#59706a]">
                          Active
                        </span>
                      </div>
                    ))}
                  </div>
                </section>
              </>
            ) : (
              <section className={`${cardClass} p-6 md:p-8`}>
                <h3 className="mb-6 text-xl font-black
text-[#0d211b]">Company Information</h3>
                <div className="grid grid-cols-1 gap-5
md:grid-cols-2">
                  <EditableField editing={isEditing}
label="Company Name" name="companyName"
value={profile.companyName} onChange={handleChange}
className="md:col-span-2" />
                  <EditableField editing={isEditing}
label="Industry" name="industry" value={profile.industry}
onChange={handleChange} />
                  <EditableField editing={isEditing}
label="Company Size" name="companySize"
value={profile.companySize} onChange={handleChange} />
                  <EditableField editing={isEditing}
label="Current Address" name="currentAddress"
value={profile.currentAddress} onChange={handleChange}
className="md:col-span-2" />
```

```
                    <EditableSelectYear editing={isEditing}
label="Founded Year" name="foundedYear"
value={profile.foundedYear} onChange={handleChange} />
                    <EditableField editing={isEditing}
label="Country" name="country" value={profile.country}
onChange={handleChange} />
                    <EditableField editing={isEditing}
label="Contact" name="contact" value={profile.contact}
onChange={handleChange} />
                    <EditableField editing={isEditing}
label="Website Link" name="websiteLink"
value={profile.websiteLink} onChange={handleChange} />
                  </div>
                </section>
              )}

              <section className={`${cardClass} p-6 md:p-8`}>
                <div className="mb-6 flex items-center
justify-between">
                  <div>
                    <h3 className="text-xl font-black
text-[#0d211b]">Upload Company File</h3>
                    <p className="mt-1 text-sm text-[#6a817b]">
                      Store your company profile, brochure, or
supporting document
                    </p>
                  </div>
                  <FileText className="text-[#40b594]" size={22}
/>
                </div>
```

```
              <div className="rounded-[28px] border
border-dashed border-[#cfe2db] bg-[#f8fcfa] p-6">
                <div className="flex flex-col gap-4 md:flex-row
md:items-center md:justify-between">
                  <div>
                    <p className="text-sm font-bold
text-[#17332b]">
                      {profile.companyFileName ||
saved.companyFileName || "No file selected"}
                    </p>
                    <p className="mt-1 text-xs text-[#7a8f89]">
                      Accepted formats: PDF, DOC, DOCX. Max
size: 10MB.
                    </p>
                  </div>

                  <div className="flex flex-wrap gap-3">
                    {isEditing && (
                      <>
                        <button
                          type="button"
                          onClick={openCompanyFilePicker}
                          className="inline-flex items-center
gap-2 rounded-2xl bg-[#051612] px-4 py-3 text-sm font-bold
text-white"
                        >
                          <Upload size={16} />
                          Upload File
                        </button>

                        {(profile.companyFileName ||
saved.companyFileName) && (
```

```
                            <button
                              type="button"
                              onClick={handleRemoveCompanyFile}
                              className="inline-flex items-center
gap-2 rounded-2xl border border-[#d7e5df] bg-white px-4 py-3
text-sm font-bold text-[#b42318]"
                            >
                              <Trash2 size={16} />
                              Remove File
                            </button>
                          )}
                        </>
                      )}

                      {!isEditing && saved.companyFileUrl && (
                        <a
                          href={saved.companyFileUrl}
                          target="_blank"
                          rel="noreferrer"
                          className="inline-flex items-center
gap-2 rounded-2xl bg-[#eff8f5] px-4 py-3 text-sm font-bold
text-[#0a7e61]"
                        >
                          <FileText size={16} />
                          View File
                        </a>
                      )}
                    </div>
                  </div>

                  <input
                    ref={companyFileInputRef}
```

```
                    type="file"

accept=".pdf,.doc,.docx,application/pdf,application/msword,application/vnd.openxmlformats-officedocument.wordprocessingml.document"
                    onChange={handleCompanyFileChange}
                    className="hidden"
                  />
                </div>
              </section>
            </div>
          </div>
        </main>
      </div>

      <ImageCropModal
        open={cropOpen}
        imageSrc={sourceImageUrl || saved.profileImageUrl}
        title="Adjust company profile image"
        onClose={handleCropClose}
        onSave={handleCropSave}
        onUploadAnother={openFilePicker}
        onDelete={handleDeleteImage}
        hasImage={!!(sourceImageUrl || saved.profileImageUrl)}
      />
    </>
  );
}

function EditableField({
  editing,
  label,
```

```
 name,
 value,
 onChange,
 className = "",
}: {
 editing: boolean;
 label: string;
 name: string;
 value: string;
 onChange: (
   e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
 ) => void;
 className?: string;
}) {
 return (
   <div className={className}>
     <p className="mb-2 text-sm font-bold
text-[#27413a]">{label}</p>
     {editing ? (
       <input
         name={name}
         value={value}
         onChange={onChange}
         className="w-full rounded-2xl border border-[#d9e8e2]
bg-[#f6fbf8] px-4 py-3 text-sm"
       />
     ) : (
       <div className="rounded-2xl border border-[#edf3f0]
bg-[#f9fcfb] px-4 py-3 text-sm font-semibold">
         {value || "Not added"}
       </div>
```

```
      )}
    </div>
  );
}

function EditableSelectYear({
  editing,
  label,
  name,
  value,
  onChange,
}: {
  editing: boolean;
  label: string;
  name: string;
  value: string;
  onChange: (
    e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
  ) => void;
}) {
  return (
    <div>
      <p className="mb-2 text-sm font-bold
text-[#27413a]">{label}</p>
      {editing ? (
        <select
          name={name}
          value={value}
          onChange={onChange}
          className="w-full rounded-2xl border border-[#d9e8e2]
bg-[#f6fbf8] px-4 py-3 text-sm"
```

```
        >
          <option value="">Select option</option>
          {Array.from({ length: 60 }, (_, i) => {
            const year = new Date().getFullYear() - i;
            return (
              <option key={year} value={String(year)}>
                {year}
              </option>
            );
          })}
        </select>
      ) : (
        <div className="rounded-2xl border border-[#edf3f0]
bg-[#f9fcfb] px-4 py-3 text-sm font-semibold">
          {value || "Not added"}
        </div>
      )}
    </div>
  );
}

function InfoRow({
  icon: Icon,
  label,
  value,
}: {
  icon: React.ComponentType<{ size?: number }>;
  label: string;
  value: string;
}) {
  return (
    <div className="flex items-center gap-4">
```

```
     <div className="flex h-10 w-10 items-center justify-center
rounded-xl bg-[#f0f4f3] text-[#40b594]">
       <Icon size={17} />
     </div>
     <div>
       <p className="text-[10px] font-bold uppercase
tracking-[0.22em] text-[#6b7f79]">
         {label}
       </p>
       <p className="text-sm font-bold
text-[#071a15]">{value}</p>
     </div>
   </div>
 );
}

"use client";

import React, { useState } from "react";
import {
 Shield,
 AlertTriangle,
 Mail,
 Save,
 Building2,
 Globe,
 MapPin,
 Trash2,
 LogOut,
 ChevronRight,
 Lock,
 Eye,
```

```
 EyeOff,
 Briefcase,
 ExternalLink,
 UserCog,
 Phone,
} from "lucide-react";
import Link from "next/link";

const EmployerSettings = () => {
 const [activeTab, setActiveTab] = useState("general");
 const [showCurrent, setShowCurrent] = useState(false);
 const [showNew, setShowNew] = useState(false);
 const [deleteText, setDeleteText] = useState("");

 const [settings, setSettings] = useState({
   adminEmail: "admin@nexhire.com",
   billingEmail: "billing@nexhire.com",
 });

 const tabs = [
   { id: "general", name: "General", icon: UserCog },
   { id: "security", name: "Security", icon: Shield },
   { id: "danger", name: "Danger Zone", icon: AlertTriangle },
 ];

 const inputClass =
   "w-full rounded-xl border border-gray-200 bg-[#f8faf9] px-4
py-3 text-sm font-medium text-[#071a15] placeholder-[#9ab0aa]
transition-all focus:border-[#40b594] focus:outline-none
focus:ring-2 focus:ring-[#40b594]/30";
```

```
 const labelClass = "mb-2 block text-sm font-extrabold
text-[#071a15]";

 return (
   <div className="min-h-screen bg-[#f0f4f3] pb-16 font-sans">
     <header className="sticky top-0 z-50 flex items-center
justify-between bg-[#051612] px-8 py-4 text-white shadow-lg">
       <div className="flex items-center gap-2.5">
         <img src="/logo.png" alt="NexHire" className="h-8 w-8"
/>
         <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
       </div>

       <nav className="hidden items-center gap-8 text-sm
font-semibold md:flex">
         <Link href="/dashboard">
           <button className="text-gray-300 transition-colors
hover:text-white">Dashboard</button>
         </Link>
         <Link href="/post_job">
           <button className="text-gray-300 transition-colors
hover:text-white">Post Job</button>
         </Link>
         <Link href="/employer_message">
           <button className="text-gray-300 transition-colors
hover:text-white">Messages</button>
         </Link>
         <Link href="/employer_notification">
           <button className="text-gray-300 transition-colors
hover:text-white">Notification</button>
         </Link>
```

```
        <Link href="/subscription">
          <button className="text-gray-300 transition-colors
hover:text-white">Subscription</button>
        </Link>
        <button className="border-b-2 border-[#40b594] pb-1
text-[#40b594]">Settings</button>
      </nav>

      <Link href="/employer_profile">
        <div className="group flex cursor-pointer items-center
gap-3">
          <div className="text-right">
            <p className="text-[10px] uppercase tracking-widest
text-gray-500">Company</p>
            <p className="text-sm font-bold text-white
transition-colors group-hover:text-[#40b594]">
              Profile
            </p>
          </div>
          <div className="flex h-10 w-10 items-center
justify-center rounded-full bg-[#40b594] text-sm font-extrabold
text-[#051612]">
            C
          </div>
        </div>
      </Link>
    </header>

    <main className="mx-auto max-w-6xl px-6 py-10 md:px-10">
      <div className="mb-8">
        <p className="mb-1 text-xs font-bold uppercase
tracking-widest text-[#40b594]">Account</p>
```

```
        <h1 className="text-4xl font-extrabold
text-[#071a15]">Settings</h1>
        <p className="mt-1 font-medium text-[#4a5a55]">
          Manage account controls, security, and workspace
preferences
        </p>
      </div>

      <div className="flex flex-col gap-6 lg:flex-row">
        <aside className="flex-shrink-0 lg:w-72">
          <div className="overflow-hidden rounded-2xl border
border-gray-100 bg-white shadow-sm">
            {tabs.map((tab, idx) => {
              const Icon = tab.icon;
              const isActive = activeTab === tab.id;
              const isDanger = tab.id === "danger";

              return (
                <button
                  key={tab.id}
                  onClick={() => setActiveTab(tab.id)}
                  className={`w-full px-5 py-4 text-sm
font-bold transition-all ${
                    idx !== 0 ? "border-t border-gray-100" : ""
                  } ${
                    isActive
                      ? isDanger
                        ? "bg-red-50 text-red-600"
                        : "bg-[#f0f9f6] text-[#071a15]"
                      : isDanger
                        ? "text-red-400 hover:bg-red-50
hover:text-red-600"
```

```
                        : "text-[#4a5a55] hover:bg-[#f8faf9]
hover:text-[#071a15]"
                  }`}
                >
                  <div className="flex items-center
justify-between">
                    <div className="flex items-center gap-3">
                      <div
                        className={`flex h-8 w-8 items-center
justify-center rounded-lg ${
                          isActive
                            ? isDanger
                              ? "bg-red-100"
                              : "bg-[#d1e8e3]"
                            : "bg-[#f0f4f3]"
                        }`}
                      >
                        <Icon
                          size={16}
                          className={
                            isActive
                              ? isDanger
                                ? "text-red-600"
                                : "text-[#40b594]"
                              : isDanger
                                ? "text-red-400"
                                : "text-[#6b7f79]"
                          }
                        />
                      </div>
                      {tab.name}
                    </div>
```

```
                    {isActive && (
                      <ChevronRight
                        size={16}
                        className={isDanger ? "text-red-400" :
"text-[#40b594]"}
                      />
                    )}
                  </div>
                </button>
              );
            })}
          </div>

          <button className="mt-4 flex w-full items-center
gap-3 rounded-2xl border border-gray-100 bg-white px-5 py-4
text-sm font-bold text-[#4a5a55] shadow-sm transition-all
hover:border-red-200 hover:bg-red-50 hover:text-red-600">
            <div className="flex h-8 w-8 items-center
justify-center rounded-lg bg-[#f0f4f3]">
              <LogOut size={16} className="text-[#6b7f79]" />
            </div>
            Sign Out
          </button>
        </aside>

        <div className="min-w-0 flex-1">
          {activeTab === "general" && (
            <div className="space-y-6">
              <div className="overflow-hidden rounded-2xl
border border-gray-100 bg-white shadow-sm">
```

```
                <div className="border-b border-gray-100 px-8
py-6">
                  <h2 className="text-lg font-extrabold
text-[#071a15]">Workspace Overview</h2>
                  <p className="mt-0.5 text-sm font-medium
text-[#4a5a55]">
                    Company information is managed from the
public employer profile page
                  </p>
                </div>

                <div className="p-8">
                  <div className="grid gap-4 md:grid-cols-2">
                    <SummaryCard icon={Building2}
label="Company Name" value="NexHire Solutions" />
                    <SummaryCard icon={Globe} label="Website"
value="www.nexhire.com" />
                    <SummaryCard icon={MapPin} label="Location"
value="Phnom Penh, Cambodia" />
                    <SummaryCard icon={Briefcase} label="Active
Jobs" value="12 Open Positions" />
                  </div>

                  <div className="mt-6 rounded-2xl border
border-[#d1e8e3] bg-[#f0f9f6] p-5">
                    <div className="flex flex-col gap-4
md:flex-row md:items-center md:justify-between">
                      <div>
                        <p className="text-sm font-extrabold
text-[#071a15]">
                          Edit company profile from the real
profile page
```

```
                        </p>
                        <p className="mt-1 text-sm font-medium
text-[#4a5a55]">
                          Keep branding, company details,
photo, and files in one place instead of duplicating forms here.
                        </p>
                      </div>
                      <Link href="/employer_profile">
                        <button className="inline-flex
items-center gap-2 rounded-xl bg-[#051612] px-5 py-3 text-sm
font-extrabold text-white transition-all hover:bg-[#0d2a23]">
                          Open Profile
                          <ExternalLink size={16} />
                        </button>
                      </Link>
                    </div>
                  </div>
                </div>
              </div>

              <div className="overflow-hidden rounded-2xl
border border-gray-100 bg-white shadow-sm">
                <div className="border-b border-gray-100 px-8
py-6">
                  <h2 className="text-lg font-extrabold
text-[#071a15]">Account Emails</h2>
                  <p className="mt-0.5 text-sm font-medium
text-[#4a5a55]">
                    Separate internal contact emails from
public company profile information
                  </p>
                </div>
```

```
                <div className="p-8">
                  <form className="space-y-6">
                    <div>
                      <label className={labelClass}>Admin
Email</label>
                      <div className="relative">
                        <Mail size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                        <input
                          type="email"
                          value={settings.adminEmail}
                          onChange={(e) =>
                            setSettings((prev) => ({ ...prev,
adminEmail: e.target.value }))
                          }
                          className={`${inputClass} pl-10`}
                        />
                      </div>
                      <p className="ml-1 mt-2 text-xs
font-semibold text-[#6b7f79]">
                        Used for login, security, and workspace
management
                      </p>
                    </div>

                    <div>
                      <label className={labelClass}>Billing
Email</label>
                      <div className="relative">
                        <Mail size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
```

```
                          <input
                            type="email"
                            value={settings.billingEmail}
                            onChange={(e) =>
                              setSettings((prev) => ({ ...prev,
billingEmail: e.target.value }))
                            }
                            className={`${inputClass} pl-10`}
                          />
                        </div>
                        <p className="ml-1 mt-2 text-xs
font-semibold text-[#6b7f79]">
                          Used for invoices and subscription
communication
                        </p>
                      </div>

                      <div className="pt-2">
                        <button
                          type="button"
                          className="flex items-center gap-2
rounded-xl bg-[#051612] px-6 py-3 text-sm font-extrabold
text-white transition-all hover:bg-[#0d2a23]"
                        >
                          <Save size={16} />
                          Save Changes
                        </button>
                      </div>
                    </form>
                  </div>
                </div>
              </div>
```

```
          )}

          {activeTab === "security" && (
            <div className="overflow-hidden rounded-2xl border
border-gray-100 bg-white shadow-sm">
              <div className="border-b border-gray-100 px-8
py-6">
                <h2 className="text-lg font-extrabold
text-[#071a15]">Security</h2>
                <p className="mt-0.5 text-sm font-medium
text-[#4a5a55]">
                  Update your account password
                </p>
              </div>

              <div className="p-8">
                <form className="max-w-md space-y-6">
                  <div>
                    <label className={labelClass}>Current
Password</label>
                    <div className="relative">
                      <Lock size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                      <input
                        type={showCurrent ? "text" :
"password"}
                        placeholder="Enter current password"
                        className={`${inputClass} pl-10 pr-10`}
                      />
                      <button
                        type="button"
```

```
                          onClick={() =>
setShowCurrent(!showCurrent)}
                          className="absolute right-4 top-1/2
-translate-y-1/2 text-[#6b7f79] transition-colors
hover:text-[#071a15]"
                        >
                          {showCurrent ? <EyeOff size={16} /> :
<Eye size={16} />}
                        </button>
                      </div>
                    </div>

                    <div>
                      <label className={labelClass}>New
Password</label>
                      <div className="relative">
                        <Lock size={16} className="absolute
left-4 top-1/2 -translate-y-1/2 text-[#6b7f79]" />
                        <input
                          type={showNew ? "text" : "password"}
                          placeholder="Enter new password"
                          className={`${inputClass} pl-10 pr-10`}
                        />
                        <button
                          type="button"
                          onClick={() => setShowNew(!showNew)}
                          className="absolute right-4 top-1/2
-translate-y-1/2 text-[#6b7f79] transition-colors
hover:text-[#071a15]"
                        >
                          {showNew ? <EyeOff size={16} /> : <Eye
size={16} />}
```

```
                      </button>
                    </div>
                  </div>

                  <div className="rounded-xl border
border-[#d1e8e3] bg-[#f0f9f6] p-4">
                    <p className="mb-2 text-xs font-extrabold
text-[#071a15]">Password requirements</p>
                    {["At least 8 characters", "One uppercase
letter", "One number or symbol"].map((req) => (
                      <div key={req} className="mt-1.5 flex
items-center gap-2">
                        <div className="h-1.5 w-1.5
rounded-full bg-[#40b594]" />
                        <p className="text-xs font-medium
text-[#4a5a55]">{req}</p>
                      </div>
                    ))}
                  </div>

                  <div className="pt-2">
                    <button
                      type="button"
                      className="flex items-center gap-2
rounded-xl bg-[#051612] px-6 py-3 text-sm font-extrabold
text-white transition-all hover:bg-[#0d2a23]"
                    >
                      <Shield size={16} />
                      Update Password
                    </button>
                  </div>
                </form>
```

```
              </div>
            </div>
          )}

          {activeTab === "danger" && (
            <div className="overflow-hidden rounded-2xl border
border-red-100 bg-white shadow-sm">
              <div className="border-b border-red-100
bg-red-50/50 px-8 py-6">
                <h2 className="text-lg font-extrabold
text-red-600">Danger Zone</h2>
                <p className="mt-0.5 text-sm font-medium
text-red-400">
                  These actions are permanent and cannot be
undone
                </p>
              </div>

              <div className="p-8">
                <div className="rounded-2xl border
border-red-100 bg-red-50/30 p-7">
                  <div className="mb-6 flex items-start gap-4">
                    <div className="flex h-10 w-10
flex-shrink-0 items-center justify-center rounded-xl
bg-red-100">
                      <Trash2 size={18}
className="text-red-600" />
                    </div>
                    <div>
                      <h3 className="text-base font-extrabold
text-[#071a15]">
                        Delete Company Workspace
```

```
                        </h3>
                        <p className="mt-1 text-sm font-medium
leading-relaxed text-[#4a5a55]">
                          This will permanently delete your
company profile, all active job listings,
                          and applicant history. This action
cannot be reversed.
                        </p>
                      </div>
                    </div>

                    <div className="space-y-4">
                      <div>
                        <label className="mb-2 block text-sm
font-extrabold text-[#071a15]">
                          Type <span className="font-black
tracking-widest text-red-600">DELETE</span> to confirm
                        </label>
                        <input
                          type="text"
                          placeholder="Type DELETE here"
                          value={deleteText}
                          onChange={(e) =>
setDeleteText(e.target.value)}
                          className="w-full max-w-sm rounded-xl
border border-red-200 bg-white px-4 py-3 text-sm font-medium
text-[#071a15] placeholder-red-200 transition-all
focus:border-red-400 focus:outline-none focus:ring-2
focus:ring-red-300/40"
                        />
                      </div>
```

```
                      <button
                        type="button"
                        disabled={deleteText !== "DELETE"}
                        className={`flex items-center gap-2
rounded-xl px-6 py-3 text-sm font-extrabold transition-all ${
                          deleteText === "DELETE"
                            ? "bg-red-600 text-white
hover:bg-red-700"
                            : "cursor-not-allowed bg-red-100
text-red-300"
                        }`}
                      >
                        <Trash2 size={16} />
                        Delete Company Account
                      </button>
                    </div>
                  </div>
                </div>
              </div>
            )}
          </div>
        </div>
      </main>
    </div>
  );
};

function SummaryCard({
  icon: Icon,
  label,
  value,
}: {
```

```
 icon: React.ComponentType<{ size?: number; className?: string
}>;
 label: string;
 value: string;
}) {
 return (
   <div className="rounded-2xl border border-gray-100
bg-[#f8faf9] p-5">
     <div className="mb-3 flex h-10 w-10 items-center
justify-center rounded-xl bg-[#eaf5f1]">
       <Icon size={18} className="text-[#40b594]" />
     </div>
     <p className="text-[11px] font-black uppercase
tracking-wider text-[#6b7f79]">{label}</p>
     <p className="mt-1 text-sm font-extrabold
text-[#071a15]">{value}</p>
   </div>
 );
}

export default EmployerSettings;
```

```
"use client";

import React, { useState } from "react";
import {
 Clock,
 FileEdit,
 Trash2,
 Briefcase,
 PlusCircle,
```

```
 ChevronDown,
} from "lucide-react";
import Link from "next/link";

const categoryOptions = [
 "IT & Software",
 "Design",
 "Marketing",
 "Sales",
 "Finance",
 "Human Resources",
 "Engineering",
 "Customer Support",
 "Operations",
 "Legal",
];

const locationOptions = [
 "Phnom Penh",
 "Siem Reap",
 "Battambang",
 "Sihanoukville",
 "Kampot",
 "Kandal",
 "Takeo",
 "Remote",
 "Other",
];

function Combobox({
 placeholder,
 options,
```

```
 inputClass,
 value,
 onChange,
}: {
 placeholder: string;
 options: string[];
 inputClass: string;
 value: string;
 onChange: (value: string) => void;
}) {
 const [open, setOpen] = useState(false);

 const filtered = options.filter((o) =>
   o.toLowerCase().includes(value.toLowerCase())
 );

 return (
   <div className="relative">
     <input
       type="text"
       value={value}
       placeholder={placeholder}
       className={`${inputClass} pr-10`}
       onChange={(e) => {
         onChange(e.target.value);
         setOpen(true);
       }}
       onFocus={() => setOpen(true)}
       onBlur={() => setTimeout(() => setOpen(false), 120)}
     />
     <ChevronDown
       size={16}
```

```
        className={`absolute right-4 top-1/2 -translate-y-1/2
text-[#6b7f79] pointer-events-none transition-transform
duration-200 ${
          open ? "rotate-180" : ""
        }`}
      />
      {open && filtered.length > 0 && (
        <ul className="absolute z-50 mt-1.5 w-full bg-white
border border-gray-200 rounded-xl shadow-lg overflow-hidden">
          {filtered.map((option) => (
            <li
              key={option}
              onMouseDown={() => {
                onChange(option);
                setOpen(false);
              }}
              className={`px-4 py-2.5 text-sm font-medium
text-[#071a15] cursor-pointer transition-colors
hover:bg-[#f0f9f6] hover:text-[#40b594] ${
                value === option ? "bg-[#f0f9f6] text-[#40b594]
font-bold" : ""
              }`}
            >
              {option}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

```
type DraftJob = {
 id: string;
 title: string;
 category: string;
 lastSaved: string;
};

const PostJob = () => {
 const [activeTab, setActiveTab] = useState<"post" |
"drafts">("post");
 const [loading, setLoading] = useState(false);
 const [draftLoading, setDraftLoading] = useState(false);
 const [draftData, setDraftData] = useState<DraftJob[]>([]);
 const [form, setForm] = useState({
   title: "",
   category: "",
   location: "",
   arrangement: "",
   employmentType: "",
   experienceLevel: "",
   salaryMin: "",
   salaryMax: "",
   applicationDeadline: "",
   description: "",
   requirements: "",
   applicationPlatform: "",
   externalApplyLink: "",
   contactEmail: "",
   status: "active",
 });

 const inputClass =
```

```
   "w-full px-4 py-3 border border-gray-200 rounded-xl
bg-[#f8faf9] text-[#071a15] text-sm font-medium
placeholder-[#9ab0aa] focus:outline-none focus:ring-2
focus:ring-[#40b594]/30 focus:border-[#40b594] transition-all";

 const selectClass =
   "w-full px-4 py-3 border border-gray-200 rounded-xl
bg-[#f8faf9] text-[#071a15] text-sm font-medium
focus:outline-none focus:ring-2 focus:ring-[#40b594]/30
focus:border-[#40b594] transition-all appearance-none";

 const labelClass = "block text-sm font-extrabold text-[#071a15]
mb-2";

 const arrangementValue =
   form.arrangement === "On-site"
     ? "on_site"
     : form.arrangement === "Remote"
       ? "remote"
       : form.arrangement === "Hybrid"
         ? "hybrid"
         : "on_site";

 const employmentValue =
   form.employmentType === "Full-time"
     ? "full_time"
     : form.employmentType === "Part-time"
       ? "part_time"
       : form.employmentType === "Contract"
         ? "contract"
         : form.employmentType === "Freelance"
           ? "freelance"
```

```
          : form.employmentType === "Internship"
            ? "internship"
            : "full_time";

const experienceValue =
  form.experienceLevel === "Entry Level"
    ? "entry"
    : form.experienceLevel === "Mid Level"
      ? "mid"
      : form.experienceLevel === "Senior"
        ? "senior"
        : form.experienceLevel === "Lead / Manager"
          ? "lead"
          : form.experienceLevel === "Executive"
            ? "executive"
            : "entry";

const handleSubmit = async (status: "active" | "draft") => {
  try {
    setLoading(true);

    const res = await fetch("/api/jobs", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        title: form.title,
        category: form.category,
        location: form.location,
        arrangement: arrangementValue,
        employmentType: employmentValue,
        experienceLevel: experienceValue,
```

```
        salaryMin: form.salaryMin ? Number(form.salaryMin) :
null,
        salaryMax: form.salaryMax ? Number(form.salaryMax) :
null,
        applicationDeadline: form.applicationDeadline || null,
        description: form.description,
        requirements: form.requirements,
        applicationPlatform: form.applicationPlatform ||
"internal",
        externalApplyLink: form.externalApplyLink || null,
        contactEmail: form.contactEmail || null,
        status,
      }),
    });

    const data = await res.json();
    if (!res.ok) {
      alert(data.error || "Failed to save job");
      return;
    }

    alert(status === "active" ? "Job published successfully" :
"Draft saved");

    setForm({
      title: "",
      category: "",
      location: "",
      arrangement: "",
      employmentType: "",
      experienceLevel: "",
      salaryMin: "",
```

```
      salaryMax: "",
      applicationDeadline: "",
      description: "",
      requirements: "",
      applicationPlatform: "",
      externalApplyLink: "",
      contactEmail: "",
      status: "active",
    });

    if (status === "draft") {
      await loadDrafts();
      setActiveTab("drafts");
    }
  } catch (error) {
    console.error("POST JOB ERROR:", error);
  } finally {
    setLoading(false);
  }
};

const loadDrafts = async () => {
  try {
    setDraftLoading(true);
    const res = await fetch("/api/jobs?mine=1", { cache:
"no-store" });
    const data = await res.json();
    if (!res.ok) return;

    const drafts = (data || [])
      .filter((job: any) => job.status === "draft")
      .map((job: any) => ({
```

```
        id: job.id,
        title: job.title,
        category: job.category,
        lastSaved: new Date(job.postedAt).toLocaleString(),
      }));

    setDraftData(drafts);
  } catch (error) {
    console.error(error);
  } finally {
    setDraftLoading(false);
  }
};

React.useEffect(() => {
  if (activeTab === "drafts") loadDrafts();
}, [activeTab]);

return (
  <div className="min-h-screen font-sans pb-16" style={{
background: "#f0f4f3" }}>
    <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
      <div className="flex items-center gap-2.5">
        <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
        <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
      </div>

      <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
```

```
        <Link href="/dashboard"><button
className="text-gray-300 hover:text-white
transition-colors">Dashboard</button></Link>
        <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Post Job</button>
        <Link href="/employer_message"><button
className="text-gray-300 hover:text-white
transition-colors">Messages</button></Link>
        <Link href="/employer_notification"><button
className="text-gray-300 hover:text-white
transition-colors">Notification</button></Link>
        <Link href="/subscription"><button
className="text-gray-300 hover:text-white
transition-colors">Subscription</button></Link>
        <Link href="/employer_setting"><button
className="text-gray-300 hover:text-white
transition-colors">Settings</button></Link>
      </nav>

      <Link href="/employer_profile">
        <div className="flex items-center gap-3 cursor-pointer
group">
          <div className="text-right">
            <p className="text-[10px] text-gray-500 uppercase
tracking-widest">Company</p>
            <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#40b594] rounded-full
flex items-center justify-center font-extrabold text-[#051612]
text-sm">
            C
```

```
          </div>
        </div>
      </Link>
    </header>

    <main className="max-w-7xl mx-auto px-6 md:px-10 py-10">
      <div className="mb-8">
        <p className="text-xs font-bold uppercase
tracking-widest text-[#40b594] mb-1">Hiring</p>
        <h1 className="text-4xl font-extrabold
text-[#071a15]">Post a Job</h1>
        <p className="text-[#4a5a55] font-medium mt-1">
          Fill in the details below to publish a new role
        </p>
      </div>

      <div className="flex gap-1 bg-white border
border-gray-100 rounded-2xl p-1.5 mb-8 shadow-sm w-fit">
        {(["post", "drafts"] as const).map((tab) => (
          <button
            key={tab}
            onClick={() => setActiveTab(tab)}
            className={`px-6 py-2.5 rounded-xl text-sm
font-extrabold transition-all ${
              activeTab === tab
                ? "bg-[#051612] text-white shadow-sm"
                : "text-[#4a5a55] hover:text-[#071a15]"
            }`}
          >
            {tab === "post" ? "Post Job" : `Drafts
(${draftData.length})`}
          </button>
```

```
          ))}
        </div>

        {activeTab === "post" && (
          <div className="flex flex-col lg:flex-row gap-6">
            <div className="lg:w-2/3 space-y-6">
              <section className="bg-white p-8 rounded-2xl
shadow-sm border border-gray-100">
                <div className="flex items-center gap-3 mb-7">
                  <div className="w-8 h-8 bg-[#f0f9f6] rounded-xl
flex items-center justify-center border border-[#d1e8e3]">
                    <Briefcase size={16}
className="text-[#40b594]" />
                  </div>
                  <h2 className="text-lg font-extrabold
text-[#071a15]">Job Details</h2>
                </div>

                <div className="space-y-6">
                  <div>
                    <label className={labelClass}>Job Title <span
className="text-red-500">*</span></label>
                    <input
                      type="text"
                      value={form.title}
                      onChange={(e) => setForm((p) => ({ ...p,
title: e.target.value }))}
                      placeholder="e.g. Senior Frontend
Developer"
                      className={inputClass}
                    />
                  </div>
```

```
                <div>
                  <label className={labelClass}>Job Category
<span className="text-red-500">*</span></label>
                  <Combobox
                    placeholder="e.g. Design, Marketing, IT &
Software"
                    options={categoryOptions}
                    inputClass={inputClass}
                    value={form.category}
                    onChange={(value) => setForm((p) => ({
...p, category: value }))}
                  />
                </div>

                <div className="grid grid-cols-1 md:grid-cols-2
gap-6">
                  <div>
                    <label className={labelClass}>Location
<span className="text-red-500">*</span></label>
                    <Combobox
                      placeholder="e.g. Phnom Penh, Remote"
                      options={locationOptions}
                      inputClass={inputClass}
                      value={form.location}
                      onChange={(value) => setForm((p) => ({
...p, location: value }))}
                    />
                  </div>

                  <div className="relative">
```

```
                    <label className={labelClass}>Work
Arrangement</label>
                    <select
                      value={form.arrangement}
                      onChange={(e) => setForm((p) => ({ ...p,
arrangement: e.target.value }))}
                      className={selectClass}
                    >
                      <option value="">On-site / Remote /
Hybrid</option>
                      <option>On-site</option>
                      <option>Remote</option>
                      <option>Hybrid</option>
                    </select>
                    <ChevronDown size={16} className="absolute
right-4 top-[42px] text-[#6b7f79] pointer-events-none" />
                  </div>
                </div>

                <div className="grid grid-cols-1 md:grid-cols-2
gap-6">
                  <div className="relative">
                    <label className={labelClass}>Employment
Type</label>
                    <select
                      value={form.employmentType}
                      onChange={(e) => setForm((p) => ({ ...p,
employmentType: e.target.value }))}
                      className={selectClass}
                    >
                      <option value="">Select type</option>
                      <option>Full-time</option>
```

```
                        <option>Part-time</option>
                        <option>Contract</option>
                        <option>Freelance</option>
                        <option>Internship</option>
                      </select>
                      <ChevronDown size={16} className="absolute
right-4 top-[42px] text-[#6b7f79] pointer-events-none" />
                    </div>

                    <div className="relative">
                      <label className={labelClass}>Experience
Level</label>
                      <select
                        value={form.experienceLevel}
                        onChange={(e) => setForm((p) => ({ ...p,
experienceLevel: e.target.value }))}
                        className={selectClass}
                      >
                        <option value="">Select level</option>
                        <option>Entry Level</option>
                        <option>Mid Level</option>
                        <option>Senior</option>
                        <option>Lead / Manager</option>
                        <option>Executive</option>
                      </select>
                      <ChevronDown size={16} className="absolute
right-4 top-[42px] text-[#6b7f79] pointer-events-none" />
                    </div>
                  </div>

                  <div className="grid grid-cols-1 md:grid-cols-2
gap-6">
```

```
                    <div>
                      <label className={labelClass}>Salary
Minimum</label>
                      <div className="relative">
                        <span className="absolute left-4 top-1/2
-translate-y-1/2 text-[#6b7f79] font-bold text-sm">$</span>
                        <input
                          type="number"
                          value={form.salaryMin}
                          onChange={(e) => setForm((p) => ({
...p, salaryMin: e.target.value }))}
                          placeholder="0"
                          className={`${inputClass} pl-8`}
                        />
                      </div>
                    </div>
                    <div>
                      <label className={labelClass}>Salary
Maximum</label>
                      <div className="relative">
                        <span className="absolute left-4 top-1/2
-translate-y-1/2 text-[#6b7f79] font-bold text-sm">$</span>
                        <input
                          type="number"
                          value={form.salaryMax}
                          onChange={(e) => setForm((p) => ({
...p, salaryMax: e.target.value }))}
                          placeholder="0"
                          className={`${inputClass} pl-8`}
                        />
                      </div>
                    </div>
```

```
                  </div>

                  <div>
                    <label className={labelClass}>Application
Deadline</label>
                    <input
                      type="date"
                      value={form.applicationDeadline}
                      onChange={(e) => setForm((p) => ({ ...p,
applicationDeadline: e.target.value }))}
                      className={inputClass}
                    />
                  </div>
                </div>
              </section>

              <section className="bg-white p-8 rounded-2xl
shadow-sm border border-gray-100">
                <h2 className="text-lg font-extrabold
text-[#071a15] mb-2">Job Description</h2>
                <p className="text-xs text-[#6b7f79]
font-semibold mb-5">
                  Describe the role, responsibilities, and what
success looks like
                </p>
                <textarea
                  rows={10}
                  value={form.description}
                  onChange={(e) => setForm((p) => ({ ...p,
description: e.target.value }))}
                  placeholder="Write the job description here..."
```

```
                  className="w-full px-4 py-3 border
border-gray-200 rounded-xl bg-[#f8faf9] text-[#071a15] text-sm
font-medium placeholder-[#9ab0aa] focus:outline-none
focus:ring-2 focus:ring-[#40b594]/30 focus:border-[#40b594]
transition-all resize-none"
                />
              </section>
            </div>

            <div className="lg:w-1/3 space-y-6">
              <section className="bg-white p-7 rounded-2xl
shadow-sm border border-gray-100">
                <h2 className="text-lg font-extrabold
text-[#071a15] mb-2">Requirements</h2>
                <p className="text-xs text-[#6b7f79]
font-semibold mb-5">
                  Skills, qualifications, and must-haves
                </p>
                <textarea
                  rows={8}
                  value={form.requirements}
                  onChange={(e) => setForm((p) => ({ ...p,
requirements: e.target.value }))}
                  placeholder="e.g. 3+ years React experience"
                  className="w-full px-4 py-3 border
border-gray-200 rounded-xl bg-[#f8faf9] text-[#071a15] text-sm
font-medium placeholder-[#9ab0aa] focus:outline-none
focus:ring-2 focus:ring-[#40b594]/30 focus:border-[#40b594]
transition-all resize-none"
                />
              </section>
```

```
            <section className="bg-white p-7 rounded-2xl
shadow-sm border border-gray-100">
              <h2 className="text-lg font-extrabold
text-[#071a15] mb-6">Application Settings</h2>

              <div className="space-y-5">
                <div className="relative">
                  <label className={labelClass}>Application
Platform</label>
                  <select
                    value={form.applicationPlatform}
                    onChange={(e) => setForm((p) => ({ ...p,
applicationPlatform: e.target.value }))}
                    className={selectClass}
                  >
                    <option value="">Select platform</option>
                    <option value="internal">Apply on
NexHire</option>
                    <option value="external">External
Link</option>
                  </select>
                  <ChevronDown size={16} className="absolute
right-4 top-[42px] text-[#6b7f79] pointer-events-none" />
                </div>

                <div>
                  <label className={labelClass}>External
Link</label>
                  <input
                    type="text"
                    value={form.externalApplyLink}
```

```
                    onChange={(e) => setForm((p) => ({ ...p,
externalApplyLink: e.target.value }))}
                    placeholder="https://yoursite.com/apply"
                    className={inputClass}
                  />
                </div>

                <div>
                  <label className={labelClass}>Contact
Email</label>
                  <input
                    type="email"
                    value={form.contactEmail}
                    onChange={(e) => setForm((p) => ({ ...p,
contactEmail: e.target.value }))}
                    placeholder="careers@company.com"
                    className={inputClass}
                  />
                </div>

                <div className="pt-2">
                  <label className="block text-sm
font-extrabold text-[#071a15] mb-3">Visibility</label>
                  <div className="grid grid-cols-2 gap-3">
                    {["Public Now", "Save as Draft"].map((opt)
=> (
                      <label
                        key={opt}
                        className="flex items-center gap-2.5
bg-[#f8faf9] border border-gray-200 rounded-xl px-4 py-3
cursor-pointer hover:border-[#40b594] transition-all"
                      >
```

```
                          <input
                            type="radio"
                            name="status"
                            checked={
                              opt === "Public Now"
                                ? form.status === "active"
                                : form.status === "draft"
                            }
                            onChange={() =>
                              setForm((p) => ({
                                ...p,
                                status: opt === "Public Now" ?
"active" : "draft",
                              }))
                            }
                            className="accent-[#40b594] w-4 h-4"
                          />
                          <span className="text-sm font-bold
text-[#071a15]">{opt}</span>
                        </label>
                      ))}
                    </div>
                  </div>
                </div>

                <div className="border-t border-gray-100 my-6" />

                <div className="space-y-3">
                  <button
                    onClick={() => handleSubmit("active")}
                    disabled={loading}
```

```
                    className="w-full bg-[#051612] text-white
py-3.5 rounded-xl font-extrabold text-sm hover:bg-[#0d2a23]
transition-all shadow-sm disabled:opacity-60"
                  >
                    {loading ? "Saving..." : "Publish Job"}
                  </button>
                  <button
                    onClick={() => handleSubmit("draft")}
                    disabled={loading}
                    className="w-full bg-[#f0f4f3] text-[#071a15]
py-3.5 rounded-xl font-extrabold text-sm border border-[#d1e8e3]
hover:bg-[#d1e8e3] transition-all disabled:opacity-60"
                  >
                    Save as Draft
                  </button>
                  <button
                    onClick={() =>
                      setForm({
                        title: "",
                        category: "",
                        location: "",
                        arrangement: "",
                        employmentType: "",
                        experienceLevel: "",
                        salaryMin: "",
                        salaryMax: "",
                        applicationDeadline: "",
                        description: "",
                        requirements: "",
                        applicationPlatform: "",
                        externalApplyLink: "",
                        contactEmail: "",
```

```
                        status: "active",
                      })
                    }
                    className="w-full text-[#6b7f79] py-2
rounded-xl font-bold text-sm hover:text-[#071a15]
transition-all"
                  >
                    Cancel
                  </button>
                </div>
              </section>
            </div>
          </div>
        )}

        {activeTab === "drafts" && (
          <div className="max-w-3xl">
            <div className="flex items-center justify-between
mb-6">
              <div>
                <p className="text-xs font-bold uppercase
tracking-widest text-[#6b7f79]">
                  Saved drafts
                </p>
                <h2 className="text-xl font-extrabold
text-[#071a15] mt-0.5">
                  {draftData.length} unfinished posts
                </h2>
              </div>
              <button
                onClick={() => setActiveTab("post")}
```

```
                className="flex items-center gap-2 bg-[#051612]
text-white px-5 py-2.5 rounded-xl font-bold text-sm
hover:bg-[#0d2a23] transition-all shadow-sm"
              >
                <PlusCircle size={17} /> New Post
              </button>
            </div>

            {draftLoading ? (
              <div className="text-sm text-[#6b7f79]">Loading
drafts...</div>
            ) : draftData.length === 0 ? (
              <div className="text-sm text-[#6b7f79]">No drafts
saved.</div>
            ) : (
              <div className="space-y-4">
                {draftData.map((draft) => (
                  <div
                    key={draft.id}
                    className="bg-white border border-gray-100
rounded-2xl p-6 flex items-center justify-between group
hover:border-[#40b594] hover:shadow-md transition-all shadow-sm"
                  >
                    <div className="flex items-center gap-5">
                      <div className="w-12 h-12 bg-[#f0f9f6]
rounded-xl flex items-center justify-center border
border-[#d1e8e3] flex-shrink-0">
                        <Briefcase size={22}
className="text-[#40b594]" />
                      </div>
                      <div>
```

```
                        <h3 className="text-base font-extrabold
text-[#071a15]">{draft.title}</h3>
                        <div className="flex items-center gap-4
mt-1">
                          <span className="text-xs font-bold
text-[#40b594] bg-[#f0f9f6] px-2.5 py-1 rounded-lg border
border-[#d1e8e3]">
                            {draft.category}
                          </span>
                          <div className="flex items-center
gap-1.5 text-[#6b7f79] text-xs font-semibold">
                            <Clock size={13} /> {draft.lastSaved}
                          </div>
                        </div>
                      </div>
                    </div>

                    <div className="flex items-center gap-2">
                      <button className="p-2.5 rounded-xl
text-[#6b7f79] hover:bg-red-50 hover:text-red-500
transition-all">
                        <Trash2 size={18} />
                      </button>
                      <Link href={`/jobs/${draft.id}`}>
                        <button className="flex items-center
gap-2 bg-[#051612] text-white px-5 py-2.5 rounded-xl font-bold
text-sm hover:bg-[#0d2a23] transition-all">
                          <FileEdit size={16} /> View Draft
                        </button>
                      </Link>
                    </div>
                  </div>
```

```
                ))}
              </div>
            )}
          </div>
        )}
      </main>
    </div>
  );
};

export default PostJob;
```

```
"use client";

import React from 'react';
import { Check, Layers, FileText, Briefcase, Mail, BarChart2,
Zap, ArrowUpRight } from 'lucide-react';
import Link from 'next/link';

const SubscriptionPlans = () => {
  const plans = [
    {
      name: 'Free',
      badge: 'Normal',
      price: '0',
      description: 'One job post per month. Perfect for casual
hiring.',
      buttonText: 'Current Plan',
      buttonVariant: 'ghost' as const,
      icon: Layers,
      iconBg: 'bg-[#0d2a23]',
```

```
    iconColor: 'text-[#40b594]',
    current: true,
    jobSlots: '1 job slot / month',
    features: [
      'Basic job listing',
      'Standard visibility',
      'Applicant dashboard',
    ],
  },
  {
    name: 'Standard',
    badge: 'Popular',
    price: '4.99',
    description: 'Post up to 3 jobs per month. Best for growing
teams.',
    buttonText: 'Subscribe Standard',
    buttonVariant: 'primary' as const,
    icon: FileText,
    iconBg: 'bg-[#133228]',
    iconColor: 'text-[#40b594]',
    current: false,
    highlight: true,
    jobSlots: '3 job slots / month',
    features: [
      'Everything in Free',
      'Standard visibility boost',
      'Email support',
      'Applicant filtering',
    ],
  },
  {
    name: 'Premium',
```

```
     badge: 'Best Value',
     price: '10.99',
     description: 'Post up to 7 jobs per month. For high-volume
hiring.',
     buttonText: 'Go Premium',
     buttonVariant: 'primary' as const,
     icon: Briefcase,
     iconBg: 'bg-[#40b594]',
     iconColor: 'text-[#051612]',
     current: false,
     jobSlots: '7 job slots / month',
     features: [
       'Everything in Standard',
       'Featured job placement',
       'Priority support',
       'Candidate analytics',
       'Advanced reporting',
     ],
   },
 ];

 return (
   <div className="min-h-screen font-sans pb-16" style={{
background: '#f0f4f3' }}>

     {/* ── HEADER — matches all employer pages ── */}
     <header className="bg-[#051612] text-white px-8 py-4 flex
items-center justify-between sticky top-0 z-50 shadow-lg">
       <div className="flex items-center gap-2.5">
         <img src="/logo.png" alt="NexHire" className="w-8 h-8"
/>
```

```
        <span className="text-xl font-extrabold
tracking-tight">NexHire</span>
      </div>

      <nav className="hidden md:flex items-center gap-8 text-sm
font-semibold">
        <Link href="/dashboard">
          <button className="text-gray-300 hover:text-white
transition-colors">Dashboard</button>
        </Link>
        <Link href="/post_job">
          <button className="text-gray-300 hover:text-white
transition-colors">Post Job</button>
        </Link>
        <Link href="/employer_message">
          <button className="text-gray-300 hover:text-white
transition-colors">Messages</button>
        </Link>
        <Link href="/employer_notification">
          <button className="text-gray-300 hover:text-white
transition-colors">Notification</button>
        </Link>
        <button className="text-[#40b594] border-b-2
border-[#40b594] pb-1">Subscription</button>
        <Link href="/employer_setting">
          <button className="text-gray-300 hover:text-white
transition-colors">Settings</button>
        </Link>
      </nav>

      <Link href="/employer_profile">
```

```
        <div className="flex items-center gap-3 cursor-pointer
group">
          <div className="text-right">
            <p className="text-[10px] text-gray-500 uppercase
tracking-widest">Company</p>
            <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors">Profile</p>
          </div>
          <div className="w-10 h-10 bg-[#40b594] rounded-full
flex items-center justify-center font-extrabold text-[#051612]
text-sm">C</div>
        </div>
      </Link>
    </header>

    <main className="max-w-7xl mx-auto px-6 md:px-10 py-10">

      {/* ── PAGE TITLE ── */}
      <div className="flex flex-col md:flex-row md:items-end
justify-between gap-6 mb-10">
        <div>
          <p className="text-xs font-bold uppercase
tracking-widest text-[#40b594] mb-1">Plans & Billing</p>
          <h1 className="text-4xl font-extrabold text-[#071a15]
leading-tight">Subscription Plans</h1>
          <p className="text-[#4a5a55] font-medium mt-1">Select
the plan that fits your hiring needs.</p>
        </div>


      </div>
```

```
        {/* ── PLAN CARDS ── */}
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
          {plans.map((plan, idx) => {
            const Icon = plan.icon;
            const isHighlight = plan.highlight;

            return (
              <div
                key={idx}
                className={`relative rounded-2xl border flex
flex-col overflow-hidden transition-all shadow-sm
hover:shadow-md ${
                  isHighlight
                    ? 'bg-[#051612] border-[#0d2a23]'
                    : 'bg-white border-gray-100'
                }`}
              >
                {/* Badge */}
                <div className="px-7 pt-7 pb-0 flex items-start
justify-between">
                  <span className={`text-xs font-extrabold px-3
py-1 rounded-full ${
                    isHighlight
                      ? 'bg-[#133228] text-[#40b594]'
                      : plan.current
                        ? 'bg-gray-100 text-[#6b7f79]'
                        : 'bg-emerald-50 text-emerald-700 ring-1
ring-emerald-200'
                  }`}>
                    {plan.badge}
                  </span>
                  <ArrowUpRight
```

```
                    size={18}
                    className={isHighlight ? 'text-[#40b594]
opacity-60' : 'text-[#6b7f79] opacity-40'}
                  />
                </div>

                {/* Icon + Name */}
                <div className="px-7 pt-6 pb-0 flex items-center
gap-4">
                  <div className={`${plan.iconBg} p-3
rounded-xl`}>
                    <Icon size={22} className={plan.iconColor} />
                  </div>
                  <div>
                    <p className={`text-xs font-bold uppercase
tracking-widest ${isHighlight ? 'text-[#40b594]' :
'text-[#6b7f79]'}`}>
                      {plan.name}
                    </p>
                    <p className={`text-xs font-semibold mt-0.5
${isHighlight ? 'text-gray-400' : 'text-[#9ab0aa]'}`}>
                      {plan.jobSlots}
                    </p>
                  </div>
                </div>

                {/* Price */}
                <div className="px-7 pt-5 pb-0">
                  <div className="flex items-baseline gap-1">
                    <span className={`text-5xl font-extrabold
leading-none ${isHighlight ? 'text-white' : 'text-[#071a15]'}`}>
                      ${plan.price}
```

```
                    </span>
                    <span className={`text-sm font-semibold
${isHighlight ? 'text-gray-400' : 'text-[#6b7f79]'}`}>/mo</span>
                  </div>
                  <p className={`text-sm font-medium mt-3
leading-relaxed ${isHighlight ? 'text-gray-300' :
'text-[#4a5a55]'}`}>
                    {plan.description}
                  </p>
                </div>

                {/* Divider */}
                <div className={`mx-7 my-6 h-px ${isHighlight ?
'bg-[#0d2a23]' : 'bg-gray-100'}`} />

                {/* Features */}
                <div className="px-7 pb-0 flex-1 space-y-3">
                  {plan.features.map((feat, fIdx) => (
                    <div key={fIdx} className="flex items-center
gap-3">
                      <div className={`w-5 h-5 rounded-full flex
items-center justify-center flex-shrink-0 ${
                        isHighlight ? 'bg-[#133228]' :
'bg-[#f0f9f6]'
                      }`}>
                        <Check size={11} className={isHighlight ?
'text-[#40b594]' : 'text-[#40b594]'} />
                      </div>
                      <span className={`text-sm font-medium
${isHighlight ? 'text-gray-300' : 'text-[#4a5a55]'}`}>
                        {feat}
                      </span>
```

```
                  </div>
                ))}
              </div>

              {/* CTA */}
              <div className="p-7">
                <button
                  className={`w-full py-3.5 rounded-xl
font-extrabold text-sm transition-all shadow-sm ${
                    plan.current
                      ? 'bg-gray-100 text-[#6b7f79]
cursor-default'
                      : isHighlight
                        ? 'bg-[#40b594] text-[#051612]
hover:bg-[#33997a]'
                        : 'bg-[#051612] text-white
hover:bg-[#0d2a23]'
                  }`}
                  disabled={plan.current}
                >
                  {plan.buttonText}
                </button>
              </div>
            </div>
          );
        })}
      </div>

      {/* ── CURRENT PLAN BANNER ── */}
      <div className="mt-8 bg-white rounded-2xl border
border-gray-100 shadow-sm px-8 py-6 flex flex-col md:flex-row
items-start md:items-center justify-between gap-4">
```

```
        <div className="flex items-center gap-4">
          <div className="w-10 h-10 bg-gray-100 rounded-xl flex
items-center justify-center">
            <Zap size={18} className="text-[#6b7f79]" />
          </div>
          <div>
            <p className="text-xs font-bold uppercase
tracking-widest text-[#6b7f79]">Current Plan</p>
            <p className="text-base font-extrabold
text-[#071a15] mt-0.5">Free Tier — 1 active job slot</p>
          </div>
        </div>
        <div className="flex items-center gap-3">
          <span className="text-xs font-bold
text-[#6b7f79]">Renews automatically · Cancel anytime</span>
          <Link href="/setting">
            <button className="text-xs font-bold text-[#40b594]
hover:underline transition-colors">
              Manage billing →
            </button>
          </Link>
        </div>
      </div>

    </main>
  </div>
 );
};

export default SubscriptionPlans;
"use client";
```

```
import React from "react";
import { Camera, ArrowLeft, Pencil, Upload, Trash2 } from
"lucide-react";
import Link from "next/link";
import { useRouter } from "next/navigation";
import ImageCropModal from "@/components/ui/ImageCropModal";

export default function EmployerProfile() {
 const router = useRouter();
 const fileInputRef = React.useRef<HTMLInputElement |
null>(null);

 const [loading, setLoading] = React.useState(false);
 const [profileImage, setProfileImage] = React.useState<File |
null>(null);
 const [sourceImageUrl, setSourceImageUrl] =
React.useState<string | null>(null);
 const [croppedPreviewUrl, setCroppedPreviewUrl] =
React.useState<string | null>(null);
 const [cropOpen, setCropOpen] = React.useState(false);

 const [form, setForm] = React.useState({
   companyDescription: "",
   companyName: "",
   industry: "",
   companySize: "",
   currentAddress: "",
   foundedYear: "",
   country: "",
   contact: "",
   websiteLink: "",
 });
```

```
 const handleChange = (
   e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
 ) => {
   setForm((prev) => ({
     ...prev,
     [e.target.name]: e.target.value,
   }));
 };

 const openFilePicker = () => {
   fileInputRef.current?.click();
 };

 const handleCircleAction = () => {
   if (sourceImageUrl) {
     setCropOpen(true);
     return;
   }

   openFilePicker();
 };

 const handleDeleteImage = () => {
   if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

   setSourceImageUrl(null);
   setCroppedPreviewUrl(null);
   setProfileImage(null);
```

```
   setCropOpen(false);

   if (fileInputRef.current) {
     fileInputRef.current.value = "";
   }
 };

 const handleImageChange = (e:
React.ChangeEvent<HTMLInputElement>) => {
   const file = e.target.files?.[0] || null;

   if (!file) return;

   if (!["image/jpeg", "image/png",
"image/webp"].includes(file.type)) {
     alert("Only JPG, PNG, and WEBP images are allowed");
     e.target.value = "";
     return;
   }

   if (file.size > 5 * 1024 * 1024) {
     alert("Image must be 5MB or smaller");
     e.target.value = "";
     return;
   }

   if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

   const objectUrl = URL.createObjectURL(file);
   setSourceImageUrl(objectUrl);
```

```
    setCroppedPreviewUrl(null);
    setProfileImage(null);
    setCropOpen(true);

    e.target.value = "";
  };

  const handleCropSave = (file: File, previewUrl: string) => {
    if (croppedPreviewUrl) {
      URL.revokeObjectURL(croppedPreviewUrl);
    }

    setProfileImage(file);
    setCroppedPreviewUrl(previewUrl);
    setCropOpen(false);
  };

  const handleCropClose = () => {
    setCropOpen(false);
  };

  React.useEffect(() => {
    return () => {
      if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
      if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
    };
  }, [sourceImageUrl, croppedPreviewUrl]);

  const handleSubmit = async () => {
    try {
      setLoading(true);
```

```
    const formData = new FormData();
    formData.append("companyDescription",
form.companyDescription);
    formData.append("companyName", form.companyName);
    formData.append("industry", form.industry);
    formData.append("companySize", form.companySize);
    formData.append("currentAddress", form.currentAddress);
    formData.append("foundedYear", form.foundedYear);
    formData.append("country", form.country);
    formData.append("contact", form.contact);
    formData.append("websiteLink", form.websiteLink);

    if (profileImage) {
      formData.append("profileImage", profileImage);
    }

    const res = await fetch("/api/auth/employer-profile", {
      method: "POST",
      body: formData,
    });

    const text = await res.text();
    const result = text ? JSON.parse(text) : {};

    if (!res.ok) {
      alert(result.error || "Failed to save profile");
      setLoading(false);
      return;
    }

    if (!result.next) {
```

```
        alert("Missing redirect page");
        setLoading(false);
        return;
      }

      router.push(result.next);
    } catch (error) {
      console.error("EMPLOYER SUBMIT ERROR:", error);
      alert("Something went wrong");
      setLoading(false);
    }
  };

  return (
    <>
      <div className="min-h-screen w-full bg-[#f8fafc]
font-sans">
        <div className="absolute left-0 top-0 z-0 h-64 w-full
bg-[#0d2a23]" />

        <div className="relative z-10 mx-auto max-w-7xl p-4
md:p-8">
          <div className="mb-10 flex items-center justify-between
px-4">
            <div className="flex items-center gap-2">
              <img src="/logo.png" alt="NexHire" className="h-8
w-8" />
              <span className="text-xl font-bold tracking-tight
text-white">
                NexHire
              </span>
            </div>
```

```
          <Link
            href="/role_choosing"
            className="flex items-center gap-2 rounded-full
bg-white px-6 py-2 text-sm font-bold text-[#0d2a23] shadow-md
transition-all hover:bg-gray-100"
          >
            <ArrowLeft size={18} />
            Back
          </Link>
        </div>

        <div className="flex flex-col gap-12 rounded-[35px]
bg-white p-8 shadow-2xl md:p-14 lg:flex-row lg:gap-20">
          <div className="flex flex-col items-center lg:w-1/3">
            <h2 className="mb-10 text-2xl font-bold
text-[#1a1a1a]">
              Employer&apos;s Profile
            </h2>

            <div className="relative mb-6">
              <div className="flex h-52 w-52 items-center
justify-center overflow-hidden rounded-full bg-[#4a907d]">
                {croppedPreviewUrl ? (
                  <img
                    src={croppedPreviewUrl}
                    alt="Employer preview"
                    className="h-full w-full object-cover"
                  />
                ) : (
                  <svg
                    className="mt-6 h-32 w-32 text-white/80"
```

```
                    fill="currentColor"
                    viewBox="0 0 24 24"
                  >
                    <path d="M12 12c2.21 0 4-1.79
4-4s-1.79-4-4-4-4 1.79-4 4 1.79 4 4 4zm0 2c-2.67 0-8 1.34-8
4v2h16v-2c0-2.66-5.33-4-8-4z" />
                  </svg>
                )}
              </div>

              <button
                type="button"
                onClick={handleCircleAction}
                title={sourceImageUrl ? "Edit image" : "Upload
image"}
                className="absolute bottom-3 right-3 flex h-12
w-12 items-center justify-center rounded-full border-4
border-white bg-[#00a37b] text-white transition-transform
hover:scale-110"
              >
                {sourceImageUrl ? <Pencil size={20} /> :
<Camera size={22} />}
              </button>

              <input
                ref={fileInputRef}
                type="file"
                accept="image/jpeg,image/png,image/webp"
                onChange={handleImageChange}
                className="hidden"
              />
            </div>
```

```
            <p className="mb-10 text-xs text-gray-400">
              Click the icon on the photo to upload first, then
edit crop later
            </p>

            <div className="w-full space-y-2">
              <label className="ml-1 text-sm font-bold
text-gray-700">
                Company Description <span
className="text-red-500">*</span>
              </label>
              <textarea
                name="companyDescription"
                value={form.companyDescription}
                onChange={handleChange}
                placeholder="Enter company description"
                className="h-40 w-full resize-none rounded-2xl
border border-gray-200 bg-[#f8fafc] p-4 text-sm text-gray-700
transition-all focus:border-[#00a37b] focus:bg-white
focus:outline-none focus:ring-2 focus:ring-[#00a37b]/15"
              />
            </div>
          </div>

          <div className="lg:w-2/3">
            <h3 className="mb-8 text-xl font-bold
text-[#1a1a1a]">
              User Information
            </h3>
```

```
            <div className="grid grid-cols-1 gap-x-8 gap-y-6
md:grid-cols-2">
              <div className="space-y-2 md:col-span-2">
                <label className="text-sm font-bold
text-gray-700">
                  Company name <span
className="text-red-500">*</span>
                </label>
                <input
                  name="companyName"
                  value={form.companyName}
                  onChange={handleChange}
                  type="text"
                  placeholder="Enter company name"
                  className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
transition-all caret-[#00a37b] focus:border-[#00a37b]
focus:bg-white focus:outline-none focus:ring-2
focus:ring-[#00a37b]/15"
                />
              </div>

              <div className="space-y-2">
                <label className="text-sm font-bold
text-gray-700">Industry</label>
                <input
                  name="industry"
                  value={form.industry}
                  onChange={handleChange}
                  type="text"
                  placeholder="Enter industry"
```

```
                  className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
transition-all caret-[#00a37b] focus:border-[#00a37b]
focus:bg-white focus:outline-none focus:ring-2
focus:ring-[#00a37b]/15"
                />
              </div>

              <div className="space-y-2">
                <label className="text-sm font-bold
text-gray-700">
                  Company Size <span
className="text-red-500">*</span>
                </label>
                <input
                  name="companySize"
                  value={form.companySize}
                  onChange={handleChange}
                  type="text"
                  placeholder="Enter company size"
                  className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
transition-all caret-[#00a37b] focus:border-[#00a37b]
focus:bg-white focus:outline-none focus:ring-2
focus:ring-[#00a37b]/15"
                />
              </div>

              <div className="space-y-2 md:col-span-2">
                <label className="text-sm font-bold
text-gray-700">
```

```
                    Current Address <span
className="text-red-500">*</span>
                  </label>
                  <input
                    name="currentAddress"
                    value={form.currentAddress}
                    onChange={handleChange}
                    type="text"
                    placeholder="Enter current address"
                    className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
transition-all caret-[#00a37b] focus:border-[#00a37b]
focus:bg-white focus:outline-none focus:ring-2
focus:ring-[#00a37b]/15"
                  />
                </div>

                <div className="space-y-2">
                  <label className="text-sm font-bold
text-gray-700">
                    Founded year <span
className="text-red-500">*</span>
                  </label>
                  <select
                    name="foundedYear"
                    value={form.foundedYear}
                    onChange={handleChange}
                    className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
transition-all focus:border-[#00a37b] focus:bg-white
focus:outline-none focus:ring-2 focus:ring-[#00a37b]/15"
                  >
```

```
                    <option value="">Select option</option>
                    {Array.from({ length: 60 }, (_, i) => {
                      const year = new Date().getFullYear() - i;
                      return (
                        <option key={year} value={String(year)}>
                          {year}
                        </option>
                      );
                    })}
                  </select>
                </div>

                <div className="space-y-2">
                  <label className="text-sm font-bold
text-gray-700">
                    Country <span
className="text-red-500">*</span>
                  </label>
                  <input
                    name="country"
                    value={form.country}
                    onChange={handleChange}
                    type="text"
                    placeholder="Enter country"
                    className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
transition-all caret-[#00a37b] focus:border-[#00a37b]
focus:bg-white focus:outline-none focus:ring-2
focus:ring-[#00a37b]/15"
                  />
                </div>
```

```
                <div className="space-y-2">
                  <label className="text-sm font-bold
text-gray-700">
                    Contact <span
className="text-red-500">*</span>
                  </label>
                  <input
                    name="contact"
                    value={form.contact}
                    onChange={handleChange}
                    type="text"
                    placeholder="Enter contact"
                    className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
transition-all caret-[#00a37b] focus:border-[#00a37b]
focus:bg-white focus:outline-none focus:ring-2
focus:ring-[#00a37b]/15"
                  />
                </div>

                <div className="space-y-2">
                  <label className="text-sm font-bold
text-gray-700">
                    Website Link
                  </label>
                  <input
                    name="websiteLink"
                    value={form.websiteLink}
                    onChange={handleChange}
                    type="text"
                    placeholder="Enter website link"
```

```
                  className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
transition-all caret-[#00a37b] focus:border-[#00a37b]
focus:bg-white focus:outline-none focus:ring-2
focus:ring-[#00a37b]/15"
                />
              </div>
            </div>

            <div className="mt-12 flex justify-end">
              <button
                type="button"
                onClick={handleSubmit}
                disabled={loading}
                className="rounded-2xl bg-[#2d4f45] px-16 py-4
text-lg font-bold text-white shadow-xl shadow-[#2d4f45]/20
transition-all hover:bg-[#1e3a32] disabled:cursor-not-allowed
disabled:opacity-60"
              >
                {loading ? "Saving..." : "Next"}
              </button>
            </div>
          </div>
        </div>
      </div>

      <ImageCropModal
        open={cropOpen}
        imageSrc={sourceImageUrl}
        title="Adjust company profile image"
        onClose={handleCropClose}
```

```
        onSave={handleCropSave}
        onUploadAnother={openFilePicker}
        onDelete={handleDeleteImage}
        hasImage={!!sourceImageUrl}
      />
    </>
 );
}

"use client";

import React from "react";
import { Camera, ArrowLeft, Pencil } from "lucide-react";
import Link from "next/link";
import { useRouter } from "next/navigation";
import ImageCropModal from "@/components/ui/ImageCropModal";

export default function JobSeekerProfile() {
 const router = useRouter();
 const fileInputRef = React.useRef<HTMLInputElement |
null>(null);

 const [loading, setLoading] = React.useState(false);
 const [profileImage, setProfileImage] = React.useState<File |
null>(null);
 const [sourceImageUrl, setSourceImageUrl] =
React.useState<string | null>(null);
 const [croppedPreviewUrl, setCroppedPreviewUrl] =
React.useState<string | null>(null);
 const [cropOpen, setCropOpen] = React.useState(false);

 const [form, setForm] = React.useState({
```

```
  description: "",
  firstName: "",
  lastName: "",
  contact: "",
  address: "",
  educationLevel: "",
  schoolUniversity: "",
  year: "",
});

const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
) => {
  setForm((prev) => ({
    ...prev,
    [e.target.name]: e.target.value,
  }));
};

const openFilePicker = () => {
  fileInputRef.current?.click();
};

const handleCircleAction = () => {
  if (sourceImageUrl) {
    setCropOpen(true);
    return;
  }

  openFilePicker();
};
```

```
 const handleDeleteImage = () => {
   if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
   if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

   setSourceImageUrl(null);
   setCroppedPreviewUrl(null);
   setProfileImage(null);
   setCropOpen(false);

   if (fileInputRef.current) {
     fileInputRef.current.value = "";
   }
 };

 const handleImageChange = (e:
React.ChangeEvent<HTMLInputElement>) => {
   const file = e.target.files?.[0] || null;

   if (!file) return;

   if (!["image/jpeg", "image/png",
"image/webp"].includes(file.type)) {
     alert("Only JPG, PNG, and WEBP images are allowed");
     e.target.value = "";
     return;
   }

   if (file.size > 5 * 1024 * 1024) {
     alert("Image must be 5MB or smaller");
     e.target.value = "";
```

```
      return;
    }

    if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
    if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);

    const objectUrl = URL.createObjectURL(file);
    setSourceImageUrl(objectUrl);
    setCroppedPreviewUrl(null);
    setProfileImage(null);
    setCropOpen(true);

    e.target.value = "";
  };

  const handleCropSave = (file: File, previewUrl: string) => {
    if (croppedPreviewUrl) {
      URL.revokeObjectURL(croppedPreviewUrl);
    }

    setProfileImage(file);
    setCroppedPreviewUrl(previewUrl);
    setCropOpen(false);
  };

  const handleCropClose = () => {
    setCropOpen(false);
  };

  React.useEffect(() => {
    return () => {
```

```
    if (sourceImageUrl) URL.revokeObjectURL(sourceImageUrl);
    if (croppedPreviewUrl)
URL.revokeObjectURL(croppedPreviewUrl);
  };
 }, [sourceImageUrl, croppedPreviewUrl]);

 const handleSubmit = async () => {
  try {
    setLoading(true);

    const formData = new FormData();
    formData.append("description", form.description);
    formData.append("firstName", form.firstName);
    formData.append("lastName", form.lastName);
    formData.append("contact", form.contact);
    formData.append("address", form.address);
    formData.append("educationLevel", form.educationLevel);
    formData.append("schoolUniversity", form.schoolUniversity);
    formData.append("year", form.year);

    if (profileImage) {
      formData.append("profileImage", profileImage);
    }

    const res = await fetch("/api/auth/job-seeker-profile", {
      method: "POST",
      body: formData,
    });

    const text = await res.text();
    const result = text ? JSON.parse(text) : {};
```

```
    if (!res.ok) {
      alert(result.error || "Failed to save profile");
      setLoading(false);
      return;
    }

    if (!result.next) {
      alert("Missing redirect page");
      setLoading(false);
      return;
    }

    router.push(result.next);
  } catch (error) {
    console.error("JOB SEEKER SUBMIT ERROR:", error);
    alert("Something went wrong");
    setLoading(false);
  }
};

return (
  <>
    <div className="min-h-screen w-full bg-[#f8fafc]
font-sans">
      <div className="absolute left-0 top-0 z-0 h-64 w-full
bg-[#0d2a23]" />

      <div className="relative z-10 mx-auto max-w-6xl p-4
md:p-8">
        <div className="mb-12 flex items-center
justify-between">
          <div className="flex items-center gap-2">
```

```
            <img src="/logo.png" alt="NexHire" className="h-8
w-8" />
            <span className="text-xl font-bold tracking-tight
text-white">NexHire</span>
          </div>
          <Link
            href="/role_choosing"
            className="flex items-center gap-2 rounded-full
bg-white px-6 py-2 text-sm font-bold text-[#0d2a23] shadow-md
transition-all hover:bg-gray-100"
          >
            <ArrowLeft size={18} />
            Back
          </Link>
        </div>

        <div className="flex flex-col gap-16 rounded-[30px]
bg-white p-8 shadow-2xl md:p-16 lg:flex-row">
          <div className="flex flex-col items-center lg:w-1/3">
            <h2 className="mb-10 text-center text-2xl font-bold
text-[#1a1a1a]">
              Job Seeker&apos;s Profile
            </h2>

            <div className="relative mb-6">
              <div className="flex h-48 w-48 items-center
justify-center overflow-hidden rounded-full bg-[#4a907d]">
                {croppedPreviewUrl ? (
                  <img
                    src={croppedPreviewUrl}
                    alt="Job seeker preview"
                    className="h-full w-full object-cover"
```

```
                    />
                  ) : (
                    <svg
                      className="mt-8 h-32 w-32 text-white/80"
                      fill="currentColor"
                      viewBox="0 0 24 24"
                    >
                      <path d="M12 12c2.21 0 4-1.79
4-4s-1.79-4-4-4-4 1.79-4 4 1.79 4 4 4zm0 2c-2.67 0-8 1.34-8
4v2h16v-2c0-2.66-5.33-4-8-4z" />
                    </svg>
                  )}
                </div>

                <button
                  type="button"
                  onClick={handleCircleAction}
                  title={sourceImageUrl ? "Edit image" : "Upload
image"}
                  className="absolute bottom-2 right-2 flex h-12
w-12 items-center justify-center rounded-full border-4
border-white bg-[#00a37b] text-white transition-transform
hover:scale-105"
                >
                  {sourceImageUrl ? <Pencil size={18} /> :
<Camera size={20} />}
                </button>

                <input
                  ref={fileInputRef}
                  type="file"
                  accept="image/jpeg,image/png,image/webp"
```

```
                onChange={handleImageChange}
                className="hidden"
              />
            </div>

            <p className="mb-8 text-xs text-gray-400">
              Click the icon on the photo to upload first, then
edit crop later
            </p>

            <div className="w-full space-y-2">
              <label className="ml-1 text-sm font-medium
text-gray-600">
                Description (optional)
              </label>
              <textarea
                name="description"
                value={form.description}
                onChange={handleChange}
                className="h-32 w-full resize-none rounded-2xl
border border-gray-200 bg-[#f8fafc] p-4 text-sm text-gray-700
transition-all focus:border-[#00a37b] focus:bg-white
focus:outline-none focus:ring-2 focus:ring-[#00a37b]/15"
              />
            </div>
          </div>

          <div className="space-y-10 lg:w-2/3">
            <section>
              <h3 className="mb-6 text-xl font-bold
text-[#1a1a1a]">User Information</h3>
```

```
                <div className="grid grid-cols-1 gap-6
md:grid-cols-2">
                  <div className="space-y-2">
                    <label className="text-sm font-bold
text-gray-700">
                      First name <span
className="text-red-500">*</span>
                    </label>
                    <input
                      name="firstName"
                      value={form.firstName}
                      onChange={handleChange}
                      type="text"
                      placeholder="Enter first name"
                      className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
focus:border-[#00a37b] focus:bg-white focus:outline-none
focus:ring-2 focus:ring-[#00a37b]/15"
                    />
                  </div>

                  <div className="space-y-2">
                    <label className="text-sm font-bold
text-gray-700">
                      Last name <span
className="text-red-500">*</span>
                    </label>
                    <input
                      name="lastName"
                      value={form.lastName}
                      onChange={handleChange}
                      type="text"
```

```
                    placeholder="Enter last name"
                    className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
focus:border-[#00a37b] focus:bg-white focus:outline-none
focus:ring-2 focus:ring-[#00a37b]/15"
                  />
                </div>

                <div className="space-y-2 md:col-span-2">
                  <label className="text-sm font-bold
text-gray-700">
                    Contact <span
className="text-red-500">*</span>
                  </label>
                  <input
                    name="contact"
                    value={form.contact}
                    onChange={handleChange}
                    type="text"
                    placeholder="Enter contact"
                    className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
focus:border-[#00a37b] focus:bg-white focus:outline-none
focus:ring-2 focus:ring-[#00a37b]/15"
                  />
                </div>

                <div className="space-y-2 md:col-span-2">
                  <label className="text-sm font-bold
text-gray-700">
                    Address <span
className="text-red-500">*</span>
```

```
                  </label>
                  <input
                    name="address"
                    value={form.address}
                    onChange={handleChange}
                    type="text"
                    placeholder="Enter address"
                    className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
focus:border-[#00a37b] focus:bg-white focus:outline-none
focus:ring-2 focus:ring-[#00a37b]/15"
                  />
                </div>
              </div>
            </section>

            <section>
              <h3 className="mb-6 text-xl font-bold
text-[#1a1a1a]">Education</h3>
              <div className="grid grid-cols-1 gap-6
md:grid-cols-2">
                <div className="space-y-2">
                  <label className="text-sm font-bold
text-gray-700">
                    Education level <span
className="text-red-500">*</span>
                  </label>
                  <select
                    name="educationLevel"
                    value={form.educationLevel}
                    onChange={handleChange}
```

```
                      className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
focus:border-[#00a37b] focus:bg-white focus:outline-none
focus:ring-2 focus:ring-[#00a37b]/15"
                    >
                      <option value="">Select option</option>
                      <option value="High School">High
School</option>
                      <option
value="Associate">Associate</option>
                      <option value="Bachelor">Bachelor</option>
                      <option value="Master">Master</option>
                      <option value="PhD">PhD</option>
                    </select>
                  </div>

                  <div className="space-y-2">
                    <label className="text-sm font-bold
text-gray-700">
                      School / University <span
className="text-red-500">*</span>
                    </label>
                    <input
                      name="schoolUniversity"
                      value={form.schoolUniversity}
                      onChange={handleChange}
                      type="text"
                      placeholder="Enter school or university"
                      className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
focus:border-[#00a37b] focus:bg-white focus:outline-none
focus:ring-2 focus:ring-[#00a37b]/15"
```

```
                    />
                  </div>

                  <div className="space-y-2">
                    <label className="text-sm font-bold
text-gray-700">
                      Year <span
className="text-red-500">*</span>
                    </label>
                    <input
                      name="year"
                      value={form.year}
                      onChange={handleChange}
                      type="text"
                      placeholder="Enter year"
                      className="w-full rounded-xl border
border-gray-200 bg-[#f8fafc] px-5 py-3.5 text-sm text-gray-700
focus:border-[#00a37b] focus:bg-white focus:outline-none
focus:ring-2 focus:ring-[#00a37b]/15"
                    />
                  </div>

                  <div className="mt-4 flex items-end
justify-end">
                    <button
                      type="button"
                      onClick={handleSubmit}
                      disabled={loading}
                      className="w-48 rounded-2xl bg-[#2d4f45]
py-3.5 font-bold text-white shadow-xl shadow-[#2d4f45]/20
transition-all hover:bg-[#1e3a32] disabled:cursor-not-allowed
disabled:opacity-60"
```

```
                    >
                      {loading ? "Saving..." : "Next"}
                    </button>
                  </div>
                </div>
              </section>
            </div>
          </div>
        </div>
      </div>

      <ImageCropModal
        open={cropOpen}
        imageSrc={sourceImageUrl}
        title="Adjust profile photo"
        onClose={handleCropClose}
        onSave={handleCropSave}
        onUploadAnother={openFilePicker}
        onDelete={handleDeleteImage}
        hasImage={!!sourceImageUrl}
      />
    </>
  );
}

import {
  pgTable,
  uuid,
  text,
  boolean,
  timestamp,
  pgEnum,
```

```
 jsonb,
 integer,
 uniqueIndex,
} from "drizzle-orm/pg-core";

export const userRoleEnum = pgEnum("user_role", ["employer",
"job_seeker"]);

export const otpPurposeEnum = pgEnum("otp_purpose", [
 "signup",
 "login",
 "forgot_password",
 "google_signup",
 "google_login",
]);

export const jobArrangementEnum = pgEnum("job_arrangement", [
 "on_site",
 "remote",
 "hybrid",
]);

export const employmentTypeEnum = pgEnum("employment_type", [
 "full_time",
 "part_time",
 "contract",
 "freelance",
 "internship",
]);

export const experienceLevelEnum = pgEnum("experience_level", [
 "entry",
```

```
 "mid",
 "senior",
 "lead",
 "executive",
]);

export const jobStatusEnum = pgEnum("job_status", [
 "draft",
 "active",
 "closed",
]);

export const applicationStatusEnum =
pgEnum("application_status", [
 "pending",
 "accepted",
 "rejected",
 "interview",
 "withdrawn",
]);

export const notificationTypeEnum = pgEnum("notification_type",
[
 "system",
 "application",
 "interview",
 "message",
]);

export const interviewModeEnum = pgEnum("interview_mode", [
 "remote",
 "onsite",
```

```
]);

export const users = pgTable("users", {
 id: uuid("id").defaultRandom().primaryKey(),
 email: text("email").notNull().unique(),
 passwordHash: text("password_hash"),
 googleId: text("google_id").unique(),
 provider: text("provider").notNull().default("credentials"),
 isEmailVerified:
boolean("is_email_verified").notNull().default(false),
 role: userRoleEnum("role"),
 onboardingCompleted:
boolean("onboarding_completed").notNull().default(false),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
 updatedAt: timestamp("updated_at", { withTimezone: true
}).defaultNow().notNull(),
});

export const otpCodes = pgTable("otp_codes", {
 id: uuid("id").defaultRandom().primaryKey(),
 email: text("email").notNull(),
 codeHash: text("code_hash").notNull(),
 purpose: otpPurposeEnum("purpose").notNull(),
 data: jsonb("data").$type<Record<string, unknown> |
null>().default(null),
 expiresAt: timestamp("expires_at", { withTimezone: true
}).notNull(),
 usedAt: timestamp("used_at", { withTimezone: true }),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
});
```

```
export const sessions = pgTable("sessions", {
 id: uuid("id").defaultRandom().primaryKey(),
 userId: uuid("user_id").notNull(),
 tokenHash: text("token_hash").notNull().unique(),
 type: text("type").notNull().default("auth"),
 expiresAt: timestamp("expires_at", { withTimezone: true
}).notNull(),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
});

export const employerProfiles = pgTable("employer_profiles", {
 id: uuid("id").defaultRandom().primaryKey(),
 userId: uuid("user_id").notNull().unique(),
 profileImage: text("profile_image"),
 companyDescription: text("company_description"),
 companyName: text("company_name").notNull(),
 industry: text("industry"),
 companySize: text("company_size"),
 currentAddress: text("current_address"),
 foundedYear: text("founded_year"),
 country: text("country"),
 contact: text("contact"),
 websiteLink: text("website_link"),
 companyFileName: text("company_file_name"),
 companyFileUrl: text("company_file_url"),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
});
```

```
export const jobSeekerProfiles = pgTable("job_seeker_profiles",
{
 id: uuid("id").defaultRandom().primaryKey(),
 userId: uuid("user_id").notNull().unique(),
 profileImage: text("profile_image"),
 description: text("description"),
 firstName: text("first_name").notNull(),
 lastName: text("last_name").notNull(),
 contact: text("contact"),
 address: text("address"),
 educationLevel: text("education_level"),
 schoolUniversity: text("school_university"),
 year: text("year"),
 cvFileName: text("cv_file_name"),
 cvUrl: text("cv_url"),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
});

export const jobs = pgTable("jobs", {
 id: uuid("id").defaultRandom().primaryKey(),
 employerId: uuid("employer_id").notNull(),
 title: text("title").notNull(),
 category: text("category").notNull(),
 location: text("location").notNull(),
 arrangement:
jobArrangementEnum("arrangement").notNull().default("on_site"),
 employmentType:
employmentTypeEnum("employment_type").notNull().default("full_ti
me"),
```

```
 experienceLevel:
experienceLevelEnum("experience_level").notNull().default("entry
"),
 salaryMin: integer("salary_min"),
 salaryMax: integer("salary_max"),
 description: text("description").notNull(),
 requirements: text("requirements"),
 applicationDeadline: timestamp("application_deadline", {
withTimezone: true }),
 applicationPlatform:
text("application_platform").notNull().default("internal"),
 externalApplyLink: text("external_apply_link"),
 contactEmail: text("contact_email"),
 status: jobStatusEnum("status").notNull().default("draft"),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
 updatedAt: timestamp("updated_at", { withTimezone: true
}).defaultNow().notNull(),
});

export const savedJobs = pgTable(
 "saved_jobs",
 {
   id: uuid("id").defaultRandom().primaryKey(),
   jobId: uuid("job_id").notNull(),
   jobSeekerId: uuid("job_seeker_id").notNull(),
   createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
 },
 (table) => ({
   savedJobsUnique:
uniqueIndex("saved_jobs_job_seeker_job_idx").on(
```

```
      table.jobId,
      table.jobSeekerId
    ),
  })
);

export const jobApplications = pgTable(
  "job_applications",
  {
    id: uuid("id").defaultRandom().primaryKey(),
    jobId: uuid("job_id").notNull(),
    employerId: uuid("employer_id").notNull(),
    jobSeekerId: uuid("job_seeker_id").notNull(),
    status:
applicationStatusEnum("status").notNull().default("pending"),
    coverLetter: text("cover_letter"),
    cvUrl: text("cv_url"),
    cvFileName: text("cv_file_name"),
    appliedAt: timestamp("applied_at", { withTimezone: true
}).defaultNow().notNull(),
    updatedAt: timestamp("updated_at", { withTimezone: true
}).defaultNow().notNull(),
  },
  (table) => ({
    jobApplicationsUnique:
uniqueIndex("job_applications_job_user_idx").on(
      table.jobId,
      table.jobSeekerId
    ),
  })
);
```

```
export const interviews = pgTable("interviews", {
 id: uuid("id").defaultRandom().primaryKey(),
 applicationId: uuid("application_id").notNull(),
 employerId: uuid("employer_id").notNull(),
 jobSeekerId: uuid("job_seeker_id").notNull(),
 mode: interviewModeEnum("mode").notNull().default("remote"),
 scheduledAt: timestamp("scheduled_at", { withTimezone: true
}).notNull(),
 location: text("location"),
 notes: text("notes"),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
});

export const notifications = pgTable("notifications", {
 id: uuid("id").defaultRandom().primaryKey(),
 recipientId: uuid("recipient_id").notNull(),
 actorId: uuid("actor_id"),
 type: notificationTypeEnum("type").notNull().default("system"),
 title: text("title").notNull(),
 description: text("description").notNull(),
 link: text("link"),
 meta: jsonb("meta").$type<Record<string, unknown> |
null>().default(null),
 readAt: timestamp("read_at", { withTimezone: true }),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
});

export const conversations = pgTable(
 "conversations",
 {
```

```
   id: uuid("id").defaultRandom().primaryKey(),
   employerId: uuid("employer_id").notNull(),
   jobSeekerId: uuid("job_seeker_id").notNull(),
   jobId: uuid("job_id"),
   createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
   lastMessageAt: timestamp("last_message_at", { withTimezone:
true }).defaultNow().notNull(),
 },
 (table) => ({
   conversationsUnique:
uniqueIndex("conversations_employer_jobseeker_job_idx").on(
     table.employerId,
     table.jobSeekerId,
     table.jobId
   ),
 })
);

export const messages = pgTable("messages", {
 id: uuid("id").defaultRandom().primaryKey(),
 conversationId: uuid("conversation_id").notNull(),
 senderId: uuid("sender_id").notNull(),
 text: text("text"),
 attachmentUrl: text("attachment_url"),
 attachmentName: text("attachment_name"),
 attachmentType: text("attachment_type"),
 editedAt: timestamp("edited_at", { withTimezone: true }),
 createdAt: timestamp("created_at", { withTimezone: true
}).defaultNow().notNull(),
});
```

```
export const messageDeletes = pgTable(
 "message_deletes",
 {
   id: uuid("id").defaultRandom().primaryKey(),
   messageId: uuid("message_id").notNull(),
   userId: uuid("user_id").notNull(),
   deletedAt: timestamp("deleted_at", { withTimezone: true
}).defaultNow().notNull(),
 },
 (table) => ({
   messageDeletesUnique:
uniqueIndex("message_deletes_message_user_idx").on(
     table.messageId,
     table.userId
   ),
 })
);

"use client";

import React, { useEffect, useState } from "react";
import Link from "next/link";

type MeResponse = {
 userId: string;
 email: string;
 role: "employer" | "job_seeker";
 displayName: string;
 avatar: string | null;
 profilePath: string;
};
```

```
export default function EmployerNavProfile() {
 const [me, setMe] = useState<MeResponse | null>(null);

 useEffect(() => {
   const load = async () => {
     try {
       const res = await fetch("/api/auth/me", { cache:
"no-store" });
       const data = await res.json();
       if (res.ok) {
         setMe(data);
       }
     } catch (error) {
       console.error("LOAD EMPLOYER NAV PROFILE ERROR:", error);
     }
   };

   load();
 }, []);

 const displayName = me?.displayName || "Company";
 const avatar = me?.avatar || null;
 const profilePath = me?.profilePath || "/employer_profile";
 const initial = displayName.charAt(0).toUpperCase() || "C";

 return (
   <Link href={profilePath}>
     <div className="flex items-center gap-3 cursor-pointer
group">
       <div className="text-right">
         <p className="text-[10px] text-gray-500 uppercase
tracking-widest">Company</p>
```

```
          <p className="text-sm font-bold text-white
group-hover:text-[#40b594] transition-colors truncate
max-w-[160px]">
            {displayName}
          </p>
        </div>

        <div className="w-10 h-10 rounded-full overflow-hidden
bg-[#40b594] flex items-center justify-center font-extrabold
text-[#051612] text-sm">
          {avatar ? (
            <img src={avatar} alt={displayName} className="h-full
w-full object-cover" />
          ) : (
            initial
          )}
        </div>
      </div>
    </Link>
  );
}

"use client";

import React from "react";
import Cropper from "react-easy-crop";
import { Upload, Trash2 } from "lucide-react";
import type { Area } from "@/lib/crop-image";
import { getCroppedImg } from "@/lib/crop-image";

type ImageCropModalProps = {
  open: boolean;
```

```
 imageSrc: string | null;
 title: string;
 onClose: () => void;
 onSave: (file: File, previewUrl: string) => void;
 onUploadAnother: () => void;
 onDelete: () => void;
 hasImage: boolean;
};

export default function ImageCropModal({
 open,
 imageSrc,
 title,
 onClose,
 onSave,
 onUploadAnother,
 onDelete,
 hasImage,
}: ImageCropModalProps) {
 const [crop, setCrop] = React.useState({ x: 0, y: 0 });
 const [zoom, setZoom] = React.useState(1);
 const [croppedAreaPixels, setCroppedAreaPixels] =
React.useState<Area | null>(null);
 const [saving, setSaving] = React.useState(false);

 React.useEffect(() => {
   if (!open) return;
   setCrop({ x: 0, y: 0 });
   setZoom(1);
   setCroppedAreaPixels(null);
 }, [open, imageSrc]);
```

```
 const onCropComplete = React.useCallback((_: Area,
croppedPixels: Area) => {
   setCroppedAreaPixels(croppedPixels);
 }, []);

 const handleSave = async () => {
   if (!imageSrc || !croppedAreaPixels) return;

   try {
     setSaving(true);
     const croppedFile = await getCroppedImg(imageSrc,
croppedAreaPixels, "profile.jpg");
     const previewUrl = URL.createObjectURL(croppedFile);
     onSave(croppedFile, previewUrl);
   } catch (error) {
     console.error("CROP SAVE ERROR:", error);
     alert("Failed to crop image");
   } finally {
     setSaving(false);
   }
 };

 if (!open || !imageSrc) return null;

 return (
   <div className="fixed inset-0 z-[100] flex items-center
justify-center bg-black/70 px-4">
     <div className="w-full max-w-2xl rounded-[28px] bg-white
p-6 shadow-2xl">
       <div className="mb-4 flex items-center justify-between">
         <h3 className="text-lg font-bold
text-[#1a1a1a]">{title}</h3>
```

```
          <button
            type="button"
            onClick={onClose}
            className="rounded-full px-3 py-1 text-sm font-medium
text-gray-500 hover:bg-gray-100"
          >
            Close
          </button>
        </div>

        <div className="relative h-[360px] overflow-hidden
rounded-2xl bg-[#0d2a23]">
          <Cropper
            image={imageSrc}
            crop={crop}
            zoom={zoom}
            aspect={1}
            cropShape="round"
            showGrid={false}
            onCropChange={setCrop}
            onCropComplete={onCropComplete}
            onZoomChange={setZoom}
          />
        </div>

        <div className="mt-5">
          <label className="mb-2 block text-sm font-semibold
text-gray-700">
            Zoom
          </label>
          <input
            type="range"
```

```
          min={1}
          max={3}
          step={0.1}
          value={zoom}
          onChange={(e) => setZoom(Number(e.target.value))}
          className="w-full accent-[#00a37b]"
        />
      </div>

      <div className="mt-5 flex flex-wrap gap-3">
        <button
          type="button"
          onClick={onUploadAnother}
          className="inline-flex items-center gap-2 rounded-xl
border border-gray-200 px-4 py-2 text-sm font-semibold
text-gray-700 hover:bg-gray-50"
        >
          <Upload size={16} />
          Upload Again
        </button>

        <button
          type="button"
          onClick={() => {
            onDelete();
            onClose();
          }}
          disabled={!hasImage}
          className="inline-flex items-center gap-2 rounded-xl
border border-red-200 px-4 py-2 text-sm font-semibold
text-red-600 hover:bg-red-50 disabled:cursor-not-allowed
disabled:opacity-50"
```

```
          >
            <Trash2 size={16} />
            Delete
          </button>
        </div>

        <div className="mt-6 flex justify-end gap-3">
          <button
            type="button"
            onClick={onClose}
            className="rounded-xl border border-gray-200 px-5
py-3 text-sm font-semibold text-gray-700 hover:bg-gray-50"
          >
            Cancel
          </button>
          <button
            type="button"
            onClick={handleSave}
            disabled={saving}
            className="rounded-xl bg-[#00a37b] px-5 py-3 text-sm
font-semibold text-white hover:bg-[#008f6c] disabled:opacity-60"
          >
            {saving ? "Saving..." : "Apply Crop"}
          </button>
        </div>
      </div>
    </div>
  );
}

"use client";
```

```
import React, { useEffect, useState } from "react";
import Link from "next/link";

type MeResponse = {
 userId: string;
 email: string;
 role: "employer" | "job_seeker";
 displayName: string;
 avatar: string | null;
 profilePath: string;
};

export default function UserNavProfile() {
 const [me, setMe] = useState<MeResponse | null>(null);

 useEffect(() => {
   const load = async () => {
     try {
       const res = await fetch("/api/auth/me", { cache:
"no-store" });
       const data = await res.json();
       if (res.ok) {
         setMe(data);
       }
     } catch (error) {
       console.error("LOAD USER NAV PROFILE ERROR:", error);
     }
   };

   load();
 }, []);
```

```
 const displayName = me?.displayName || "Profile";
 const avatar = me?.avatar || null;
 const profilePath = me?.profilePath || "/profile";
 const initial = displayName.charAt(0).toUpperCase() || "U";

 return (
   <Link href={profilePath}>
     <div className="flex items-center gap-4">
       <div className="text-right">
         <p className="text-[10px] uppercase tracking-wider
text-gray-400">User name</p>
         <p className="text-sm font-bold truncate
max-w-[140px]">{displayName}</p>
       </div>

       <div className="flex h-10 w-10 items-center
justify-center overflow-hidden rounded-full bg-[#2d4f45]
font-bold text-white">
         {avatar ? (
           <img src={avatar} alt={displayName} className="h-full
w-full object-cover" />
         ) : (
           initial
         )}
       </div>
     </div>
   </Link>
 );
}

import { NextResponse } from "next/server";
import { eq } from "drizzle-orm";
```

```
import { db } from "@/app/db";
import { interviews, jobApplications, jobs, notifications } from
"@/app/db/schema";
import { requireUser } from "@/lib/current-user";

export async function POST(req: Request, { params }: { params: {
applicationId: string } }) {
 const auth = await requireUser("employer");
 if ("error" in auth) return auth.error;

 const body = await req.json();
 const decision = body.decision as "accepted" | "rejected" |
"interview";

 const [application] = await db
   .select()
   .from(jobApplications)
   .where(eq(jobApplications.id, params.applicationId))
   .limit(1);

 if (!application) return NextResponse.json({ error:
"Application not found" }, { status: 404 });
 if (application.employerId !== auth.user.userId) {
   return NextResponse.json({ error: "Forbidden" }, { status:
403 });
 }

 const [job] = await db.select().from(jobs).where(eq(jobs.id,
application.jobId)).limit(1);

 await db
   .update(jobApplications)
```

```
    .set({ status: decision, updatedAt: new Date() })
    .where(eq(jobApplications.id, application.id));

  if (decision === "interview") {
    await db.insert(interviews).values({
      applicationId: application.id,
      employerId: application.employerId,
      jobSeekerId: application.jobSeekerId,
      mode: body.mode ?? "remote",
      scheduledAt: new Date(body.scheduledAt),
      location: body.location ?? "",
      notes: body.notes ?? "",
    });
  }

  const titleMap = {
    accepted: `Application accepted for ${job?.title ?? "your
application"}`,
    rejected: `Application update for ${job?.title ?? "your
application"}`,
    interview: `Interview scheduled for ${job?.title ?? "your
application"}`,
  };

  const descriptionMap = {
    accepted: body.message || "The employer accepted your
application.",
    rejected: body.message || "The employer did not move forward
with your application.",
    interview: body.message || `Interview scheduled on
${body.scheduledAt}.`,
  };
```

```
 await db.insert(notifications).values({
   recipientId: application.jobSeekerId,
   actorId: auth.user.userId,
   type: decision === "interview" ? "interview" : "application",
   title: titleMap[decision],
   description: descriptionMap[decision],
   link: decision === "interview" ? "/interviews" : "/applied",
   meta: {
     applicationId: application.id,
     jobId: application.jobId,
     decision,
     scheduledAt: body.scheduledAt ?? null,
     location: body.location ?? null,
   },
 });

 return NextResponse.json({ ok: true });
}

import { NextResponse } from "next/server";
import { desc, eq } from "drizzle-orm";
import { db } from "@/app/db";
import { employerProfiles, jobApplications, jobs } from
"@/app/db/schema";
import { requireUser } from "@/lib/current-user";

function employmentTypeLabel(value: string | null) {
 if (value === "full_time") return "Full time";
 if (value === "part_time") return "Part time";
 if (value === "contract") return "Contract";
 if (value === "freelance") return "Freelance";
```

```
 if (value === "internship") return "Internship";
 return "";
}

function arrangementLabel(value: string | null) {
 if (value === "on_site") return "On-site";
 if (value === "remote") return "Remote";
 if (value === "hybrid") return "Hybrid";
 return "";
}

function experienceLabel(value: string | null) {
 if (value === "entry") return "Entry level";
 if (value === "mid") return "Mid level";
 if (value === "senior") return "Senior";
 if (value === "lead") return "Lead / Manager";
 if (value === "executive") return "Executive";
 return "";
}

export async function GET() {
 const auth = await requireUser("job_seeker");
 if ("error" in auth) return auth.error;

 const rows = await db
   .select({
     id: jobApplications.id,
     status: jobApplications.status,
     appliedAt: jobApplications.appliedAt,
     updatedAt: jobApplications.updatedAt,
     jobTitle: jobs.title,
     company: employerProfiles.companyName,
```

```
      salaryMin: jobs.salaryMin,
      salaryMax: jobs.salaryMax,
      employmentType: jobs.employmentType,
      arrangement: jobs.arrangement,
      experienceLevel: jobs.experienceLevel,
    })
    .from(jobApplications)
    .innerJoin(jobs, eq(jobs.id, jobApplications.jobId))
    .innerJoin(employerProfiles, eq(employerProfiles.userId,
jobs.employerId))
    .where(eq(jobApplications.jobSeekerId, auth.user.userId))
    .orderBy(desc(jobApplications.updatedAt));

  const archived = rows.filter(
    (row) => row.status === "withdrawn" || row.status ===
"rejected"
  );

  return NextResponse.json(
    archived.map((row) => ({
      ...row,
      employmentTypeLabel:
employmentTypeLabel(row.employmentType),
      arrangementLabel: arrangementLabel(row.arrangement),
      experienceLabel: experienceLabel(row.experienceLevel),
    }))
  );
}

import { NextResponse } from "next/server";
import { eq } from "drizzle-orm";
import { z } from "zod";
```

```
import { db } from "@/app/db";
import { employerProfiles, users } from "@/app/db/schema";
import { getCurrentUser } from "@/lib/auth";
import { saveProfileImage } from "@/lib/save-profile-image";
import { saveUpload } from "@/lib/save-upload";

const schema = z.object({
 companyDescription: z.string().optional(),
 companyName: z.string().min(1),
 industry: z.string().optional(),
 companySize: z.string().optional(),
 currentAddress: z.string().optional(),
 foundedYear: z.string().optional(),
 country: z.string().optional(),
 contact: z.string().optional(),
 websiteLink: z.string().optional(),
});

export async function GET() {
 try {
   const me = await getCurrentUser("auth");

   if (!me) {
     return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
   }

   const [profile] = await db
     .select()
     .from(employerProfiles)
     .where(eq(employerProfiles.userId, me.id))
     .limit(1);
```

```
    return NextResponse.json(profile ?? null);
  } catch (error) {
    console.error("EMPLOYER PROFILE GET ERROR:", error);

    return NextResponse.json(
      {
        error: error instanceof Error ? error.message : "Internal
server error",
      },
      { status: 500 }
    );
  }
}

export async function POST(req: Request) {
  try {
    const me = await getCurrentUser("auth");

    if (!me) {
      return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
    }

    const formData = await req.formData();
    const image = formData.get("profileImage");
    const companyFile = formData.get("companyFile");

    const parsed = schema.safeParse({
      companyDescription:
String(formData.get("companyDescription") || ""),
      companyName: String(formData.get("companyName") || ""),
```

```
      industry: String(formData.get("industry") || ""),
      companySize: String(formData.get("companySize") || ""),
      currentAddress: String(formData.get("currentAddress") ||
""),
      foundedYear: String(formData.get("foundedYear") || ""),
      country: String(formData.get("country") || ""),
      contact: String(formData.get("contact") || ""),
      websiteLink: String(formData.get("websiteLink") || ""),
    });

    if (!parsed.success) {
      return NextResponse.json({ error: "Invalid profile data" },
{ status: 400 });
    }

    const profileImage =
      image instanceof File && image.size > 0
        ? await saveProfileImage(image)
        : null;

    const companyUpload =
      companyFile instanceof File && companyFile.size > 0
        ? await saveUpload(
            companyFile,
            "company-files",
            [
              "application/pdf",
              "application/msword",

"application/vnd.openxmlformats-officedocument.wordprocessingml.
document",
            ],
```

```
          10 * 1024 * 1024
        )
      : null;

  const [existing] = await db
    .select()
    .from(employerProfiles)
    .where(eq(employerProfiles.userId, me.id))
    .limit(1);

  const payload = {
    ...parsed.data,
    ...(profileImage ? { profileImage } : {}),
    ...(companyUpload
      ? {
          companyFileName: companyUpload.fileName,
          companyFileUrl: companyUpload.url,
        }
      : {}),
  };

  if (existing) {
    await db
      .update(employerProfiles)
      .set(payload)
      .where(eq(employerProfiles.userId, me.id));
  } else {
    await db.insert(employerProfiles).values({
      userId: me.id,
      profileImage,
      companyFileName: companyUpload?.fileName ?? null,
      companyFileUrl: companyUpload?.url ?? null,
```

```
      ...parsed.data,
    });
  }

  await db
    .update(users)
    .set({
      onboardingCompleted: true,
      updatedAt: new Date(),
    })
    .where(eq(users.id, me.id));

  const [profile] = await db
    .select()
    .from(employerProfiles)
    .where(eq(employerProfiles.userId, me.id))
    .limit(1);

  return NextResponse.json({
    success: true,
    next: "/dashboard",
    profile,
    profileImageUrl: profile?.profileImage ?? null,
    companyFileName: profile?.companyFileName ?? null,
    companyFileUrl: profile?.companyFileUrl ?? null,
  });
} catch (error) {
  console.error("EMPLOYER PROFILE ERROR:", error);

  return NextResponse.json(
    {
```

```
      error: error instanceof Error ? error.message : "Internal
server error",
     },
     { status: 500 }
   );
 }
}

import { NextResponse } from "next/server";
import { eq } from "drizzle-orm";
import { z } from "zod";
import { db } from "@/app/db";
import { getCurrentUser } from "@/lib/auth";
import { jobSeekerProfiles, users } from "@/app/db/schema";
import { saveProfileImage } from "@/lib/save-profile-image";
import { saveUpload } from "@/lib/save-upload";

const schema = z.object({
 description: z.string().optional(),
 firstName: z.string().min(1),
 lastName: z.string().min(1),
 contact: z.string().optional(),
 address: z.string().optional(),
 educationLevel: z.string().optional(),
 schoolUniversity: z.string().optional(),
 year: z.string().optional(),
});

export async function GET() {
 try {
   const me = await getCurrentUser("auth");
```

```
   if (!me) {
     return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
   }

   const [profile] = await db
     .select()
     .from(jobSeekerProfiles)
     .where(eq(jobSeekerProfiles.userId, me.id))
     .limit(1);

   return NextResponse.json(profile ?? null);
 } catch (error) {
   console.error("JOB SEEKER PROFILE GET ERROR:", error);

   return NextResponse.json(
     {
       error: error instanceof Error ? error.message : "Internal
server error",
     },
     { status: 500 }
   );
 }
}

export async function POST(req: Request) {
 try {
   const me = await getCurrentUser("auth");

   if (!me) {
     return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
```

```
  }

  const formData = await req.formData();
  const image = formData.get("profileImage");
  const cvFile = formData.get("cvFile");

  const parsed = schema.safeParse({
    description: String(formData.get("description") || ""),
    firstName: String(formData.get("firstName") || ""),
    lastName: String(formData.get("lastName") || ""),
    contact: String(formData.get("contact") || ""),
    address: String(formData.get("address") || ""),
    educationLevel: String(formData.get("educationLevel") ||
""),
    schoolUniversity: String(formData.get("schoolUniversity")
|| ""),
    year: String(formData.get("year") || ""),
  });

  if (!parsed.success) {
    return NextResponse.json({ error: "Invalid profile data" },
{ status: 400 });
  }

  const profileImage =
    image instanceof File && image.size > 0
      ? await saveProfileImage(image)
      : null;

  const cvUpload =
    cvFile instanceof File && cvFile.size > 0
      ? await saveUpload(
```

```
          cvFile,
          "cvs",
          [
            "application/pdf",
            "application/msword",

"application/vnd.openxmlformats-officedocument.wordprocessingml.
document",
          ],
          10 * 1024 * 1024
        )
      : null;

  const [existing] = await db
    .select()
    .from(jobSeekerProfiles)
    .where(eq(jobSeekerProfiles.userId, me.id))
    .limit(1);

  const payload = {
    ...parsed.data,
    ...(profileImage ? { profileImage } : {}),
    ...(cvUpload
      ? {
          cvFileName: cvUpload.fileName,
          cvUrl: cvUpload.url,
        }
      : {}),
  };

  if (existing) {
    await db
```

```
      .update(jobSeekerProfiles)
      .set(payload)
      .where(eq(jobSeekerProfiles.userId, me.id));
  } else {
    await db.insert(jobSeekerProfiles).values({
      userId: me.id,
      profileImage,
      cvFileName: cvUpload?.fileName ?? null,
      cvUrl: cvUpload?.url ?? null,
      ...parsed.data,
    });
  }

  await db
    .update(users)
    .set({
      onboardingCompleted: true,
      updatedAt: new Date(),
    })
    .where(eq(users.id, me.id));

  const [profile] = await db
    .select()
    .from(jobSeekerProfiles)
    .where(eq(jobSeekerProfiles.userId, me.id))
    .limit(1);

  return NextResponse.json({
    success: true,
    next: "/home_page",
    profile,
    profileImageUrl: profile?.profileImage ?? null,
```

```
      cvFileName: profile?.cvFileName ?? null,
      cvUrl: profile?.cvUrl ?? null,
    });
  } catch (error) {
    console.error("JOB SEEKER PROFILE ERROR:", error);

    return NextResponse.json(
      {
        error: error instanceof Error ? error.message : "Internal
server error",
      },
      { status: 500 }
    );
  }
}

import { NextRequest, NextResponse } from "next/server";
import { and, desc, eq, isNull } from "drizzle-orm";
import { db } from "@/app/db";
import {
  conversations,
  employerProfiles,
  jobSeekerProfiles,
} from "@/app/db/schema";
import { getCurrentUser } from "@/lib/auth";

export async function GET() {
  try {
    const me = await getCurrentUser("auth");

    if (!me) {
```

```
    return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
  }

  const rows = await db
    .select({
      id: conversations.id,
      employerId: conversations.employerId,
      jobSeekerId: conversations.jobSeekerId,
      jobId: conversations.jobId,
      lastMessageAt: conversations.lastMessageAt,
      employerName: employerProfiles.companyName,
      employerImage: employerProfiles.profileImage,
      seekerFirstName: jobSeekerProfiles.firstName,
      seekerLastName: jobSeekerProfiles.lastName,
      seekerImage: jobSeekerProfiles.profileImage,
    })
    .from(conversations)
    .innerJoin(
      employerProfiles,
      eq(employerProfiles.userId, conversations.employerId)
    )
    .innerJoin(
      jobSeekerProfiles,
      eq(jobSeekerProfiles.userId, conversations.jobSeekerId)
    )
    .where(
      me.role === "employer"
        ? eq(conversations.employerId, me.id)
        : eq(conversations.jobSeekerId, me.id)
    )
    .orderBy(desc(conversations.lastMessageAt));
```

```
    return NextResponse.json(rows);
  } catch (error) {
    console.error("GET CONVERSATIONS ERROR:", error);
    return NextResponse.json(
      {
        error: error instanceof Error ? error.message : "Internal
server error",
      },
      { status: 500 }
    );
  }
}

export async function POST(req: NextRequest) {
  try {
    const me = await getCurrentUser("auth");

    if (!me) {
      return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
    }

    const body = await req.json();

    const employerId = me.role === "employer" ? me.id :
body.employerId;
    const jobSeekerId = me.role === "job_seeker" ? me.id :
body.jobSeekerId;
    const jobId = body.jobId ?? null;

    if (!employerId || !jobSeekerId) {
```

```
    return NextResponse.json(
      { error: "Missing employer or job seeker" },
      { status: 400 }
    );
  }

  const existingRows = await db
    .select()
    .from(conversations)
    .where(
      and(
        eq(conversations.employerId, employerId),
        eq(conversations.jobSeekerId, jobSeekerId),
        jobId ? eq(conversations.jobId, jobId) :
isNull(conversations.jobId)
      )
    )
    .limit(1);

  const existing = existingRows[0];
  if (existing) {
    return NextResponse.json(existing);
  }

  const [conversation] = await db
    .insert(conversations)
    .values({
      employerId,
      jobSeekerId,
      jobId,
      lastMessageAt: new Date(),
    })
```

```
      .returning();

    return NextResponse.json(conversation, { status: 201 });
  } catch (error) {
    console.error("CREATE CONVERSATION ERROR:", error);
    return NextResponse.json(
      {
        error: error instanceof Error ? error.message : "Internal
server error",
      },
      { status: 500 }
    );
  }
}

import { NextResponse } from "next/server";
import { and, asc, eq, notInArray } from "drizzle-orm";
import { db } from "@/app/db";
import {
  conversations,
  messageDeletes,
  messages,
  notifications,
} from "@/app/db/schema";
import { getCurrentUser } from "@/lib/auth";
import { saveUpload } from "@/lib/save-upload";

async function loadConversation(conversationId: string) {
  const [conversation] = await db
    .select()
    .from(conversations)
    .where(eq(conversations.id, conversationId))
```

```
   .limit(1);

 return conversation;
}

export async function GET(
 _: Request,
 context: { params: Promise<{ conversationId: string }> }
) {
 try {
   const me = await getCurrentUser("auth");

   if (!me) {
     return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
   }

   const { conversationId } = await context.params;
   const conversation = await loadConversation(conversationId);

   if (!conversation) {
     return NextResponse.json({ error: "Conversation not found"
}, { status: 404 });
   }

   const allowed =
     conversation.employerId === me.id ||
conversation.jobSeekerId === me.id;

   if (!allowed) {
     return NextResponse.json({ error: "Forbidden" }, { status:
403 });
```

```
    }

    const deleted = await db
      .select({ messageId: messageDeletes.messageId })
      .from(messageDeletes)
      .where(eq(messageDeletes.userId, me.id));

    const deletedIds = deleted.map((x) => x.messageId);

    const rows = await db
      .select()
      .from(messages)
      .where(
        deletedIds.length
          ? and(
              eq(messages.conversationId, conversationId),
              notInArray(messages.id, deletedIds)
            )
          : eq(messages.conversationId, conversationId)
      )
      .orderBy(asc(messages.createdAt));

    return NextResponse.json(rows);
  } catch (error) {
    console.error("GET MESSAGES ERROR:", error);
    return NextResponse.json(
      {
        error: error instanceof Error ? error.message : "Internal
server error",
      },
      { status: 500 }
    );
```

```
 }
}

export async function POST(
 req: Request,
 context: { params: Promise<{ conversationId: string }> }
) {
 try {
   const me = await getCurrentUser("auth");

   if (!me) {
     return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
   }

   const { conversationId } = await context.params;
   const conversation = await loadConversation(conversationId);

   if (!conversation) {
     return NextResponse.json({ error: "Conversation not found"
}, { status: 404 });
   }

   const allowed =
     conversation.employerId === me.id ||
conversation.jobSeekerId === me.id;

   if (!allowed) {
     return NextResponse.json({ error: "Forbidden" }, { status:
403 });
   }
```

```
  const form = await req.formData();
  const text = String(form.get("text") || "");
  const attachment = (form.get("attachment") as File | null) ??
null;

  if (!text && !attachment) {
    return NextResponse.json({ error: "Message cannot be empty"
}, { status: 400 });
  }

  const upload = await saveUpload(attachment, "chat",
undefined, 15 * 1024 * 1024);

  const [message] = await db
    .insert(messages)
    .values({
      conversationId,
      senderId: me.id,
      text: text || null,
      attachmentUrl: upload?.url ?? null,
      attachmentName: upload?.fileName ?? null,
      attachmentType: upload?.mimeType ?? null,
    })
    .returning();

  await db
    .update(conversations)
    .set({ lastMessageAt: new Date() })
    .where(eq(conversations.id, conversationId));

  const recipientId =
    conversation.employerId === me.id
```

```
      ? conversation.jobSeekerId
      : conversation.employerId;

  await db.insert(notifications).values({
    recipientId,
    actorId: me.id,
    type: "message",
    title: "New message",
    description: text
      ? text.slice(0, 120)
      : `Sent an attachment: ${upload?.fileName ?? "file"}`,
    link: me.role === "employer" ? "/message" :
"/employer_message",
    meta: { conversationId, messageId: message.id },
  });

  return NextResponse.json(message, { status: 201 });
 } catch (error) {
  console.error("SEND MESSAGE ERROR:", error);
  return NextResponse.json(
    {
      error: error instanceof Error ? error.message : "Internal
server error",
    },
    { status: 500 }
  );
 }
}

import { NextResponse } from "next/server";
import { desc, eq } from "drizzle-orm";
import { db } from "@/app/db";
```

```
import { employerProfiles, interviews, jobApplications, jobs }
from "@/app/db/schema";
import { requireUser } from "@/lib/current-user";

function employmentTypeLabel(value: string | null) {
 if (value === "full_time") return "Full time";
 if (value === "part_time") return "Part time";
 if (value === "contract") return "Contract";
 if (value === "freelance") return "Freelance";
 if (value === "internship") return "Internship";
 return "";
}

function experienceLabel(value: string | null) {
 if (value === "entry") return "Entry level";
 if (value === "mid") return "Mid level";
 if (value === "senior") return "Senior";
 if (value === "lead") return "Lead / Manager";
 if (value === "executive") return "Executive";
 return "";
}

export async function GET() {
 const auth = await requireUser();
 if ("error" in auth) return auth.error;

 if (auth.user.role === "job_seeker") {
   const rows = await db
     .select({
       id: interviews.id,
       scheduledAt: interviews.scheduledAt,
       location: interviews.location,
```

```
      mode: interviews.mode,
      jobTitle: jobs.title,
      company: employerProfiles.companyName,
      salaryMin: jobs.salaryMin,
      salaryMax: jobs.salaryMax,
      employmentType: jobs.employmentType,
      experienceLevel: jobs.experienceLevel,
    })
    .from(interviews)
    .innerJoin(jobApplications, eq(jobApplications.id,
interviews.applicationId))
    .innerJoin(jobs, eq(jobs.id, jobApplications.jobId))
    .innerJoin(employerProfiles, eq(employerProfiles.userId,
jobs.employerId))
    .where(eq(interviews.jobSeekerId, auth.user.userId))
    .orderBy(desc(interviews.scheduledAt));

  return NextResponse.json(
    rows.map((row) => ({
      ...row,
      employmentTypeLabel:
employmentTypeLabel(row.employmentType),
      experienceLabel: experienceLabel(row.experienceLevel),
    }))
  );
}

const rows = await db
  .select({
    id: interviews.id,
    scheduledAt: interviews.scheduledAt,
    location: interviews.location,
```

```
     mode: interviews.mode,
     applicationId: interviews.applicationId,
     jobTitle: jobs.title,
     firstName: jobApplications.jobSeekerId,
   })
   .from(interviews)
   .innerJoin(jobApplications, eq(jobApplications.id,
interviews.applicationId))
   .innerJoin(jobs, eq(jobs.id, jobApplications.jobId))
   .where(eq(interviews.employerId, auth.user.userId))
   .orderBy(desc(interviews.scheduledAt));

 return NextResponse.json(rows);
}

import { NextRequest, NextResponse } from "next/server";
import { and, desc, eq, ilike, or } from "drizzle-orm";
import { db } from "@/app/db";
import { employerProfiles, jobs } from "@/app/db/schema";
import { getCurrentUser } from "@/lib/auth";

const arrangementMap: Record<string, "on_site" | "remote" |
"hybrid"> = {
 "On-site": "on_site",
 Remote: "remote",
 Hybrid: "hybrid",
};

const employmentTypeMap: Record<
 string,
 "full_time" | "part_time" | "contract" | "freelance" |
"internship"
```

```
> = {
 "Full-time": "full_time",
 "Part-time": "part_time",
 Contract: "contract",
 Freelance: "freelance",
 Internship: "internship",
};

const experienceMap: Record<
 string,
 "entry" | "mid" | "senior" | "lead" | "executive"
> = {
 "Entry Level": "entry",
 "Mid Level": "mid",
 Senior: "senior",
 "Lead / Manager": "lead",
 Executive: "executive",
};

export async function GET(req: NextRequest) {
 try {
   const me = await getCurrentUser("auth");
   const q = req.nextUrl.searchParams.get("q") ?? "";
   const jobType = req.nextUrl.searchParams.get("jobType") ??
"";
   const arrangement =
req.nextUrl.searchParams.get("arrangement") ?? "";
   const experience = req.nextUrl.searchParams.get("experience")
?? "";
   const mine = req.nextUrl.searchParams.get("mine") === "1";

   const filters: any[] = [];
```

```
  if (mine && me?.role === "employer") {
    filters.push(eq(jobs.employerId, me.id));
  } else {
    filters.push(eq(jobs.status, "active"));
  }

  if (q) {
    filters.push(
      or(
        ilike(jobs.title, `%${q}%`),
        ilike(jobs.category, `%${q}%`),
        ilike(employerProfiles.companyName, `%${q}%`)
      )!
    );
  }

  if (jobType && employmentTypeMap[jobType]) {
    filters.push(eq(jobs.employmentType,
employmentTypeMap[jobType]));
  }

  if (arrangement && arrangementMap[arrangement]) {
    filters.push(eq(jobs.arrangement,
arrangementMap[arrangement]));
  }

  if (experience && experienceMap[experience]) {
    filters.push(eq(jobs.experienceLevel,
experienceMap[experience]));
  }
```

```
  const rows = await db
    .select({
      id: jobs.id,
      employerId: jobs.employerId,
      title: jobs.title,
      category: jobs.category,
      location: jobs.location,
      arrangement: jobs.arrangement,
      type: jobs.employmentType,
      experience: jobs.experienceLevel,
      description: jobs.description,
      requirements: jobs.requirements,
      salaryMin: jobs.salaryMin,
      salaryMax: jobs.salaryMax,
      deadline: jobs.applicationDeadline,
      postedAt: jobs.createdAt,
      contactEmail: jobs.contactEmail,
      status: jobs.status,
      company: employerProfiles.companyName,
      profileImage: employerProfiles.profileImage,
    })
    .from(jobs)
    .innerJoin(employerProfiles, eq(employerProfiles.userId,
jobs.employerId))
    .where(and(...filters))
    .orderBy(desc(jobs.createdAt));

  return NextResponse.json(rows);
 } catch (error) {
  console.error("GET JOBS ERROR:", error);

  return NextResponse.json(
```

```
      { error: error instanceof Error ? error.message : "Internal
server error" },
      { status: 500 }
    );
  }
}

export async function POST(req: Request) {
  try {
    const me = await getCurrentUser("auth");

    if (!me) {
      return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
    }

    if (me.role !== "employer") {
      return NextResponse.json({ error: "Forbidden" }, { status:
403 });
    }

    const body = await req.json();

    if (!body.title || !body.category || !body.location ||
!body.description) {
      return NextResponse.json(
        { error: "Title, category, location, and description are
required" },
        { status: 400 }
      );
    }
```

```
  const [job] = await db
    .insert(jobs)
    .values({
      employerId: me.id,
      title: body.title,
      category: body.category,
      location: body.location,
      arrangement: body.arrangement ?? "on_site",
      employmentType: body.employmentType ?? "full_time",
      experienceLevel: body.experienceLevel ?? "entry",
      salaryMin: body.salaryMin ? Number(body.salaryMin) :
null,
      salaryMax: body.salaryMax ? Number(body.salaryMax) :
null,
      description: body.description,
      requirements: body.requirements ?? "",
      applicationDeadline: body.applicationDeadline
        ? new Date(body.applicationDeadline)
        : null,
      applicationPlatform: body.applicationPlatform ??
"internal",
      externalApplyLink: body.externalApplyLink ?? null,
      contactEmail: body.contactEmail ?? null,
      status: body.status ?? "draft",
      updatedAt: new Date(),
    })
    .returning();

  return NextResponse.json(job, { status: 201 });
 } catch (error) {
  console.error("POST JOB ERROR:", error);
```

```
   return NextResponse.json(
     { error: error instanceof Error ? error.message : "Internal
server error" },
     { status: 500 }
   );
 }
}

import { NextResponse } from "next/server";
import { eq } from "drizzle-orm";
import { db } from "@/app/db";
import { employerProfiles, jobs } from "@/app/db/schema";
import { requireUser } from "@/lib/current-user";

export async function GET(_: Request, { params }: { params: {
jobId: string } }) {
 const [job] = await db
   .select({
     id: jobs.id,
     employerId: jobs.employerId,
     title: jobs.title,
     category: jobs.category,
     location: jobs.location,
     arrangement: jobs.arrangement,
     type: jobs.employmentType,
     experience: jobs.experienceLevel,
     description: jobs.description,
     requirements: jobs.requirements,
     salaryMin: jobs.salaryMin,
     salaryMax: jobs.salaryMax,
     deadline: jobs.applicationDeadline,
     contactEmail: jobs.contactEmail,
```

```
      applicationPlatform: jobs.applicationPlatform,
      externalApplyLink: jobs.externalApplyLink,
      status: jobs.status,
      createdAt: jobs.createdAt,
      company: employerProfiles.companyName,
      companyDescription: employerProfiles.companyDescription,
      companyImage: employerProfiles.profileImage,
      companyWebsite: employerProfiles.websiteLink,
    })
    .from(jobs)
    .innerJoin(employerProfiles, eq(employerProfiles.userId,
jobs.employerId))
    .where(eq(jobs.id, params.jobId))
    .limit(1);

  if (!job) {
    return NextResponse.json({ error: "Job not found" }, {
status: 404 });
  }

  return NextResponse.json(job);
}

export async function PATCH(req: Request, { params }: { params:
{ jobId: string } }) {
  const auth = await requireUser("employer");
  if ("error" in auth) return auth.error;

  const body = await req.json();
```

```
 const [existing] = await
db.select().from(jobs).where(eq(jobs.id,
params.jobId)).limit(1);
 if (!existing) return NextResponse.json({ error: "Job not
found" }, { status: 404 });
 if (existing.employerId !== auth.user.userId) {
   return NextResponse.json({ error: "Forbidden" }, { status:
403 });
 }

 const [updated] = await db
   .update(jobs)
   .set({
     title: body.title ?? existing.title,
     category: body.category ?? existing.category,
     location: body.location ?? existing.location,
     arrangement: body.arrangement ?? existing.arrangement,
     employmentType: body.employmentType ??
existing.employmentType,
     experienceLevel: body.experienceLevel ??
existing.experienceLevel,
     salaryMin: body.salaryMin ?? existing.salaryMin,
     salaryMax: body.salaryMax ?? existing.salaryMax,
     description: body.description ?? existing.description,
     requirements: body.requirements ?? existing.requirements,
     applicationDeadline: body.applicationDeadline ? new
Date(body.applicationDeadline) : existing.applicationDeadline,
     applicationPlatform: body.applicationPlatform ??
existing.applicationPlatform,
     externalApplyLink: body.externalApplyLink ??
existing.externalApplyLink,
     contactEmail: body.contactEmail ?? existing.contactEmail,
```

```
      status: body.status ?? existing.status,
      updatedAt: new Date(),
    })
    .where(eq(jobs.id, params.jobId))
    .returning();

 return NextResponse.json(updated);
}

import { NextResponse } from "next/server";
import { eq } from "drizzle-orm";
import { db } from "@/app/db";
import { jobApplications, jobSeekerProfiles, jobs, notifications
} from "@/app/db/schema";
import { requireUser } from "@/lib/current-user";
import { saveUpload } from "@/lib/save-upload";

export async function POST(req: Request, { params }: { params: {
jobId: string } }) {
 const auth = await requireUser("job_seeker");
 if ("error" in auth) return auth.error;

 const [job] = await db.select().from(jobs).where(eq(jobs.id,
params.jobId)).limit(1);
 if (!job) return NextResponse.json({ error: "Job not found" },
{ status: 404 });

 const form = await req.formData();
 const coverLetter = String(form.get("coverLetter") || "");
 const cvFile = (form.get("cvFile") as File | null) ?? null;

 let cvUrl: string | null = null;
```

```
let cvFileName: string | null = null;

if (cvFile) {
  const upload = await saveUpload(
    cvFile,
    "applications",
    [
      "application/pdf",
      "application/msword",

"application/vnd.openxmlformats-officedocument.wordprocessingml.
document",
    ],
    10 * 1024 * 1024
  );
  cvUrl = upload?.url ?? null;
  cvFileName = upload?.fileName ?? null;
}

const [profile] = await db
  .select()
  .from(jobSeekerProfiles)
  .where(eq(jobSeekerProfiles.userId, auth.user.userId))
  .limit(1);

const [application] = await db
  .insert(jobApplications)
  .values({
    jobId: job.id,
    employerId: job.employerId,
    jobSeekerId: auth.user.userId,
    coverLetter,
```

```
      cvUrl,
      cvFileName,
    })
    .onConflictDoNothing()
    .returning();

  if (!application) {
    return NextResponse.json({ error: "Already applied to this
job" }, { status: 409 });
  }

  await db.insert(notifications).values({
    recipientId: job.employerId,
    actorId: auth.user.userId,
    type: "application",
    title: `${profile?.firstName ?? "A candidate"} applied for
${job.title}`,
    description: "A new application is ready for review.",
    link: "/dashboard",
    meta: { jobId: job.id, applicationId: application.id },
  });

  return NextResponse.json(application, { status: 201 });
}

import { NextRequest, NextResponse } from "next/server";
import { eq } from "drizzle-orm";
import { db } from "@/app/db";
import { messageDeletes, messages } from "@/app/db/schema";
import { getCurrentUser } from "@/lib/auth";

const EDIT_WINDOW_MS = 15 * 60 * 1000;
```

```
export async function PATCH(
 req: NextRequest,
 context: { params: Promise<{ messageId: string }> }
) {
 try {
   const me = await getCurrentUser("auth");

   if (!me) {
     return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
   }

   const { messageId } = await context.params;
   const body = await req.json();

   const [message] = await db
     .select()
     .from(messages)
     .where(eq(messages.id, messageId))
     .limit(1);

   if (!message) {
     return NextResponse.json({ error: "Message not found" }, {
status: 404 });
   }

   if (message.senderId !== me.id) {
     return NextResponse.json({ error: "Only sender can edit" },
{ status: 403 });
   }
```

```
   const tooLate =
     Date.now() - new Date(message.createdAt).getTime() >
EDIT_WINDOW_MS;

   if (tooLate) {
     return NextResponse.json({ error: "Edit window expired" },
{ status: 400 });
   }

   const [updated] = await db
     .update(messages)
     .set({
       text: body.text,
       editedAt: new Date(),
     })
     .where(eq(messages.id, messageId))
     .returning();

   return NextResponse.json(updated);
 } catch (error) {
   console.error("EDIT MESSAGE ERROR:", error);
   return NextResponse.json(
     {
       error: error instanceof Error ? error.message : "Internal
server error",
     },
     { status: 500 }
   );
 }
}

export async function DELETE(
```

```
 _: NextRequest,
 context: { params: Promise<{ messageId: string }> }
) {
 try {
   const me = await getCurrentUser("auth");

   if (!me) {
     return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
   }

   const { messageId } = await context.params;

   const [message] = await db
     .select()
     .from(messages)
     .where(eq(messages.id, messageId))
     .limit(1);

   if (!message) {
     return NextResponse.json({ error: "Message not found" }, {
status: 404 });
   }

   await db
     .insert(messageDeletes)
     .values({
       messageId: message.id,
       userId: me.id,
     })
     .onConflictDoNothing();
```

```
   return NextResponse.json({ ok: true });
 } catch (error) {
   console.error("DELETE MESSAGE ERROR:", error);
   return NextResponse.json(
     {
       error: error instanceof Error ? error.message : "Internal
server error",
     },
     { status: 500 }
   );
 }
}

import { NextRequest, NextResponse } from "next/server";
import { desc, eq } from "drizzle-orm";
import { db } from "@/app/db";
import { jobApplications, notifications } from
"@/app/db/schema";
import { getCurrentUser } from "@/lib/auth";

export async function GET() {
 try {
   const me = await getCurrentUser("auth");

   if (!me) {
     return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
   }

   const rows = await db
     .select()
     .from(notifications)
```

```
      .where(eq(notifications.recipientId, me.id))
      .orderBy(desc(notifications.createdAt));

    return NextResponse.json(rows);
  } catch (error) {
    console.error("GET NOTIFICATIONS ERROR:", error);

    return NextResponse.json(
      {
        error: error instanceof Error ? error.message : "Internal
server error",
      },
      { status: 500 }
    );
  }
}

export async function POST(req: NextRequest) {
  try {
    const me = await getCurrentUser("auth");

    if (!me) {
      return NextResponse.json({ error: "Unauthorized" }, {
status: 401 });
    }

    if (me.role !== "employer") {
      return NextResponse.json({ error: "Forbidden" }, { status:
403 });
    }

    const body = await req.json();
```

```
  if (!body.applicationId || !body.title || !body.description)
{
    return NextResponse.json(
      { error: "Missing applicationId, title, or description"
},
      { status: 400 }
    );
  }

  const [application] = await db
    .select()
    .from(jobApplications)
    .where(eq(jobApplications.id, body.applicationId))
    .limit(1);

  if (!application) {
    return NextResponse.json({ error: "Application not found"
}, { status: 404 });
  }

  if (application.employerId !== me.id) {
    return NextResponse.json({ error: "Forbidden" }, { status:
403 });
  }

  const [notif] = await db
    .insert(notifications)
    .values({
      recipientId: application.jobSeekerId,
      actorId: me.id,
      type: body.type ?? "system",
```

```
      title: body.title,
      description: body.description,
      link: body.link ?? "/notification",
      meta: {
        applicationId: application.id,
        jobId: application.jobId,
      },
    })
    .returning();

  return NextResponse.json(notif, { status: 201 });
 } catch (error) {
  console.error("CREATE NOTIFICATION ERROR:", error);

  return NextResponse.json(
    {
      error: error instanceof Error ? error.message : "Internal
server error",
    },
    { status: 500 }
  );
 }
}

import { NextRequest, NextResponse } from "next/server";
import { and, desc, eq } from "drizzle-orm";
import { db } from "@/app/db";
import { employerProfiles, jobs, savedJobs } from
"@/app/db/schema";
import { requireUser } from "@/lib/current-user";

export async function GET() {
```

```
 const auth = await requireUser("job_seeker");
 if ("error" in auth) return auth.error;

 const rows = await db
   .select({
     id: savedJobs.id,
     jobId: jobs.id,
     title: jobs.title,
     company: employerProfiles.companyName,
     description: jobs.description,
     salaryMin: jobs.salaryMin,
     salaryMax: jobs.salaryMax,
     type: jobs.employmentType,
     arrangement: jobs.arrangement,
     experience: jobs.experienceLevel,
     createdAt: savedJobs.createdAt,
   })
   .from(savedJobs)
   .innerJoin(jobs, eq(jobs.id, savedJobs.jobId))
   .innerJoin(employerProfiles, eq(employerProfiles.userId,
jobs.employerId))
   .where(eq(savedJobs.jobSeekerId, auth.user.userId))
   .orderBy(desc(savedJobs.createdAt));

 return NextResponse.json(rows);
}

export async function POST(req: NextRequest) {
 const auth = await requireUser("job_seeker");
 if ("error" in auth) return auth.error;

 const body = await req.json();
```

```
 const [saved] = await db
   .insert(savedJobs)
   .values({
     jobId: body.jobId,
     jobSeekerId: auth.user.userId,
   })
   .onConflictDoNothing()
   .returning();

 return NextResponse.json(saved ?? { ok: true });
}

export async function DELETE(req: NextRequest) {
 const auth = await requireUser("job_seeker");
 if ("error" in auth) return auth.error;

 const body = await req.json();

 await db
   .delete(savedJobs)
   .where(and(eq(savedJobs.jobId, body.jobId),
eq(savedJobs.jobSeekerId, auth.user.userId)));

 return NextResponse.json({ ok: true });
}

export type Area = {
 width: number;
 height: number;
 x: number;
 y: number;
```

```
};

function createImage(url: string): Promise<HTMLImageElement> {
 return new Promise((resolve, reject) => {
   const image = new Image();
   image.addEventListener("load", () => resolve(image));
   image.addEventListener("error", (error) => reject(error));
   image.src = url;
 });
}

export async function getCroppedImg(
 imageSrc: string,
 pixelCrop: Area,
 fileName = "cropped-image.jpg"
): Promise<File> {
 const image = await createImage(imageSrc);
 const canvas = document.createElement("canvas");
 const ctx = canvas.getContext("2d");

 if (!ctx) {
   throw new Error("Could not create canvas context");
 }

 canvas.width = pixelCrop.width;
 canvas.height = pixelCrop.height;

 ctx.drawImage(
   image,
   pixelCrop.x,
   pixelCrop.y,
   pixelCrop.width,
```

```
    pixelCrop.height,
    0,
    0,
    pixelCrop.width,
    pixelCrop.height
  );

  return new Promise((resolve, reject) => {
    canvas.toBlob(
      (blob) => {
        if (!blob) {
          reject(new Error("Canvas is empty"));
          return;
        }

        resolve(
          new File([blob], fileName, {
            type: "image/jpeg",
          })
        );
      },
      "image/jpeg",
      0.92
    );
  });
}

import { cookies } from "next/headers";
import { and, eq, gt } from "drizzle-orm";
import { db } from "@/app/db";
import { sessions, users } from "@/app/db/schema";
```

```
export async function getCurrentUser() {
 const cookieStore = await cookies();
 const token = cookieStore.get("session_token")?.value;

 if (!token) return null;

 const [session] = await db
   .select({
     sessionId: sessions.id,
     userId: sessions.userId,
     expiresAt: sessions.expiresAt,
     id: users.id,
     email: users.email,
     role: users.role,
     isEmailVerified: users.isEmailVerified,
     onboardingCompleted: users.onboardingCompleted,
   })
   .from(sessions)
   .innerJoin(users, eq(users.id, sessions.userId))
   .where(and(eq(sessions.tokenHash, token),
gt(sessions.expiresAt, new Date())))
   .limit(1);

 return session ?? null;
}

export async function requireUser(requiredRole?: "employer" |
"job_seeker") {
 const user = await getCurrentUser();

 if (!user) {
   return {
```

```
     error: new Response(JSON.stringify({ error: "Unauthorized"
}), {
       status: 401,
       headers: { "Content-Type": "application/json" },
     }),
   };
 }

 if (requiredRole && user.role !== requiredRole) {
   return {
     error: new Response(JSON.stringify({ error: "Forbidden" }),
{
       status: 403,
       headers: { "Content-Type": "application/json" },
     }),
   };
 }

 return { user };
}

import { mkdir, writeFile } from "fs/promises";
import { randomUUID } from "crypto";
import path from "path";

const ALLOWED_TYPES = new Set(["image/jpeg", "image/png",
"image/webp"]);
const MAX_SIZE = 5 * 1024 * 1024;

export async function saveProfileImage(file: File | null) {
 if (!file || file.size === 0) {
   return null;
```

```
 }

 if (!ALLOWED_TYPES.has(file.type)) {
   throw new Error("Only JPG, PNG, and WEBP images are
allowed");
 }

 if (file.size > MAX_SIZE) {
   throw new Error("Image must be 5MB or smaller");
 }

 const buffer = Buffer.from(await file.arrayBuffer());

 const extension =
   file.type === "image/png"
     ? "png"
     : file.type === "image/webp"
       ? "webp"
       : "jpg";

 const fileName = `${randomUUID()}.${extension}`;
 const uploadDir = path.join(process.cwd(), "public", "uploads",
"profiles");

 await mkdir(uploadDir, { recursive: true });
 await writeFile(path.join(uploadDir, fileName), buffer);

 return `/uploads/profiles/${fileName}`;
}

import { mkdir, writeFile } from "fs/promises";
import { randomUUID } from "crypto";
```

```
import path from "path";

export async function saveUpload(
 file: File | null,
 folder: string,
 allowedTypes?: string[],
 maxSize = 10 * 1024 * 1024
) {
 if (!file || file.size === 0) return null;

 if (allowedTypes && !allowedTypes.includes(file.type)) {
   throw new Error("Unsupported file type");
 }

 if (file.size > maxSize) {
   throw new Error("File too large");
 }

 const buffer = Buffer.from(await file.arrayBuffer());
 const ext = file.name.includes(".") ?
file.name.split(".").pop() : "bin";
 const fileName = `${randomUUID()}.${ext}`;
 const uploadDir = path.join(process.cwd(), "public", "uploads",
folder);

 await mkdir(uploadDir, { recursive: true });
 await writeFile(path.join(uploadDir, fileName), buffer);

 return {
   url: `/uploads/${folder}/${fileName}`,
   fileName: file.name,
   mimeType: file.type,
```

```
  };

}
```